# Advanced Machine Learning Interview Questions

Random forest, boosting, SVM, clustering, deep learning, time series, recommenders and more, with answers.

Total questions: 160. Part 1 lists every question; Part 2 (the answer key, at the end) gives a full answer for each, using the same question number.

## Contents


| Section | Questions | Numbers |
|---|---|---|
| Advanced ML: Trees, Ensembles and SVM | 80 | Q1 to Q80 |
| Advanced ML: Unsupervised, Deep Learning, Time Series and Recommenders | 80 | Q81 to Q160 |


Tags: Concept = theory; Coding = write code/query/formula; Scenario = a company presents a case and asks you to solve it; Behavioural = soft-skill. Level: Easy / Medium / Hard.

# Part 1: Questions

## Advanced ML: Trees, Ensembles and SVM

**Q1.** *[Concept | Medium]*

> Explain Gini impurity and entropy for a decision tree split. How is information gain computed and do the two criteria differ in practice?

**Q2.** *[Concept | Medium]*

> Why do fully grown decision trees overfit, and what are pre-pruning and post-pruning (cost-complexity pruning)?

**Q3.** *[Concept | Medium]*

> How does a decision tree handle continuous features, categorical features and missing values? What are the practical limitations?

**Q4.** *[Concept | Medium]*

> Explain bagging and why it reduces variance but not bias. How does a random forest differ from plain bagging of trees?

**Q5.** *[Concept | Medium]*

> What is out-of-bag (OOB) error in a random forest and how is it computed? When is it unreliable?

**Q6.** *[Concept | Hard]*

> Compare impurity-based (Gini) feature importance, permutation importance and SHAP importance in random forests. What biases does each have?

**Q7.** *[Concept | Medium]*

> Which random forest hyperparameters matter most and how do they affect bias and variance?

**Q8.** *[Concept | Medium]*

> Explain gradient boosting step by step. Why is it called "gradient" boosting?

**Q9.** *[Concept | Medium]*

> How does AdaBoost work, and how does it differ from gradient boosting?

**Q10.** *[Concept | Medium]*

> Contrast bagging and boosting in terms of bias, variance, parallelism and sensitivity to noise.

**Q11.** *[Concept | Hard]*

What makes XGBoost different from classic gradient boosting? Explain its regularized objective and split gain formula.

**Q12.** *[Concept | Hard]*

Describe the key XGBoost hyperparameters and a sensible tuning order.

**Q13.** *[Concept | Hard]*

How does LightGBM differ from XGBoost? Explain leaf-wise growth, GOSS and EFB.

**Q14.** *[Concept | Hard]*

What is special about CatBoost? Explain ordered boosting, ordered target statistics and symmetric trees.

**Q15.** *[Concept | Medium]*

When would you choose XGBoost, LightGBM or CatBoost? Give a decision summary.

**Q16.** *[Concept | Medium]*

How do you prevent overfitting in gradient boosting models? Which levers are most effective?

**Q17.** *[Concept | Hard]*

Explain stacking versus blending. What are the pitfalls and how do you avoid leakage?

**Q18.** *[Concept | Hard]*

Derive the intuition behind the SVM maximum-margin classifier. What are support vectors and how does the soft-margin parameter C work?

**Q19.** *[Concept | Hard]*

Explain the kernel trick and compare the linear, polynomial and RBF kernels. What does gamma do?

**Q20.** *[Concept | Medium]*

How do SVMs produce probabilities and handle multi-class problems? What are SVR and the epsilon-tube?

**Q21.** *[Concept | Medium]*

KNN: how do you choose k, the distance metric and handle the curse of dimensionality? What are its computational drawbacks?

**Q22.** *[Concept | Medium]*

Compare grid search, random search and Bayesian optimization for hyperparameter tuning. When would you use each?

**Q23.** *[Concept | Hard]*

Explain SHAP: what are Shapley values, what properties do they satisfy, and how does TreeSHAP work?

**Q24.** *[Concept | Medium]*

Compare LIME and SHAP. When would you choose one over the other, and what are the limitations of LIME?

**Q25.** *[Concept | Medium]*

Beyond SHAP and LIME, what other interpretability tools exist for tree ensembles (PDP, ICE, ALE, surrogate models)?

**Q26.** *[Concept | Medium]*

Why is accuracy misleading for imbalanced classification, and which metrics should you use instead?

**Q27.** *[Concept | Hard]*

Explain SMOTE, its variants and the correct way to use it in a cross-validation pipeline. What are its downsides?

**Q28.** *[Concept | Medium]*

Compare class weights, resampling and threshold moving for imbalanced data. Which would you try first?

**Q29.** *[Concept | Hard]*

What is probability calibration? How do you detect miscalibration and fix it with Platt scaling or isotonic regression?

**Q30.** *[Concept | Medium]*

What is data leakage? Give the main types and ways to detect and prevent it.

**Q31.** *[Concept | Medium]*

What are the cross-validation schemes you would use for time series, grouped data and imbalanced data, and why?

**Q32.** *[Concept | Hard]*

Distinguish data drift, concept drift, label drift and prediction drift. How do you detect each in production?

**Q33.** *[Concept | Medium]*

Describe the end-to-end process of deploying a tree-ensemble model to production and what to monitor afterward.

**Q34.** *[Concept | Hard]*

Why can tree-based models not extrapolate, and how do you handle trending targets such as demand or prices?

**Q35.** *[Concept | Medium]*

How do you encode high-cardinality categorical features for tree models without leakage?

**Q36.** *[Concept | Hard]*

Explain the bias-variance trade-off for ensembles. How do random forest, boosting, and stacking each shift it?

**Q37.** *[Concept | Hard]*

Why do correlated features hurt interpretation of tree ensembles but typically not their predictive accuracy? How would you deal with them?

**Q38.** *[Concept | Hard]*

How do you choose the loss function and objective in boosting for different tasks (log loss, focal loss, Huber, quantile, Poisson, ranking)?

**Q39.** *[Concept | Hard]*

What are the pros and cons of random forests versus gradient boosting versus SVM versus KNN on tabular data? Summarize when each is preferred.

**Q40.** *[Concept | Hard]*

How does the choice of learning rate interact with number of trees and tree depth in boosting, and how do you set them under a compute budget?

**Q41.** *[Concept | Hard]*

What is the relationship between the SVM hinge loss and logistic loss, and how does the dual formulation yield sparsity in support vectors?

**Q42.** *[Concept | Medium]*

What is the difference between weighting classes in the loss and adjusting the decision threshold, in terms of the model's ranking and probability outputs?

**Q43.** *[Concept | Hard]*

Explain how to handle concept drift with retraining strategies: sliding window, expanding window, weighting and online learning. What are the trade-offs?

**Q44.** *[Concept | Medium]*

What is the Population Stability Index (PSI) and how is it computed and interpreted?

**Q45.** *[Concept | Hard]*

Compare ROC-AUC and PR-AUC and explain what the Gini coefficient and KS statistic mean in credit scoring.

**Q46.** *[Concept | Medium]*

What are the main differences between level-wise, leaf-wise and symmetric tree growth, and what are the consequences for overfitting and speed?

**Q47.** *[Concept | Hard]*

How would you explain the difference between global and local interpretability, and what are the regulatory implications (for example reason codes in credit decisions)?

**Q48.** *[Concept | Medium]*

What is the difference between feature importance used for feature selection and recursive feature elimination (RFE)? When does RFE fail?

**Q49.** *[Concept | Medium]*

How do you handle missing values for tree-based models compared with SVM or KNN?

**Q50.** *[Concept | Hard]*

What is the effect of tree depth and number of leaves on the types of interactions a boosting model can learn, and how can you check whether your model uses interactions?

**Q51.** *[Coding | Medium]*

Write Python to compute Gini impurity and the best split (information gain) of a single numeric feature for a binary-labelled array, from scratch.

**Q52.** *[Coding | Medium]*

Show how to train a random forest with OOB scoring and compare impurity importance with permutation importance on a held-out set using scikit-learn.

**Q53.** *[Coding | Medium]*

Write code to train an XGBoost classifier with early stopping, stratified cross-validation and scale_pos_weight for an imbalanced dataset, reporting PR-AUC.

**Q54.** *[Coding | Hard]*

Implement stacking correctly with out-of-fold predictions (without using StackingClassifier) for a random forest, a gradient boosting model and a logistic regression meta-learner.

**Q55.** *[Coding | Medium]*

Write a scikit-learn pipeline that tunes an RBF SVM (C and gamma) with a randomized search, including scaling, so that there is no leakage.

**Q56.** *[Coding | Medium]*

Show how to use imblearn correctly with SMOTE inside cross-validation, and how to compare it against class weights.

**Q57.** *[Coding | Medium]*

Show how to compute SHAP values for a trained LightGBM model, produce global and local plots, and map the top local contributions to reason codes.

**Q58.** *[Coding | Hard]*

Write Optuna code to tune a LightGBM classifier with pruning, using stratified CV and PR-AUC as the objective.

**Q59.** *[Coding | Medium]*

Write a function that computes PSI between a reference and a current sample for a numeric feature and flags drift.

**Q60.** *[Coding | Hard]*

Implement a simple gradient boosting regressor from scratch (squared error) using sklearn decision trees as weak learners.

**Q61.** *[Scenario | Hard]*

A payments company (card-not-present e-commerce) wants a real-time fraud detection model. Fraud is 0.15 percent of transactions, labels (chargebacks) arrive 30-90 days late, decisions must be made in under 50 ms, and each missed fraud costs about 40 times a false decline. Which model and approach would you choose and why?

**Q62.** *[Scenario | Hard]*

A retail bank wants to replace its logistic regression application scorecard with a more accurate default-prediction model, but regulators require explainability and adverse-action reason codes. How do you proceed?

**Q63.** *[Scenario | Hard]*

A grocery retailer needs weekly demand forecasts for 20,000 SKUs across 500 stores, with many intermittent (zero-heavy) series, promotions, holidays and a growing trend. Would you use gradient boosting, and how do you set it up?

**Q64.** *[Scenario | Medium]*

A telecom operator wants a churn model. The first random forest has 0.97 AUC on validation, but a colleague finds that the top feature is "retention_offer_accepted". What is going on and how do you fix it?

**Q65.** *[Scenario | Hard]*

A health-tech startup has 3,000 patient records with 12,000 genomic and lab features and a rare disease label (4 percent positive). They ask which algorithm to use and how to validate. What do you recommend?

**Q66.** *[Scenario | Hard]*

An insurance company wants to price auto policies by predicting claim cost per customer. Data: 2 million policies, 93 percent have zero claims, positives are heavy-tailed. Which modeling approach would you use?

**Q67.** *[Scenario | Hard]*

A ride-hailing company deploys an ETA/price model and after two months its performance declines. Offline validation looked fine. How do you investigate and respond?

**Q68.** *[Scenario | Hard]*

An e-commerce marketplace wants to predict seller default on payouts using a feature that has 40,000 distinct categories (seller_category_code) plus numeric history. The data scientist one-hot encoded it and the model overfit. What do you do?

**Q69.** *[Scenario | Medium]*

A subscription streaming company trained an XGBoost churn model with 500 trees at depth 10 and learning rate 0.3. Training AUC is 0.99 and validation AUC is 0.78. Walk through how you would fix it.

**Q70.** *[Scenario | Hard]*

A bank's anti-money-laundering team has 50 confirmed laundering cases among 5 million accounts, and investigators can review only 500 alerts per week. How would you build a detection system?

**Q71.** *[Scenario | Hard]*

A SaaS company serves a lead-scoring model through an API with a 20 ms p99 latency limit. The current stacked ensemble of 5 models (XGBoost, LightGBM, random forest, SVM, neural net) takes 180 ms. What do you do?

**Q72.** *[Scenario | Medium]*

A marketing team wants to target a promotion at customers most likely to respond. Only 5 percent of customers respond, and each promotion costs 2 dollars while a response yields 30 dollars of profit. How do you choose the model, threshold and evaluate success?

**Q73.** *[Scenario | Hard]*

A bank's fraud model performs well offline (PR-AUC 0.45) but in production the same model catches far fewer frauds, and analysts suspect fraudsters changed tactics. A product manager asks whether to simply retrain. How do you answer?

**Q74.** *[Scenario | Medium]*

A real-estate platform wants to predict house prices with gradient boosting. The data has price in a wide range (50k to 20M), and the model makes large percentage errors on cheap houses and extrapolates badly for new luxury listings. How do you improve it?

**Q75.** *[Scenario | Hard]*

A hospital wants to deploy a sepsis early-warning model. In validation AUC is 0.88, but clinicians complain about alert fatigue. How do you decide the threshold and make the model clinically useful?

**Q76.** *[Scenario | Hard]*

An ad-tech company predicts click-through rate with 200 million rows, extremely sparse categorical features with millions of values, and a 0.5 percent click rate. Which algorithm and infrastructure would you choose?

**Q77.** *[Scenario | Hard]*

A manufacturer wants to predict equipment failures from sensor data. Failures are rare (0.3 percent of machine-days), the data is time ordered, and machines differ. How do you build and validate the model?

**Q78.** *[Scenario | Medium]*

A data scientist reports that SMOTE improved the F1 score of a random forest from 0.42 to 0.71 on the test set. You suspect something is wrong. What do you check?

**Q79.** *[Scenario | Hard]*

A bank is migrating a credit-card default model to production with monthly scoring. Describe your monitoring and retraining plan, including the metrics, thresholds and governance.

**Q80.** *[Scenario | Hard]*

A recommender team at a news publisher uses KNN on user embeddings to find similar articles. Latency is high at 100 million items, and quality degrades for new users. How do you improve it?

## Advanced ML: Unsupervised, Deep Learning, Time Series and Recommenders

**Q81.** *[Concept | Medium]*

Explain how the K-means algorithm works, what objective it optimises, and list its main limitations.

**Q82.** *[Concept | Medium]*

How do you choose the number of clusters k? Compare the elbow method, silhouette analysis and the gap statistic.

**Q83.** *[Concept | Medium]*

What is the silhouette coefficient, what are its strengths and weaknesses, and what other internal or external clustering metrics exist?

**Q84.** *[Concept | Medium]*

Explain agglomerative hierarchical clustering. What are the common linkage criteria and when would you prefer it to K-means?

**Q85.** *[Concept | Medium]*

Describe DBSCAN: its parameters, point types, advantages and weaknesses, and how you would choose eps and min_samples.

**Q86.** *[Concept | Hard]*

How do Gaussian Mixture Models differ from K-means? Explain the EM algorithm and how to choose the number of components.

**Q87.** *[Concept | Medium]*

Explain PCA mathematically. How is it computed, what do the components and explained variance mean, and what are its assumptions?

**Q88.** *[Concept | Medium]*

Compare PCA and t-SNE. When would you use each?

**Q89.** *[Concept | Hard]*

What are the common pitfalls when interpreting t-SNE plots, and what is perplexity?

**Q90.** *[Concept | Hard]*

How does UMAP work and how does it compare to t-SNE?

**Q91.** *[Concept | Medium]*

What is Linear Discriminant Analysis (LDA) as a dimensionality reduction technique and how does it differ from PCA?

**Q92.** *[Concept | Medium]*

Explain how Isolation Forest detects anomalies and compare it with Local Outlier Factor and One-Class SVM.

**Q93.** *[Concept | Hard]*

What categories of anomaly detection approaches exist, and how do you evaluate an anomaly detector when labels are rare or absent?

**Q94.** *[Concept | Medium]*

Explain backpropagation. Why is it efficient and what role does the chain rule play?

**Q95.** *[Concept | Medium]*

Compare common activation functions (sigmoid, tanh, ReLU, Leaky ReLU, GELU, softmax) and where each is used.

**Q96.** *[Concept | Hard]*

What are vanishing and exploding gradients, why do they happen, and how do you mitigate them?

**Q97.** *[Concept | Hard]*

Compare SGD, momentum, RMSProp and Adam. Also explain learning-rate schedules and why AdamW is often preferred to Adam with L2 regularisation.

**Q98.** *[Concept | Medium]*

How does batch normalisation work, and what is the difference between its behaviour in training and inference?

**Q99.** *[Concept | Medium]*

Compare batch normalisation, layer normalisation and group normalisation. Why do transformers use layer norm?

**Q100.** *[Concept | Medium]*

What is dropout, how does it work at training and test time, and what other techniques regularise neural networks?

**Q101.** *[Concept | Medium]*

Explain how a convolutional neural network works, including convolution, pooling, parameter sharing and receptive field.

**Q102.** *[Concept | Hard]*

Explain recurrent neural networks and backpropagation through time. What are their limitations?

**Q103.** *[Concept | Hard]*

Describe the LSTM cell and its gates. Why does it mitigate vanishing gradients, and how does a GRU differ?

**Q104.** *[Concept | Hard]*

Explain the attention mechanism, including scaled dot-product attention and why the scaling factor is used.

**Q105.** *[Concept | Hard]*

Describe the transformer architecture: multi-head attention, positional encoding, residual connections, and the difference between encoder-only, decoder-only and encoder-decoder models.

**Q106.** *[Concept | Easy]*

What is TF-IDF, how is it computed, and what are its limitations?

**Q107.** *[Concept | Medium]*

Compare static word embeddings (word2vec, GloVe, fastText) with contextual embeddings (BERT-style). How would you use each?

**Q108.** *[Concept | Medium]*

Outline a robust approach to a text classification problem, from preprocessing to evaluation, including class imbalance.

**Q109.** *[Concept | Medium]*

What is stationarity in time series, why does it matter, and how do you test for and achieve it?

**Q110.** *[Concept | Hard]*

Explain the ARIMA(p,d,q) model and how you select p, d and q using ACF and PACF and information criteria.

**Q111.** *[Concept | Medium]*

What is SARIMA and how do you handle seasonality in a time series? Compare options for multiple seasonalities.

**Q112.** *[Concept | Medium]*

How does Facebook/Meta Prophet model a time series, and what are its strengths and weaknesses?

**Q113.** *[Concept | Hard]*

How should forecasting models be validated, and which error metrics are appropriate (MAE, RMSE, MAPE, sMAPE, MASE)?

**Q114.** *[Concept | Hard]*

When would you use machine learning models such as gradient boosting or deep learning for forecasting rather than ARIMA or exponential smoothing? How do you build features?

**Q115.** *[Concept | Medium]*

Explain user-based and item-based collaborative filtering. What are the trade-offs?

**Q116.** *[Concept | Hard]*

Explain matrix factorisation for recommendations, including ALS versus SGD and how it handles implicit feedback.

**Q117.** *[Concept | Medium]*

What is the cold start problem in recommender systems and what strategies address it?

**Q118.** *[Concept | Hard]*

How do you evaluate a recommender system offline and online? Cover ranking metrics, beyond-accuracy metrics and pitfalls.

**Q119.** *[Concept | Hard]*

What is the fundamental problem of causal inference? Explain confounding, propensity scores and how they are used to estimate treatment effects from observational data.

**Q120.** *[Concept | Hard]*

Explain difference-in-differences, instrumental variables and regression discontinuity. When is each appropriate and what are their key assumptions?

**Q121.** *[Concept | Hard]*

What is uplift modelling and how does it differ from standard response prediction? Describe the main modelling approaches and how uplift models are evaluated.

**Q122.** *[Concept | Medium]*

Explain the basics of reinforcement learning: MDPs, policies, value functions, and Q-learning. How does it differ from supervised learning?

**Q123.** *[Concept | Medium]*

What is the exploration-exploitation trade-off and how do multi-armed bandits (epsilon-greedy, UCB, Thompson sampling) address it?

**Q124.** *[Concept | Medium]*

What is data drift and concept drift, and how would you monitor a deployed machine learning model?

**Q125.** *[Concept | Medium]*

Describe the main components of an MLOps workflow: reproducibility, CI/CD for models, feature stores, model registry and deployment patterns.

**Q126.** *[Concept | Hard]*

Describe the two-stage architecture (retrieval then ranking) of large-scale recommender systems and what a two-tower model is.

**Q127.** *[Coding | Medium]*

Write Python code with scikit-learn that standardises a customer feature matrix, fits K-means for k from 2 to 10, selects k using the silhouette score, and prints cluster profiles.

**Q128.** *[Coding | Medium]*

Implement PCA from scratch using NumPy (via SVD), returning the projected data and the explained variance ratio, and show how to project new data.

**Q129.** *[Coding | Medium]*

Write code that detects anomalies in a numeric feature table with Isolation Forest, and evaluates it against a small set of labelled anomalies using precision-recall metrics.

**Q130.** *[Coding | Hard]*

Write a PyTorch training loop for a small feed-forward classifier that uses batch normalisation, dropout, AdamW, a learning-rate scheduler, gradient clipping and early stopping on validation loss.

**Q131.** *[Coding | Hard]*

Write Python code to perform rolling-origin (walk-forward) validation of a SARIMAX model on a daily series with weekly seasonality, reporting MAE and MASE against a seasonal naive baseline.

**Q132.** *[Coding | Medium]*

Build a text classification pipeline with TF-IDF and logistic regression in scikit-learn, tune it with cross-validation, and handle class imbalance.

**Q133.** *[Coding | Hard]*

Implement matrix factorisation with biases for explicit ratings using SGD in NumPy, and show how to compute RMSE on a held-out set and generate top-N recommendations.

**Q134.** *[Coding | Hard]*

Implement scaled dot-product attention with optional causal masking in NumPy, and explain the shapes involved.

**Q135.** *[Scenario | Hard]*

Company context: A video streaming service (millions of subscribers, a large catalogue) wants to improve its home page recommendations. A large share of new subscribers churn in the first 30 days, and new titles get little exposure. Design a recommender that handles new users and new titles.

**Q136.** *[Scenario | Medium]*

Company context: A grocery and general merchandise retailer with 3 million loyalty card customers wants a customer segmentation for marketing. They have two years of transactions, product categories, store visits and demographics. How would you build and validate the segmentation?

**Q137.** *[Scenario | Hard]*

Company context: An energy utility needs hourly electricity load forecasts for the next 48 hours for its region to plan generation purchases. Load depends on temperature, time of day, day of week, holidays and long-term trends. Describe your end-to-end approach.

**Q138.** *[Scenario | Hard]*

Company context: A telecom operator wants to detect network anomalies (cell tower outages, degraded throughput, unusual call-drop rates) from KPI time series streamed every 5 minutes for 50,000 cells. Few incidents are labelled. Design the anomaly detection system.

**Q139.** *[Scenario | Hard]*

Company context: A retail bank has a transaction fraud dataset where only 0.1 percent of transactions are confirmed fraud, and labels arrive with a delay of several weeks. Fraud patterns change as criminals adapt. How would you combine unsupervised and supervised approaches?

**Q140.** *[Scenario | Hard]*

Company context: An e-commerce retailer sends discount coupons by email and currently targets customers with the highest predicted purchase probability. Margins are thin and the marketing director suspects many of those customers would buy anyway. Propose an approach to target coupons profitably.

**Q141.** *[Scenario | Hard]*

Company context: A streaming service launched a new "autoplay next episode" feature to all users in one country first, without randomisation, and leadership asks whether it increased watch time. A neighbouring country with similar users did not get the feature. How do you estimate the causal effect?

**Q142.** *[Scenario | Hard]*

Company context: A retailer with 2,000 stores and 50,000 SKUs needs weekly demand forecasts per store-SKU for replenishment. Many items sell rarely, promotions create spikes, and new products launch frequently. Describe your forecasting solution.

**Q143.** *[Scenario | Medium]*

Company context: A subscription software company deployed a churn prediction model six months ago with an AUC of 0.84. The retention team reports the model now "feels worse". Dashboards show AUC on recent labelled data of 0.74. Diagnose and fix the problem.

**Q144.** *[Scenario | Medium]*

Company context: A software company receives 20,000 customer support tickets per week and wants to route each ticket to the correct team (12 categories). Only 3,000 tickets are labelled, labels are partly inconsistent, and some categories are rare. How would you build the classifier?

**Q145.** *[Scenario | Hard]*

Company context: A manufacturer has vibration, temperature and pressure sensors on 500 industrial pumps sampled every second. Failures are rare (a few per year) and maintenance logs are incomplete. Design a predictive maintenance solution.

**Q146.** *[Scenario | Hard]*

Company context: A news and media platform recommends articles using implicit feedback (clicks and reading time). The team notices the recommender mostly shows popular articles, new content decays quickly in value, and editors worry about filter bubbles. How would you redesign the system?

**Q147.** *[Scenario | Hard]*

Company context: A ride-hailing company wants to decide which incentive (bonus level) to offer drivers in different zones and times to improve supply during peaks. Incentive effects are uncertain and change over time. Would you use a bandit or full reinforcement learning, and how would you set it up?

**Q148.** *[Scenario | Hard]*

> Company context: A coffee chain opens a new kind of drive-through format in 12 pilot stores and wants to know its effect on sales. The stores were chosen by regional managers who picked high-potential locations. No randomisation was possible. How do you estimate the incremental sales?

**Q149.** *[Scenario | Medium]*

> Company context: Your team trains a deep neural network for demand prediction. The training loss decreases for 2 epochs, then suddenly jumps to NaN. Another model trains without error but the validation loss never drops below the baseline. Walk through your debugging process.

**Q150.** *[Scenario | Hard]*

> Company context: A subscription meal-kit company forecasts weekly orders with Prophet. Orders were stable until a pandemic surge, then a permanent step change, and now a price increase has reduced orders. Forecast errors have grown. How do you adapt the forecasting approach?

**Q151.** *[Scenario | Hard]*

> Company context: An online marketplace must serve personalised recommendations to 20 million users with a response time under 100 ms on the product page, while keeping recommendations fresh as users browse. Design the serving architecture and the supporting ML pipeline.

**Q152.** *[Scenario | Hard]*

> Company context: A credit scoring team trained a gradient boosting model in notebooks with features computed by ad hoc SQL. After deployment the production performance is much worse than offline. Engineers suspect training-serving skew. How do you find the cause and set up the pipeline so this does not recur?

**Q153.** *[Scenario | Hard]*

> Company context: An online learning platform is considering an LSTM or transformer model to predict which students will drop out of a course in the next two weeks, using clickstream logs (daily activity counts, quiz scores, forum posts). The data science lead asks you to compare approaches and justify your choice. What do you recommend?

**Q154.** *[Scenario | Hard]*

> Company context: A mobile game company wants to personalise in-game offers (for example discounted bundles) to maximise long-term revenue and retention, and a researcher proposes full reinforcement learning. The offers could also annoy players and increase churn. How would you approach the problem safely?

**Q155.** *[Scenario | Hard]*

Company context: A telecom operator clusters its subscribers using usage data (call minutes, data volume, roaming, top-up patterns) to design tariffs. Initial K-means with k=4 yields one giant cluster containing 85 percent of customers and tiny clusters of extreme users. How do you fix the segmentation?

**Q156.** *[Scenario | Medium]*

Company context: A streaming music service's offline evaluation shows a new matrix factorisation model improves RMSE on held-out ratings by 4 percent, but the product team is unsure it will improve engagement. The service mostly has implicit feedback (plays and skips). What evaluation plan do you propose?

**Q157.** *[Concept | Hard]*

What is the curse of dimensionality and what are its consequences for clustering and nearest-neighbour methods? How do you deal with it?

**Q158.** *[Concept | Medium]*

Which loss functions are used in deep learning for regression, classification and ranking, and how do you choose among them?

**Q159.** *[Concept | Medium]*

How do learning curves help diagnose underfitting and overfitting in neural network training, and what sanity checks do you run first?

**Q160.** *[Concept | Hard]*

Explain exponential smoothing methods (simple, Holt, Holt-Winters, ETS) and how they relate to ARIMA.

# Part 2: Answer Key

## Advanced ML: Trees, Ensembles and SVM

**A1. Explain Gini impurity and entropy for a decision tree split. How is information gain computed and do the two criteria differ in practice?**

For a node with class proportions p_k, Gini = 1 - sum(p_k^2) and entropy = -sum(p_k * log2(p_k)). Both are 0 for a pure node and maximal for a uniform class mix (Gini max 0.5 for two classes, entropy max 1 bit).

Information gain = impurity(parent) - weighted average of impurity(children), where weights are n_child / n_parent. With entropy this is the classic ID3/C4.5 information gain; with Gini it is called Gini gain (CART). The tree greedily picks the feature/threshold with the largest gain.

In practice the two give very similar trees; they disagree in only a small fraction of splits. Gini is slightly cheaper (no logarithm) and tends to isolate the largest class in its own branch; entropy is slightly more balanced in splits. C4.5 uses gain ratio (gain divided by split information) because plain information gain is biased toward high-cardinality features such as IDs. For regression trees the criterion is variance reduction (MSE).

**A2. Why do fully grown decision trees overfit, and what are pre-pruning and post-pruning (cost-complexity pruning)?**

A fully grown tree keeps splitting until leaves are pure, so it memorizes noise: low bias, very high variance, and training accuracy near 100 percent.

Pre-pruning (early stopping) limits growth with hyperparameters: max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, min_impurity_decrease. It is cheap but can be short-sighted (a poor split may enable a great later split).

Post-pruning grows a full tree and then cuts back. Cost-complexity pruning (CART, sklearn ccp_alpha) minimizes R_alpha(T) = R(T) + alpha * |leaves(T)|, where R(T) is the total misclassification or impurity of the leaves. For each alpha there is a unique smallest subtree minimizing this; larger alpha gives smaller trees. You compute the path of effective alphas with cost_complexity_pruning_path, then choose alpha by cross-validation. Reduced-error pruning (using a validation set) and pessimistic pruning (C4.5) are alternatives.

**A3. How does a decision tree handle continuous features, categorical features and missing values? What are the practical limitations?**

Continuous features: the algorithm sorts values and evaluates candidate thresholds at midpoints between consecutive distinct values, picking the one with best impurity decrease. Histogram-based implementations bin values to speed this up.

Categorical features: classic CART considers binary partitions of categories (2^(k-1)-1 subsets; for binary targets one can order categories by target mean and scan, which is optimal). sklearn trees need numeric encoding, so one-hot or ordinal encoding is needed; one-hot with high cardinality makes trees sparse and deep. LightGBM and CatBoost handle categoricals natively.

Missing values: CART uses surrogate splits; C4.5 distributes fractional instances; XGBoost/LightGBM learn a default direction for missing values at each split; recent sklearn trees support NaN natively.

Limitations: axis-aligned splits cannot capture diagonal boundaries efficiently, piecewise constant predictions (no extrapolation beyond the training range), instability (small data changes alter the tree), and bias toward high-cardinality features.

**A4. Explain bagging and why it reduces variance but not bias. How does a random forest differ from plain bagging of trees?**

Bagging (bootstrap aggregating) trains B models on bootstrap samples (n draws with replacement) and averages predictions (or votes). If each model has variance sigma^2 and pairwise correlation rho, the variance of the average is rho*sigma^2 + (1-rho)*sigma^2/B. Averaging shrinks the second term, so variance falls, but the bias of the ensemble is about the same as a single model, which is why bagging works best with low-bias, high-variance learners such as deep trees.

Random forest adds a second source of randomness: at each split only a random subset of features (max_features, typically sqrt(p) for classification and p/3 for regression) is considered. This decorrelates the trees (reducing rho), so the averaged variance drops further than with plain bagging, where a dominant strong feature makes all trees look alike. The cost is slightly higher bias per tree. Random forests are also embarrassingly parallel.

**A5. What is out-of-bag (OOB) error in a random forest and how is it computed? When is it unreliable?**

Each bootstrap sample contains about 63.2 percent of the unique rows (1 - (1-1/n)^n tends to 1 - 1/e). The remaining ~36.8 percent are out-of-bag for that tree. For each training row, you aggregate predictions only from trees where that row was OOB, then compare to the true label. The average over all rows is the OOB error, an almost unbiased estimate of generalization error (similar to leave-one-out CV) without needing a separate validation set or retraining.

Use: oob_score=True in sklearn, tuning n_estimators and max_features cheaply.

Caveats: it assumes iid rows. With grouped data (multiple rows per customer), time series, or duplicates, OOB rows are leaked from correlated in-bag rows and the estimate is optimistic. It is also not suitable for comparing against models evaluated with a different protocol, and with very few trees some rows have no OOB predictions. For imbalanced problems use OOB probabilities with an appropriate metric (AUC, PR-AUC) rather than accuracy.

**A6. Compare impurity-based (Gini) feature importance, permutation importance and SHAP importance in random forests. What biases does each have?**

Impurity-based (mean decrease in impurity, MDI): sum of impurity reductions from splits on a feature, averaged over trees. It is free to compute but computed on training data, so it is biased toward high-cardinality and continuous features, can give non-zero importance to features that are pure noise when the model overfits, and splits credit unpredictably among correlated features.

Permutation importance: shuffle one feature on held-out data and measure the drop in a metric. It is model-agnostic and measured on test data so it reflects real predictive value. Weaknesses: with correlated features, permuting one creates unrealistic rows and the importance is shared or understated; it costs one prediction pass per feature per repeat; it depends on the chosen metric. Mitigate by clustering correlated features and permuting groups.

SHAP importance (mean absolute SHAP value): grounded in Shapley values, gives local and global consistency, signed direction and interactions; TreeSHAP is exact and polynomial for trees. It explains model behavior, not

necessarily true causal importance, and with correlated features the interventional vs conditional choice matters.

Best practice: report permutation importance on validation data and SHAP for explanations, and treat MDI as a quick screen only.

### A7. Which random forest hyperparameters matter most and how do they affect bias and variance?

n_estimators: more trees reduce variance and plateau; never overfits by adding trees, only costs time. Pick where OOB error flattens.

max_features: smaller values decorrelate trees (lower variance) but raise per-tree bias; most impactful parameter. Try sqrt(p), log2(p), and fractions like 0.3-0.5.

max_depth / min_samples_leaf / max_leaf_nodes: control tree complexity; larger min_samples_leaf (e.g. 5-50) smooths predictions and is especially helpful for noisy data and probability quality.

max_samples: bootstrap fraction; smaller values add diversity and speed up training.

class_weight / criterion: class_weight='balanced_subsample' for imbalance; criterion gini vs entropy rarely matters.

bootstrap: turning it off with ExtraTrees-style random thresholds is another route to decorrelation.

Tune with random search over these, using OOB or CV, and optimize a metric aligned with the business objective.

### A8. Explain gradient boosting step by step. Why is it called "gradient" boosting?

Gradient boosting builds an additive model F_m(x) = F_{m-1}(x) + nu * h_m(x) stage by stage, where each new weak learner h_m (a shallow regression tree) is fit to the negative gradient of the loss with respect to the current predictions.

Steps: (1) Initialize F_0 with a constant minimizing the loss (mean for squared error, log-odds for log loss). (2) For m = 1..M: compute pseudo-residuals r_i = -dL(y_i, F(x_i))/dF(x_i) at F = F_{m-1}; fit a tree h_m to r; optionally compute optimal leaf values via a line search (for log loss, a Newton step per leaf); update F_m = F_{m-1} + nu * h_m with learning rate nu.

It is "gradient" because it performs gradient descent in function space: for squared error the pseudo-residuals are just ordinary residuals, but the framework works for any differentiable loss (log loss, Huber, quantile, Poisson). Regularization comes from small nu, shallow trees, subsampling and early stopping. Unlike bagging, it mainly reduces bias, and trees are built sequentially.

### A9. How does AdaBoost work, and how does it differ from gradient boosting?

AdaBoost trains weak learners (typically decision stumps) sequentially on reweighted data. Initially every sample has weight 1/n. At round t: fit h_t on the weighted data; compute weighted error e_t; set learner weight alpha_t = 0.5 * ln((1 - e_t)/e_t); increase weights of misclassified samples by exp(alpha_t) and decrease others, then normalize. Final prediction is sign(sum alpha_t * h_t(x)).

It can be shown AdaBoost is gradient boosting (forward stagewise additive modeling) with the exponential loss. Differences: AdaBoost focuses on hard examples by reweighting samples, while gradient boosting fits residual gradients and supports arbitrary losses; the exponential loss makes AdaBoost very sensitive to outliers and label noise because misclassified points get exponentially large weights. Gradient boosting with

log loss, Huber loss or regularization is more robust and generally the modern default. AdaBoost has few hyperparameters (n_estimators, learning_rate, base learner depth).

**A10. Contrast bagging and boosting in terms of bias, variance, parallelism and sensitivity to noise.**

Bagging: trains independent models on bootstrap samples; reduces variance; base learners are deep, low-bias, high-variance trees; trivially parallel; robust to label noise and rarely overfits as trees are added.

Boosting: trains models sequentially, each correcting the errors of the previous; mainly reduces bias (and some variance with regularization); base learners are shallow, high-bias trees (depth 3-8); inherently sequential (parallelism only inside tree construction); can overfit if too many rounds, and is more sensitive to noisy labels and outliers because it keeps trying to fit hard points.

Practical rule: random forest is a strong low-tuning baseline; gradient boosting usually wins on tabular accuracy after careful tuning with early stopping. Random forest gives well-behaved OOB estimates; boosting requires a validation set for early stopping.

**A11. What makes XGBoost different from classic gradient boosting? Explain its regularized objective and split gain formula.**

XGBoost optimizes a regularized objective: Obj = sum L(y_i, yhat_i) + sum Omega(f_k), with Omega(f) = gamma*T + 0.5*lambda*sum(w_j^2) where T is the number of leaves and w_j the leaf weights. It uses a second-order Taylor expansion of the loss, with gradients g_i and Hessians h_i. For a leaf with sums G and H, the optimal weight is w* = -G/(H + lambda) and the objective contribution is -0.5*G^2/(H + lambda) + gamma.

Split gain = 0.5 * [ G_L^2/(H_L+lambda) + G_R^2/(H_R+lambda) - (G_L+G_R)^2/(H_L+H_R+lambda) ] - gamma. A split is made only if gain is positive, so gamma acts as built-in pruning, and trees are pruned backward (max_depth then prune negative gain).

Engineering differences: sparsity-aware split finding with learned default direction for missing values, weighted quantile sketch and histogram methods (tree_method=hist) for speed, column block structure for parallel split search, cache-aware access, out-of-core computation, column and row subsampling, and built-in L1/L2 regularization. Compared with older GBM, the Hessian-based Newton step and explicit regularization improve accuracy and generalization.

**A12. Describe the key XGBoost hyperparameters and a sensible tuning order.**

Core: n_estimators with learning_rate (eta): small eta (0.01-0.1) with many rounds and early_stopping_rounds on a validation set. max_depth (3-10): interaction order/complexity. min_child_weight: minimum sum of Hessian in a child; larger means more conservative (important for imbalanced data). gamma (min_split_loss): minimum gain to split. subsample (0.5-1) and colsample_bytree / bylevel / bynode: stochastic regularization. reg_lambda (L2) and reg_alpha (L1). scale_pos_weight: negative/positive ratio for imbalance. max_delta_step: helps logistic regression with extreme imbalance. tree_method: hist for speed; grow_policy lossguide for leaf-wise.

Tuning order: (1) fix eta around 0.05-0.1 and find n_estimators by early stopping; (2) tune max_depth and min_child_weight; (3) gamma; (4) subsample and colsample; (5) lambda/alpha; (6) lower eta and retrain with proportionally more trees. Use random search or Bayesian optimization (Optuna) with stratified CV and a metric like AUC or logloss. Always keep a final untouched test set.

**A13. How does LightGBM differ from XGBoost? Explain leaf-wise growth, GOSS and EFB.**

Leaf-wise (best-first) growth: LightGBM splits the leaf with the largest loss reduction regardless of level, giving deeper asymmetric trees and lower loss for the same number of leaves than level-wise growth, but greater overfit risk on small data. Control via num_leaves (main parameter, keep below 2^max_depth), max_depth, min_data_in_leaf.

Histogram-based: features are binned (max_bin default 255), reducing memory and split-search cost; histogram subtraction trick computes a sibling histogram from parent minus child.

GOSS (Gradient-based One-Side Sampling): keep all samples with large gradients (under-trained), randomly sample a fraction of small-gradient ones and up-weight them to keep the distribution unbiased, so each tree trains on fewer rows.

EFB (Exclusive Feature Bundling): bundles mutually exclusive sparse features (rarely non-zero simultaneously, e.g. one-hot columns) into one, cutting feature count.

Also: native categorical support via optimal partition split (categorical_feature), faster training and lower memory than XGBoost on large data. Caveats: leaf-wise growth overfits small datasets (use min_data_in_leaf, num_leaves smaller), and the categorical handling can overfit high-cardinality features (use cat_smooth, min_data_per_group).

**A14. What is special about CatBoost? Explain ordered boosting, ordered target statistics and symmetric trees.**

CatBoost targets two problems: target leakage in categorical encoding and prediction shift in boosting.

Ordered target statistics: a categorical value is encoded by the mean target of previous rows of the same category in a random permutation, smoothed with a prior: (sum_prev_y + a*prior)/(count_prev + a). Each row's encoding uses only earlier rows, avoiding the leakage of naive target (mean) encoding. It also builds combinations of categorical features automatically.

Ordered boosting: standard boosting computes residuals on the same data the model was trained on, causing a biased (shifted) gradient estimate. CatBoost maintains models trained on permutation prefixes so the residual for a sample is estimated by a model that never saw that sample.

Symmetric (oblivious) trees: every node at the same depth uses the same feature and threshold, giving balanced trees that are fast at inference, act as regularization, and resist overfitting.

Practical: strong default performance with little tuning, excellent with many high-cardinality categoricals, GPU support. Main hyperparameters: iterations, learning_rate, depth, l2_leaf_reg, cat_features, one_hot_max_size, border_count. Training can be slower than LightGBM on pure numeric data.

**A15. When would you choose XGBoost, LightGBM or CatBoost? Give a decision summary.**

LightGBM: very large datasets (millions of rows), many features, need for fast training and low memory; accept careful tuning of num_leaves/min_data_in_leaf to avoid overfitting. Good default for speed.

XGBoost: mature, stable, widely supported (distributed, GPU, many language bindings, monotone and interaction constraints, custom objectives); a solid choice for medium data when you want level-wise robust trees and extensive production tooling.

CatBoost: lots of categorical features (especially high-cardinality), small to medium datasets where robust defaults matter, or when you need less tuning and strong resistance to overfitting; inference is fast with symmetric trees.

Common to all: histogram methods, regularization, early stopping, missing-value handling, SHAP via TreeSHAP. In practice differences in final accuracy are small; the winning factors are feature engineering, validation design and tuning. Benchmark all three with the same CV folds and consider blending them.

### A16. How do you prevent overfitting in gradient boosting models? Which levers are most effective?

1. Early stopping on a validation fold (the single most useful tool); pick n_estimators from it.

2. Lower learning rate with more trees (shrinkage).

3. Constrain tree complexity: small max_depth (3-6), fewer num_leaves, larger min_child_weight / min_data_in_leaf.

4. Stochastic boosting: subsample rows (0.5-0.9) and columns (colsample).

5. Explicit regularization: lambda (L2), alpha (L1), gamma (min split gain).

6. Monotonic and interaction constraints, which inject domain knowledge.

7. Better validation: group or time-aware CV so that leakage does not mask overfit.

8. Feature hygiene: remove leaky and noisy high-cardinality features; for categoricals use smoothed or ordered encoding.

9. Clean label noise and clip outliers; use robust losses (Huber) when needed.

Monitor the gap between train and validation metrics and learning curves, not just final test score.

### A17. Explain stacking versus blending. What are the pitfalls and how do you avoid leakage?

Stacking: train several diverse base models, produce out-of-fold (OOF) predictions for the training set via K-fold CV, and train a meta-learner (often logistic or ridge regression, or a shallow GBM) on those OOF predictions. For the test set, base-model predictions are averaged over fold models or from models refit on all data.

Blending: a simpler variant using a single holdout set: base models train on the training part, predict the holdout, and the meta-model is trained on the holdout predictions. It is simpler and leakage-safe but wastes data and gives noisier meta-training.

Pitfalls: (1) leakage if the meta-learner is trained on in-sample predictions of base models, which are overconfident; always use OOF. (2) Use the same folds consistently and group/time-aware splits where needed. (3) Overfit meta-learner: keep it simple and regularized, optionally non-negative weights. (4) Correlated base models add little: seek diversity (linear, trees, KNN, GBM, neural nets, different feature sets). (5) Evaluate the whole pipeline with nested CV or a final untouched test set. (6) Complexity vs gain: stacking often yields small gains and adds latency and maintenance cost in production.

### A18. Derive the intuition behind the SVM maximum-margin classifier. What are support vectors and how does the soft-margin parameter C work?

For linearly separable data, the SVM chooses the hyperplane $w.x + b = 0$ that maximizes the geometric margin $2/||w||$, with constraints $y_i (w.x_i + b) >= 1$. Equivalent to minimizing $0.5||w||^2$. Maximizing margin yields better generalization bounds (VC theory) because the capacity depends on margin rather than dimension.

Support vectors are the training points lying on or inside the margin; they alone determine the hyperplane (in the dual, only points with $alpha_i > 0$). Removing other points does not change the solution.

Soft margin: allow slack xi_i >= 0, minimize 0.5||w||^2 + C * sum xi_i with y_i(w.x_i + b) >= 1 - xi_i. Equivalent to hinge loss plus L2 penalty. Large C penalizes violations heavily: narrow margin, low bias, high variance, risk of overfitting. Small C tolerates misclassification: wide margin, more regularization, higher bias. C is the inverse of regularization strength. Always scale features before SVM and tune C on a log grid.

**A19. Explain the kernel trick and compare the linear, polynomial and RBF kernels. What does gamma do?**

The SVM dual depends on the data only through inner products x_i.x_j. A kernel K(x,z) = phi(x).phi(z) computes inner products in a (possibly infinite-dimensional) feature space without constructing phi, allowing nonlinear boundaries with the same convex optimization. A valid kernel must be symmetric positive semi-definite (Mercer).

Linear: K = x.z; best for high-dimensional sparse data (text) where data is nearly separable; fast with LinearSVC.

Polynomial: K = (gamma*x.z + r)^d; models feature interactions up to degree d; sensitive to scaling and numerically unstable at high degree.

RBF: K = exp(-gamma*||x - z||^2); universal approximator, default choice. Gamma sets the radius of influence of a single training point: large gamma means very local, wiggly boundaries (low bias, high variance, overfitting, each point its own island); small gamma means smooth, nearly linear boundaries (underfitting). Gamma and C interact: tune them jointly on a log grid (e.g. C in 1e-2..1e3, gamma in 1e-4..1e1) after standardizing features. Training cost is O(n^2) to O(n^3), so SVMs are impractical beyond roughly 100k rows without approximations (Nystroem, random Fourier features).

**A20. How do SVMs produce probabilities and handle multi-class problems? What are SVR and the epsilon-tube?**

SVM outputs a signed distance to the hyperplane, not a probability. Platt scaling fits a logistic regression on decision scores (using internal cross-validation; sklearn probability=True, which slows training) to produce probabilities; isotonic calibration is an alternative.

Multi-class: SVM is inherently binary. One-vs-rest trains K classifiers; one-vs-one trains K(K-1)/2 and votes (libsvm default, better for kernel SVMs since each subproblem is smaller).

SVR: fits a function with an epsilon-insensitive loss; errors smaller than epsilon are ignored, forming a tube around the prediction. Points outside the tube become support vectors. Parameters: epsilon (tube width, controls sparsity), C (penalty for points outside the tube), gamma for RBF. Larger epsilon gives fewer support vectors and a smoother fit.

**A21. KNN: how do you choose k, the distance metric and handle the curse of dimensionality? What are its computational drawbacks?**

Choice of k: small k gives low bias and high variance (jagged boundary); large k smooths and increases bias. Choose by cross-validation, odd k for binary problems to avoid ties, often between 5 and sqrt(n).

Distance metric: Euclidean for dense scaled numeric features, Manhattan for high dimensions or outliers, cosine for text/embeddings, Hamming for categorical, Mahalanobis to account for correlation. Feature scaling is mandatory. Distance-weighted voting (weights 1/d) reduces the effect of far neighbors.

Curse of dimensionality: in high dimensions, distances concentrate and the nearest and farthest neighbors are nearly equidistant, so neighbors become meaningless. Remedies: feature selection, PCA or other embeddings, metric learning, and removing irrelevant features.

Computation: no training but prediction is O(n*d) per query, and memory stores the full dataset. Speed up with KD-trees or ball trees (good only for low d), approximate nearest neighbor indexes (HNSW, FAISS, Annoy), and dimension reduction. Also sensitive to class imbalance (majority class dominates), so use weighted voting or resampling.

**A22. Compare grid search, random search and Bayesian optimization for hyperparameter tuning. When would you use each?**

Grid search enumerates all combinations; cost grows exponentially with the number of parameters, wastes budget on unimportant dimensions, and only tests values you specified. Fine for 1-2 parameters.

Random search samples configurations from distributions (log-uniform for learning rate, C, gamma). Bergstra and Bengio showed it is more efficient because usually only a few hyperparameters matter, so random sampling explores more distinct values of the important ones. Trivially parallel; a strong baseline.

Bayesian optimization (Gaussian process, TPE as in Optuna/Hyperopt, SMAC) builds a surrogate model of score vs hyperparameters and picks the next point with an acquisition function (expected improvement) balancing exploration and exploitation. Best when each evaluation is expensive and the budget is small (tens to hundreds of trials); sequential nature limits parallelism, though batch variants exist.

Add-ons: successive halving / Hyperband / Optuna pruners stop poor trials early; nested CV avoids optimistic bias in the reported score; always tune with CV on training data and keep a final test set. Search spaces matter more than the algorithm.

**A23. Explain SHAP: what are Shapley values, what properties do they satisfy, and how does TreeSHAP work?**

Shapley values come from cooperative game theory: the contribution of feature i to a prediction is its average marginal contribution over all subsets S of other features: phi_i = sum over S of |S|!(M-|S|-1)!/M! * [f(S u {i}) - f(S)]. The model prediction equals base value (expected prediction) plus the sum of phi_i (local accuracy / additivity).

Properties: efficiency (contributions sum to prediction minus baseline), symmetry (equal contributors get equal values), dummy (features with no effect get zero), and linearity/additivity across models (so ensembles are sums). SHAP is the unique additive attribution satisfying these (plus consistency: if a model changes so a feature matters more, its attribution does not decrease).

f(S) is defined by marginalizing out missing features: interventional (break dependence using a background dataset) or conditional (respect correlations). KernelSHAP approximates via weighted linear regression and is model-agnostic but slow. TreeSHAP exploits tree structure to compute exact values in O(T * L * D^2) instead of exponential time, and also gives interaction values. DeepSHAP and LinearSHAP exist for other model types.

Use: summary (beeswarm) plots for global importance and direction, dependence plots, force/waterfall plots for individual predictions. Caveats: explains the model, not the world; correlated features split credit; additive attributions can mislead in the presence of strong interactions; compute cost for kernel methods.

**A24. Compare LIME and SHAP. When would you choose one over the other, and what are the limitations of LIME?**

LIME explains one prediction by sampling perturbations around the instance, weighting them by proximity, querying the black-box model and fitting a sparse interpretable surrogate (linear model or small tree). The surrogate coefficients are the local explanation.

SHAP gives additive attributions with theoretical guarantees (consistency, local accuracy); TreeSHAP is exact and fast for tree ensembles, and aggregates naturally to global importance.

Choose SHAP for tree models, regulated settings needing consistent attributions, and global plus local views. Choose LIME for quick model-agnostic local explanations on text or images (super-pixel/word perturbations) when SHAP is too slow.

LIME limitations: instability (different samples give different explanations), sensitivity to kernel width and the definition of locality, perturbation sampling that ignores feature correlations and produces out-of-distribution points, no global consistency, and vulnerability to adversarial manipulation. Both methods explain correlation within the model, not causation.

**A25. Beyond SHAP and LIME, what other interpretability tools exist for tree ensembles (PDP, ICE, ALE, surrogate models)?**

Partial dependence plot (PDP): average prediction as a feature varies, marginalizing others. Easy but assumes independence between the plotted feature and the rest; with correlated features it averages over unrealistic combinations.

ICE (individual conditional expectation): one curve per instance; reveals heterogeneity and interactions hidden by the PDP average.

ALE (accumulated local effects): accumulates local prediction differences within narrow bins of the feature, conditional on neighbors, so it is unbiased under correlation and faster than PDP.

Global surrogate: fit an interpretable model (shallow tree, linear model) to the black-box predictions; report fidelity (R2 against black box).

Others: permutation importance, H-statistic for interactions, counterfactual explanations (the smallest change that flips the decision, useful for credit adverse action), monotonic constraints and inherently interpretable models (EBM/GA2M, scorecards) when explainability is a requirement.

**A26. Why is accuracy misleading for imbalanced classification, and which metrics should you use instead?**

With 1 percent positives, a model that always predicts negative has 99 percent accuracy but zero value. Accuracy is dominated by the majority class.

Better metrics: precision (of flagged, how many are real), recall/sensitivity (of real, how many are caught), F1 or F-beta (beta > 1 favors recall), PR-AUC (average precision; focuses on the positive class and is more informative than ROC-AUC under heavy imbalance since ROC-AUC can look good due to the huge number of true negatives), balanced accuracy, MCC, Cohen's kappa, and cost-based metrics (expected cost with FN and FP costs, lift, recall at fixed precision or at fixed alert volume).

Also report calibration (Brier score) if probabilities drive decisions. Choose the operating threshold from the precision-recall curve using business costs, rather than defaulting to 0.5. Use stratified splits to keep the positive rate stable in each fold.

**A27. Explain SMOTE, its variants and the correct way to use it in a cross-validation pipeline. What are its downsides?**

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic minority samples by picking a minority point, choosing one of its k nearest minority neighbors, and interpolating: x_new = x + u*(x_neighbor - x), u in [0,1]. It reduces overfitting that plain duplication causes.

Variants: Borderline-SMOTE (only near the decision boundary), ADASYN (more samples in hard regions), SMOTE-NC (mixed categorical/numeric), SVM-SMOTE, and combos with cleaning such as SMOTE+Tomek links or SMOTE+ENN.

Correct usage: apply resampling only on the training fold, inside the CV loop, e.g. with imblearn.pipeline.Pipeline([('smote', SMOTE()), ('clf', model)]). Never resample before splitting; otherwise synthetic points (derived from validation neighbors) leak and scores are inflated. Never resample the validation or test set.

Downsides: creates unrealistic points in overlapping or noisy regions, ill-defined for high-dimensional or categorical data, increases training time, distorts probabilities (need recalibration), and often brings no gain over class weights or threshold tuning with strong gradient boosting models. Try cheaper options first.

**A28. Compare class weights, resampling and threshold moving for imbalanced data. Which would you try first?**

Class weights (class_weight='balanced', scale_pos_weight): scale the loss contribution of minority samples; no data change, cheap, works with most algorithms. Effectively shifts the decision boundary; probabilities become biased toward the minority class.

Resampling: random undersampling (fast, loses information, good for huge data), random oversampling (duplicates, overfit risk), SMOTE-like synthesis. Changes training distribution so probabilities need recalibration.

Threshold moving: train normally on the true distribution, keep well-calibrated probabilities, then choose the cut-off maximizing business utility or F-beta on validation data. Does not change ranking (AUC unchanged).

Also: cost-sensitive learning, focal loss, anomaly-detection framing when positives are extremely rare, and ensemble methods like balanced random forest or EasyEnsemble.

Order of attack: (1) proper metrics and stratified CV, (2) train a strong GBM with the natural distribution and tune threshold, (3) add class weights, (4) try undersampling for scale or SMOTE if there is evidence it helps, (5) recalibrate if probabilities are used downstream.

**A29. What is probability calibration? How do you detect miscalibration and fix it with Platt scaling or isotonic regression?**

A model is calibrated if among samples predicted with probability p, a fraction p are truly positive. Ranking quality (AUC) can be excellent while probabilities are off.

Detection: reliability diagram (bin predictions, plot mean predicted vs observed frequency; diagonal is perfect), Expected Calibration Error, Brier score (mean squared error of probabilities, decomposes into calibration and refinement), and log loss.

Typical patterns: random forests and bagged trees are under-confident near 0 and 1 (averaging pushes probabilities toward the middle), boosted trees and SVMs are sigmoid-shaped, naive Bayes is over-confident, and class weighting/resampling shifts probabilities.

Fixes (fit on a held-out calibration set or via CV, never on training data): Platt scaling fits a sigmoid p = 1/(1+exp(A*s + B)) to scores; parametric, works with little data, assumes sigmoid distortion. Isotonic regression fits a monotone step function; non-parametric and flexible but needs more data (roughly 1000+ samples) and can overfit. Temperature scaling for neural nets. Use sklearn CalibratedClassifierCV(method='sigmoid' or 'isotonic', cv=...). Calibration matters when probabilities feed expected-loss computations, pricing, thresholds or risk limits, e.g. credit PD or fraud costs.

**A30. What is data leakage? Give the main types and ways to detect and prevent it.**

Data leakage is when information unavailable at prediction time influences training, producing inflated validation scores and failure in production.

Types: (1) Target leakage: a feature derived from or caused by the outcome, e.g. "days since last collection call" predicting default, or a refund flag predicting fraud. (2) Train-test contamination: fitting scalers, imputers, encoders, PCA, SMOTE or feature selection on the whole dataset before splitting. (3) Temporal leakage: random splits on time-dependent data, using future info or features aggregated over windows reaching past the prediction time. (4) Group/duplicate leakage: same customer, device or near-duplicate rows in both train and test. (5) Leakage through the target in encodings (naive target encoding without out-of-fold).

Detection: suspiciously high scores, a single feature with extreme importance, large gap between offline and online performance, point-in-time checks of feature timestamps, adversarial validation.

Prevention: split first, do all fitting inside pipelines on training folds, time-based or GroupKFold splits, point-in-time feature stores, feature review with domain owners, and a shadow/backtest period before launch.

**A31. What are the cross-validation schemes you would use for time series, grouped data and imbalanced data, and why?**

Time series: forward-chaining / expanding or rolling window splits (TimeSeriesSplit) where validation always occurs after training, optionally with a gap (embargo) between train and validation to avoid leakage from lagged features or label horizons. Random K-fold would use the future to predict the past.

Grouped data: GroupKFold or StratifiedGroupKFold so that all rows of the same customer, patient, merchant or device fall in the same fold. Otherwise the model memorizes entity identity and scores look inflated.

Imbalanced data: StratifiedKFold keeps class ratios equal across folds; with extreme imbalance, use repeated stratified K-fold to reduce variance and ensure enough positives per fold.

Other: nested CV for unbiased tuning plus evaluation (inner loop tunes, outer loop evaluates), and purged K-fold for overlapping label windows in finance. The validation scheme must mimic how the model will be used in production.

**A32. Distinguish data drift, concept drift, label drift and prediction drift. How do you detect each in production?**

Data (covariate) drift: P(X) changes while P(y|X) is stable, e.g. new customer segment or a new app version changes feature distributions. Detect with PSI (population stability index; <0.1 stable, 0.1-0.25 moderate, >0.25 major), KS test, chi-square for categoricals, Jensen-Shannon or Wasserstein distance, or a domain classifier trained to separate reference from current data (AUC far from 0.5 means drift).

Concept drift: P(y|X) changes, e.g. fraudsters adapt, or a recession changes default behavior. Requires labels; detect by performance degradation (AUC, precision, calibration) on recent labeled data, or drift detectors (ADWIN, DDM, Page-Hinkley) on error streams.

Label (prior) drift: P(y) changes, e.g. fraud rate rises; monitor the target rate and recalibrate/shift thresholds.

Prediction drift: distribution of model scores changes; available immediately without labels and a useful early warning.

Monitoring practice: track feature, prediction and (delayed) performance metrics on a schedule, set alert thresholds, slice by segment, handle label delay with proxy metrics, and define a retraining trigger (scheduled and drift-triggered), with champion-challenger comparison before promotion.

**A33. Describe the end-to-end process of deploying a tree-ensemble model to production and what to monitor afterward.**

Packaging: serialize the full pipeline (preprocessing + model) with versioning (joblib, ONNX, native XGBoost/LightGBM formats, or MLflow model registry) so training and serving transformations are identical. Pin library versions and test for train/serve skew.

Serving patterns: batch scoring (scheduled jobs writing scores to a table), online REST/gRPC service (containerized, autoscaled, latency budgets, feature store lookups), streaming scoring, or embedded edge. Optimize latency by limiting tree count/depth, using treelite or ONNX runtime, and caching.

Release: CI/CD with unit tests, data validation, shadow mode, canary or A/B rollout, rollback plan, champion-challenger.

Monitoring: operational (latency, error rate, throughput), data quality (missing values, schema, ranges), feature and prediction drift (PSI), performance with delayed labels, calibration, fairness metrics across segments, and business KPIs. Log inputs, outputs and model version for auditability and retraining. Define retraining cadence, owner and alert runbooks. For regulated domains keep documentation, explanations (SHAP, reason codes) and approvals.

**A34. Why can tree-based models not extrapolate, and how do you handle trending targets such as demand or prices?**

A tree predicts the mean of training targets in a leaf, so predictions are piecewise constant and bounded by the range of training targets. If the target trends upward beyond the training range, forests and boosting plateau at the last seen level.

Remedies: (1) model the target after detrending or differencing, or predict a ratio/percentage change versus a baseline (e.g. sales / last-year sales) and invert after. (2) Use a hybrid: linear/ARIMA/Prophet for trend plus GBM on residuals. (3) LightGBM linear_tree=True fits linear models in leaves. (4) Use a log transform to stabilize multiplicative growth. (5) Include normalized lag and rolling features rather than absolute levels. (6) Use models with natural extrapolation (linear, GAM, neural nets) where trend extrapolation is critical. (7) Retrain frequently so the range tracks the data. Always validate with time-ordered backtests that include trend regime shifts.

**A35. How do you encode high-cardinality categorical features for tree models without leakage?**

One-hot encoding explodes dimensionality and makes trees shallow-sparse. Alternatives:

Target (mean) encoding with smoothing: encoded = (n*mean_cat + m*global_mean)/(n + m), computed out-of-fold (K-fold target encoding) or with ordered statistics (CatBoost) so a row's own label never contributes to its encoding. Add noise to reduce overfitting; handle unseen categories with the global mean.

Frequency/count encoding: robust, no label use, but loses meaning if frequency is unrelated.

Native handling: LightGBM categorical_feature (optimal partition) or CatBoost cat_features.

Hashing for very large vocabularies; embeddings from neural nets; grouping rare categories into "other".

Pitfalls: naive target encoding on the full training set leaks labels and yields overfit trees; in time-dependent data compute encodings using only past data; in CV recompute within each training fold.

**A36. Explain the bias-variance trade-off for ensembles. How do random forest, boosting, and stacking each shift it?**

Expected error = bias^2 + variance + irreducible noise.

Random forest: low-bias deep trees; averaging cuts variance, controlled by tree correlation. Adding trees never raises error; lowering max_features reduces correlation at the cost of some bias.

Boosting: starts with high-bias weak learners and iteratively reduces bias; variance grows with rounds and tree depth, so regularization (shrinkage, subsampling, early stopping) is needed to hold variance in check. Too many rounds leads to overfitting.

Stacking/blending: combines models with different error patterns; reduces variance (like averaging) and may reduce bias when base models capture different structure. Gain depends on diversity; a flexible meta-learner can add variance.

Diagnosing: high train error and high validation error means bias (add complexity/features, boost); low train error with high validation error means variance (more data, regularize, bag). Learning curves show which regime you are in.

**A37. Why do correlated features hurt interpretation of tree ensembles but typically not their predictive accuracy? How would you deal with them?**

Trees pick one of several correlated features at a split, nearly arbitrarily (depends on bootstrap or tie-breaking). The prediction benefits from either, so accuracy is largely unaffected, but importance is diluted across the group: each feature looks less important than the group truly is, and which feature "wins" is unstable across seeds. Permutation importance on one feature understates its importance because the correlated twin compensates; SHAP spreads credit among them.

Dealing with it: cluster features using Spearman correlation or hierarchical clustering and keep one representative per cluster, or compute group importance (permute the whole cluster, or sum SHAP of the cluster); apply PCA when interpretability of components is not required; use conditional permutation importance or SHAP with conditional expectations; for linear models also address multicollinearity by regularization (VIF, ridge). Document that importance is shared and avoid claims of causal effect for a single member.

**A38. How do you choose the loss function and objective in boosting for different tasks (log loss, focal loss, Huber, quantile, Poisson, ranking)?**

Binary classification: logistic loss (binary:logistic) gives calibrated-ish probabilities; use AUC or PR-AUC as eval metric. Focal loss (down-weights easy examples) can help extreme imbalance but needs a custom objective and breaks calibration.

Regression: squared error is sensitive to outliers; Huber or pseudo-Huber is robust; MAE/L1 targets the median; log-transform the target for multiplicative errors (or use gamma/tweedie).

Count data: Poisson objective (log link) for counts such as arrivals; negative-binomial style when overdispersed. Tweedie for zero-inflated positive continuous data such as insurance claims or intermittent demand.

Quantile regression: objective quantile loss to produce prediction intervals (train one model per quantile, e.g. 0.1 and 0.9) or asymmetric costs (stock-out vs overstock).

Ranking: lambdarank / rank:ndcg objectives optimize NDCG for search/recommendations, with group ids.

Custom objectives need gradient and Hessian; the Hessian must be positive for stable Newton steps. Match training loss to the business metric when possible, and evaluate with the business metric.

**A39. What are the pros and cons of random forests versus gradient boosting versus SVM versus KNN on tabular data? Summarize when each is preferre...**

Random forest: robust, little tuning, handles mixed features and nonlinearity, OOB estimate, parallel. Cons: large models, slower inference, lower ceiling accuracy than tuned GBM, probabilities not well calibrated, cannot extrapolate.

Gradient boosting: typically best accuracy on tabular data, flexible losses, missing-value handling, constraints. Cons: more hyperparameters, sensitive to noise, sequential training, needs early stopping.

SVM: strong in high dimensions with small to medium n (text, genomics), margin-based generalization, kernel flexibility. Cons: O(n^2-n^3) training, requires scaling, no native probabilities, hard to interpret, tuning of C/gamma.

KNN: simple, non-parametric, no training, good as a local baseline or for recommendations/embeddings. Cons: slow predictions at scale, needs scaling, hurt by irrelevant features and high dimensions, sensitive to imbalance.

Guideline: start with a regularized linear baseline, then a GBM for accuracy, a random forest as a stability check, SVM for small high-dimensional problems, and KNN for similarity retrieval. Pick by data size, dimensionality, interpretability needs and latency.

**A40. How does the choice of learning rate interact with number of trees and tree depth in boosting, and how do you set them under a compute budge...**

Learning rate nu scales each tree's contribution. Smaller nu needs more trees (roughly n_trees proportional to 1/nu) but generalizes better because each step is small and errors average out; this shrinkage is the strongest regularizer in boosting. Deeper trees capture higher-order interactions and reduce bias faster, so fewer trees are needed, but they increase variance and per-tree cost.

Rule of thumb: depth d approximates interaction order (depth 1 = additive model, depth 3-6 typical). With a fixed budget: (1) start with nu around 0.1, depth 4-6, early stopping to get a trees count; (2) tune structure

parameters at this fast setting; (3) at the end lower nu to 0.01-0.03 and retrain with early stopping; gains are usually small but consistent. Under severe budget keep nu 0.1 and prioritize subsampling and histogram methods (bins, GPU). Remember that early stopping must use a validation set separate from the final test set, and when refitting on all data scale the tree count modestly (about 1.1x) for the extra data.

**A41. What is the relationship between the SVM hinge loss and logistic loss, and how does the dual formulation yield sparsity in support vectors?**

SVM objective: minimize 0.5||w||^2 + C * sum max(0, 1 - y_i f(x_i)). The hinge loss is exactly zero for points classified correctly beyond the margin (y f >= 1), so those points contribute no gradient. Logistic loss log(1 + exp(-y f)) is smooth and never exactly zero, so every point influences the fit and the model yields probabilities. Both are convex upper bounds of the 0-1 loss; hinge is a piecewise linear bound.

In the dual, w = sum alpha_i y_i x_i with 0 <= alpha_i <= C and the KKT conditions give alpha_i = 0 for points strictly outside the margin, 0 < alpha_i < C for points exactly on the margin, and alpha_i = C for margin violators. Only points with alpha_i > 0 (support vectors) enter the decision function f(x) = sum alpha_i y_i K(x_i, x) + b. This yields a sparse model, with prediction cost proportional to the number of support vectors. Many support vectors (e.g. over half of the data) signal a noisy or too complex problem.

**A42. What is the difference between weighting classes in the loss and adjusting the decision threshold, in terms of the model's ranking and proba...**

Threshold adjustment leaves the model untouched: scores and ranking (AUC) are identical; only the cut-off moves, so probability estimates remain calibrated to the true prior.

Class weights change the training loss so that errors on the minority class cost more; the fitted function changes (different splits in trees, a different hyperplane), which can improve minority-class representation inside the model, especially when the minority class is tiny and trees would otherwise ignore it. The side effect is probability inflation for the minority class: with weight w on positives, the predicted odds are roughly multiplied by w. You can correct by dividing odds by w or by recalibration.

Practical lesson: if you only need a decision rule and the model is flexible enough, threshold tuning is often equivalent. If you need the model to focus capacity on rare positives, weights help. When business use requires probabilities (expected loss), prefer a natural training distribution plus threshold, or recalibrate after weighting.

**A43. Explain how to handle concept drift with retraining strategies: sliding window, expanding window, weighting and online learning. What are th...**

Expanding window: retrain on all history; stable, uses maximum data, but old regimes dilute recent behavior if the relationship changed.

Sliding window: train on the last N months; adapts quickly to drift but has less data and higher variance; choose N by backtesting multiple window lengths on out-of-time folds.

Recency weighting: use all data with sample weights decaying exponentially with age (half-life tuned in backtest); a compromise.

Online/incremental learning: update continuously (SGD models, Hoeffding trees, xgboost with continued training, ensembles that add and prune members); fast adaptation but risks catastrophic noise sensitivity, label delay issues and harder reproducibility.

Triggers: scheduled retraining (weekly/monthly) vs drift-triggered retraining (PSI or performance drop). Label delay (e.g. chargebacks arrive after 60-90 days) means the freshest labels are incomplete; account for maturity windows. Always validate a new model against the current champion on a recent out-of-time slice and use shadow or canary deployment before promotion. Keep a feature and training-data snapshot for audit and rollback.

### A44. What is the Population Stability Index (PSI) and how is it computed and interpreted?

PSI measures how much a distribution has shifted between a reference (training) sample and a current sample. Bin the variable (deciles of the reference or fixed categories). For each bin i, with reference proportion e_i and current proportion a_i: PSI = sum (a_i - e_i) * ln(a_i / e_i). It is a symmetric divergence (sum of two KL terms).

Rule of thumb: PSI < 0.1 no meaningful shift; 0.1 to 0.25 moderate shift worth investigating; > 0.25 significant shift, likely needs retraining or investigation. Add a small epsilon to avoid zero bins and avoid too many bins with few samples.

Use PSI on input features, on the model score (prediction drift) and on the score by segment. Limitations: depends on binning, does not tell whether performance dropped (covariate shift may be harmless), and does not detect changes in P(y|X). Combine with performance monitoring, KS/Wasserstein tests and domain checks.

### A45. Compare ROC-AUC and PR-AUC and explain what the Gini coefficient and KS statistic mean in credit scoring.

ROC-AUC is the probability that a random positive is ranked above a random negative; it is insensitive to class prior, so it can look strong when positives are rare because the false positive rate has an enormous denominator. PR-AUC (average precision) incorporates precision, which depends on the prior, so it reflects the real cost of false alerts in imbalanced problems and its baseline equals the positive rate.

Gini coefficient = 2*AUC - 1 (the accuracy ratio of the CAP curve); ranges 0 to 1, with 0.5 for a typical application scorecard. KS statistic = maximum difference between the cumulative distributions of scores for positives (bads) and negatives (goods); values of 0.3-0.5 are typical. Both measure rank-ordering power only, not calibration; so credit teams also check PD calibration by decile (observed vs predicted default rate), PSI for stability, and performance by vintage. Choose the metric by the decision: ranking (AUC/Gini/KS), alert volume constrained (precision@k, recall@k), probability use (Brier, calibration).

### A46. What are the main differences between level-wise, leaf-wise and symmetric tree growth, and what are the consequences for overfitting and spe...

Level-wise (XGBoost default, sklearn GBM): grow all nodes at one depth before the next; balanced trees, bounded by max_depth, predictable cost; may split nodes that bring little gain.

Leaf-wise / best-first (LightGBM, XGBoost lossguide): always split the leaf with the highest gain, producing deeper, asymmetric trees that reach lower loss with the same leaf count; faster convergence but greater overfitting risk on small data; control with num_leaves, min_data_in_leaf, max_depth.

Symmetric/oblivious (CatBoost): all nodes at a level share the same split condition; trees are balanced, inference is extremely fast (lookup by bit index), and the restricted structure acts as strong regularization, sometimes at the cost of fit on complex interactions.

Choose based on data size and latency: leaf-wise for large data, symmetric for low-latency serving and small noisy data, level-wise as a predictable default.

**A47. How would you explain the difference between global and local interpretability, and what are the regulatory implications (for example reason...**

Global interpretability explains the model as a whole: which features matter, the shape of the effects (PDP/ALE), feature interactions. Local interpretability explains a single prediction: why this applicant was declined (SHAP waterfall, LIME, counterfactuals).

Regulatory context: in lending, laws such as ECOA/Reg B in the US require adverse action notices stating the principal reasons for denial; GDPR gives rights regarding automated decisions and meaningful information about the logic; model risk guidance (SR 11-7) demands documentation, validation, and ongoing monitoring. Therefore complex models need robust local explanations mapped to human-readable reason codes (e.g. top-k negative SHAP contributions grouped into reason categories), monotonic constraints so that reasons make sense, stability checks of explanations, and fairness testing (disparate impact, equal opportunity across protected groups, with proxy-feature review). Often inherently interpretable models (scorecards, EBM) are preferred or used as challengers. Explanations must be reproducible, logged with the decision and consistent over time.

**A48. What is the difference between feature importance used for feature selection and recursive feature elimination (RFE)? When does RFE fail?**

Single-shot importance selection ranks features once and keeps the top-k. RFE trains a model, drops the least important feature(s) (a step of 1 or a fraction), refits, and repeats until the target count is reached, optionally choosing the number with cross-validation (RFECV). Refitting re-estimates importance after removing redundancy, which handles correlated features better than a single ranking.

Failures: expensive (many refits) for large feature sets; unstable when importance is noisy (correlated features swap places across runs); the importance measure used (MDI) may be biased; selection that is not nested within CV overfits the validation score; and it ranks marginal usefulness given the current set, which does not equal best subset. Mitigate by using permutation or SHAP importance, bigger steps, repeated runs with stability selection, and doing the whole procedure inside CV. Compare with Boruta, which tests features against randomized shadow copies.

**A49. How do you handle missing values for tree-based models compared with SVM or KNN?**

Trees: XGBoost and LightGBM learn a default direction for missing values at every split (the direction maximizing gain), so NaNs can be left in; this lets missingness itself be informative. CatBoost treats missing as min or max by setting. sklearn's random forest and histogram GBM support NaN in recent versions; older versions need imputation. Adding a missing-indicator flag can still help.

SVM and KNN cannot handle NaN: they require imputation (median/mode, KNN imputer, iterative imputer/MICE) and scaling. Imputation must be fitted on training folds only. Mean imputation distorts distances and variance; for KNN, imputation choice directly influences neighbors.

Prefer to understand the missingness mechanism (MCAR, MAR, MNAR); if missingness depends on the outcome (e.g. income not provided by risky applicants) keep an indicator and avoid blindly imputing. Document the handling so serving applies the same logic.

**A50. What is the effect of tree depth and number of leaves on the types of interactions a boosting model can learn, and how can you check whether...**

A tree of depth d can represent interactions among up to d features along a root-to-leaf path. Depth 1 (stumps) gives a purely additive model (like a GAM). Depth 2 captures pairwise interactions, depth 3 three-way, etc. Because boosting adds trees on residuals, many shallow trees can still build higher-order structure.

To check: (1) compare cross-validated performance of depth 1 vs depth 2 vs deeper; if depth 1 matches, interactions are weak and an additive (interpretable) model suffices. (2) Compute SHAP interaction values (TreeSHAP) and the Friedman H-statistic. (3) Inspect 2D partial dependence plots. (4) Use interaction_constraints in XGBoost/LightGBM to limit which features may interact, improving regularization and interpretability. Explainable Boosting Machines use depth-1 trees with selected pairwise terms to retain interpretability.

**A51. Write Python to compute Gini impurity and the best split (information gain) of a single numeric feature for a binary-labelled array, from sc...**

Approach: sort by feature, scan candidate thresholds at midpoints, maintain running class counts, and pick the threshold with maximum impurity reduction.

```
import numpy as np

def gini(counts):
    n = counts.sum()
    if n == 0:
        return 0.0
    p = counts / n
    return 1.0 - np.sum(p ** 2)

def best_split(x, y):
    order = np.argsort(x)
    x, y = x[order], y[order]
    n = len(y)
    total = np.bincount(y, minlength=2).astype(float)
    parent = gini(total)
    left = np.zeros(2)
    best_gain, best_thr = 0.0, None
    for i in range(n - 1):
        left[y[i]] += 1
        if x[i] == x[i + 1]:
            continue                        # cannot split between equal values
        right = total - left
        nl, nr = i + 1, n - i - 1
        child = (nl * gini(left) + nr * gini(right)) / n
        gain = parent - child
        if gain > best_gain:
            best_gain, best_thr = gain, (x[i] + x[i + 1]) / 2
    return best_thr, best_gain

x = np.array([2.1, 3.5, 1.2, 7.8, 6.5, 8.1])
y = np.array([0, 0, 0, 1, 1, 1])
print(best_split(x, y))   # threshold 5.0, gain 0.5
```

Complexity is O(n log n) for the sort plus O(n) for the scan. A full tree repeats this for each feature in the (random subset of) features at each node.

**A52. Show how to train a random forest with OOB scoring and compare impurity importance with permutation importance on a held-out set using sciki...**

```
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=5000, n_features=15, n_informative=5,
                           n_redundant=3, weights=[0.9, 0.1], random_state=0)
X = pd.DataFrame(X, columns=[f"f{i}" for i in range(15)])
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, test_size=0.25,
                                          random_state=0)

rf = RandomForestClassifier(n_estimators=400, max_features="sqrt",
                            min_samples_leaf=3, oob_score=True,
                            class_weight="balanced_subsample",
                            n_jobs=-1, random_state=0).fit(X_tr, y_tr)
print("OOB accuracy:", rf.oob_score_)

mdi = pd.Series(rf.feature_importances_, index=X.columns)
perm = permutation_importance(rf, X_te, y_te, scoring="average_precision",
                              n_repeats=10, random_state=0, n_jobs=-1)
perm = pd.Series(perm.importances_mean, index=X.columns)
print(pd.DataFrame({"mdi": mdi, "perm": perm}).sort_values("perm", ascending=False))
```

Interpretation: MDI is computed on training data and favors continuous features; permutation importance uses the test set and a business-relevant metric (average precision here). Large disagreements point to overfitting or correlated features.

**A53. Write code to train an XGBoost classifier with early stopping, stratified cross-validation and scale_pos_weight for an imbalanced dataset, r...**

```
import numpy as np
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import average_precision_score

def cv_xgb(X, y, seed=0):
    spw = (y == 0).sum() / (y == 1).sum()
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
    oof = np.zeros(len(y))
    best_iters = []
    for tr, va in skf.split(X, y):
        model = xgb.XGBClassifier(
            n_estimators=2000, learning_rate=0.05, max_depth=5,
```

```
            min_child_weight=3, subsample=0.8, colsample_bytree=0.8,
            reg_lambda=2.0, scale_pos_weight=spw, tree_method="hist",
            eval_metric="aucpr", early_stopping_rounds=100, n_jobs=-1,
            random_state=seed)
        model.fit(X[tr], y[tr], eval_set=[(X[va], y[va])], verbose=False)
        oof[va] = model.predict_proba(X[va])[:, 1]
        best_iters.append(model.best_iteration)
    print("OOF PR-AUC:", average_precision_score(y, oof))
    print("Median best iteration:", int(np.median(best_iters)))
    return oof, int(np.median(best_iters))
```

Notes: early stopping uses the validation fold, so the OOF score is slightly optimistic; for an unbiased estimate keep a separate final test set. Because scale_pos_weight distorts probabilities, recalibrate (isotonic or Platt) if probabilities are consumed downstream. The final model is refit on all training data with n_estimators from the median best iteration.

**A54. Implement stacking correctly with out-of-fold predictions (without using StackingClassifier) for a random forest, a gradient boosting model ...**

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

def fit_stack(X, y, X_test, seed=0, n_splits=5):
    base = [RandomForestClassifier(n_estimators=300, n_jobs=-1, random_state=seed),
            HistGradientBoostingClassifier(learning_rate=0.05, random_state=seed)]
    skf = StratifiedKFold(n_splits, shuffle=True, random_state=seed)
    oof = np.zeros((len(X), len(base)))
    test_meta = np.zeros((len(X_test), len(base)))
    for j, model in enumerate(base):
        for tr, va in skf.split(X, y):
            m = clone(model).fit(X[tr], y[tr])
            oof[va, j] = m.predict_proba(X[va])[:, 1]       # out-of-fold only
            test_meta[:, j] += m.predict_proba(X_test)[:, 1] / n_splits
    meta = LogisticRegression(C=1.0, max_iter=1000).fit(oof, y)
    return meta.predict_proba(test_meta)[:, 1], meta
```

Key points: the meta-learner is trained only on OOF predictions, so it never sees a base prediction on a row that base model trained on. Test predictions are the average of fold models. To evaluate honestly, wrap this in an outer CV or a separate holdout. Use logit-transformed probabilities for the meta-features if desired. If the data has groups or time order, swap in GroupKFold/TimeSeriesSplit.

**A55. Write a scikit-learn pipeline that tunes an RBF SVM (C and gamma) with a randomized search, including scaling, so that there is no leakage.**

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("svc", SVC(kernel="rbf", class_weight="balanced")),
])
param_dist = {
    "svc__C": loguniform(1e-2, 1e3),
    "svc__gamma": loguniform(1e-4, 1e1),
}
search = RandomizedSearchCV(
    pipe, param_dist, n_iter=40, scoring="roc_auc",
    cv=StratifiedKFold(5, shuffle=True, random_state=0),
    n_jobs=-1, random_state=0, refit=True)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
print("Test AUC:", search.score(X_test, y_test))
```

The scaler is inside the pipeline, so in each CV split it is fitted only on the training portion. Log-uniform sampling is used because C and gamma act multiplicatively. For large data use LinearSVC, SGDClassifier(loss='hinge') or kernel approximation (Nystroem) followed by a linear model, since kernel SVM scales quadratically.

**A56. Show how to use imblearn correctly with SMOTE inside cross-validation, and how to compare it against class weights.**

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(5, shuffle=True, random_state=42)

smote_pipe = ImbPipeline([
    ("smote", SMOTE(k_neighbors=5, random_state=42)),
    ("rf", RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)),
])
weight_rf = RandomForestClassifier(n_estimators=300, class_weight="balanced_subsample",
                                   random_state=42, n_jobs=-1)
plain_rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)

for name, model in [("smote", smote_pipe), ("weights", weight_rf), ("plain", plain_rf)]:
    s = cross_val_score(model, X, y, cv=cv, scoring="average_precision")
    print(name, s.mean().round(4), s.std().round(4))
```

The imblearn Pipeline applies SMOTE only when fitting each training fold and never to the validation fold, which prevents leakage. Scoring on average precision (PR-AUC) reflects the minority class. Frequently the plain model with a tuned threshold or class weights matches SMOTE, so always compare before adding complexity. Pre-resampling the full dataset before cross_val_score would be a leakage bug.

**A57. Show how to compute SHAP values for a trained LightGBM model, produce global and local plots, and map the top local contributions to reason ...**

```
import numpy as np
import pandas as pd
import shap
import lightgbm as lgb

model = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.05,
                           num_leaves=31, random_state=0).fit(X_train, y_train)

explainer = shap.TreeExplainer(model)
sv = explainer(X_valid)                 # shap.Explanation, log-odds units
if sv.values.ndim == 3:                 # some versions return one slice per class
    sv = sv[..., 1]

shap.plots.beeswarm(sv)                 # global: importance and direction
shap.plots.bar(sv)                      # mean |SHAP| ranking
shap.plots.waterfall(sv[0])             # local explanation of one row

def reason_codes(i, k=3):
    contrib = pd.Series(sv.values[i], index=X_valid.columns)
    return contrib.sort_values(ascending=False).head(k).index.tolist()

print(reason_codes(0))
```

SHAP values are additive in log-odds: base value + sum of contributions = raw model output. Positive values push toward the positive class (e.g. default). For adverse-action notices, take features with the largest risk-increasing contributions, map each to a plain-language reason, group correlated features into one reason, and log the codes with the decision. Use a representative background for interventional SHAP when features are correlated.

**A58. Write Optuna code to tune a LightGBM classifier with pruning, using stratified CV and PR-AUC as the objective.**

```
import optuna
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import average_precision_score

skf = StratifiedKFold(5, shuffle=True, random_state=0)

def objective(trial):
    params = {
        "n_estimators": 2000,
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "num_leaves": trial.suggest_int("num_leaves", 8, 128, log=True),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 200),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "subsample_freq": 1,
```

```
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.4, 1.0),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 50, log=True),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-3, 10, log=True),
        "random_state": 0, "verbose": -1,
    }
    scores = []
    for step, (tr, va) in enumerate(skf.split(X, y)):
        m = lgb.LGBMClassifier(**params)
        m.fit(X[tr], y[tr], eval_set=[(X[va], y[va])], eval_metric="average_precision",
              callbacks=[lgb.early_stopping(100, verbose=False)])
        scores.append(average_precision_score(y[va], m.predict_proba(X[va])[:, 1]))
        trial.report(np.mean(scores), step)
        if trial.should_prune():
            raise optuna.TrialPruned()
    return float(np.mean(scores))

study = optuna.create_study(direction="maximize",
                            sampler=optuna.samplers.TPESampler(seed=0),
                            pruner=optuna.pruners.MedianPruner(n_warmup_steps=2))
study.optimize(objective, n_trials=60)
print(study.best_params, study.best_value)
```

TPE builds a probabilistic model of good versus bad regions; the median pruner stops trials performing below the median of earlier trials at the same fold. Use log scales for rate-like parameters. Evaluate the final configuration on a separate untouched test set.

**A59. Write a function that computes PSI between a reference and a current sample for a numeric feature and flags drift.**

```
import numpy as np

def psi(reference, current, bins=10, eps=1e-6):
    reference = np.asarray(reference, dtype=float)
    current = np.asarray(current, dtype=float)
    # bin edges from reference quantiles
    edges = np.unique(np.quantile(reference, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    ref_pct = np.histogram(reference, bins=edges)[0] / len(reference)
    cur_pct = np.histogram(current, bins=edges)[0] / len(current)
    ref_pct = np.clip(ref_pct, eps, None)
    cur_pct = np.clip(cur_pct, eps, None)
    return float(np.sum((cur_pct - ref_pct) * np.log(cur_pct / ref_pct)))

def drift_flag(value):
    if value < 0.1:
        return "stable"
    if value < 0.25:
        return "moderate drift - investigate"
    return "significant drift - retrain or review"

rng = np.random.default_rng(0)
ref = rng.normal(0, 1, 10000)
cur = rng.normal(0.4, 1.2, 10000)
```

```
v = psi(ref, cur)
print(round(v, 3), drift_flag(v))
```

Using reference quantile bins guarantees roughly equal reference proportions per bin; the clip avoids log of zero. Run this for each feature and the model score, per segment, on a schedule, and alert on thresholds. Complement with KS tests and performance monitoring since PSI shifts do not always hurt accuracy.

**A60. Implement a simple gradient boosting regressor from scratch (squared error) using sklearn decision trees as weak learners.**

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

class SimpleGBR:
    def __init__(self, n_estimators=200, lr=0.1, max_depth=3, subsample=0.8, seed=0):
        self.n, self.lr, self.depth = n_estimators, lr, max_depth
        self.subsample, self.rng = subsample, np.random.default_rng(seed)
        self.trees, self.f0 = [], 0.0

    def fit(self, X, y):
        self.f0 = y.mean()                      # best constant for squared error
        pred = np.full(len(y), self.f0)
        for _ in range(self.n):
            resid = y - pred                    # negative gradient of 0.5*(y-F)^2
            idx = self.rng.choice(len(y), int(self.subsample * len(y)), replace=False)
            tree = DecisionTreeRegressor(max_depth=self.depth)
            tree.fit(X[idx], resid[idx])
            pred += self.lr * tree.predict(X)   # update on all rows
            self.trees.append(tree)
        return self

    def predict(self, X):
        out = np.full(X.shape[0], self.f0)
        for t in self.trees:
            out += self.lr * t.predict(X)
        return out
```

Each iteration fits a shallow tree to the residuals (pseudo-residuals for squared error), then adds a shrunken version of it. To generalize to other losses, replace resid with the negative gradient and optionally do per-leaf line search. Add early stopping using a validation set to choose the number of trees.

**A61. A payments company (card-not-present e-commerce) wants a real-time fraud detection model. Fraud is 0.15 percent of transactions, labels (cha...**

Step 1, frame the problem: binary classification with extreme imbalance, delayed and noisy labels, adversarial drift, strict latency, asymmetric cost. The business wants dollars saved, not accuracy.

Step 2, data and validation: build point-in-time features (velocity counts per card/device/IP/merchant over 1 min, 1 h, 24 h; amount deviation from customer history; device and geo mismatch; email age). Split by time (train on older months, validate on later months, test on the most recent), with a maturity gap equal to the label delay so immature labels are not treated as non-fraud. Group by card to avoid leakage.

Step 3, model choice: gradient boosted trees (LightGBM or XGBoost) because they handle mixed, sparse, skewed features and missing values, give state-of-the-art tabular accuracy, and score in well under a millisecond per row. Baseline with logistic regression for sanity. Consider a rules layer for hard blocks and an anomaly detector for new attack patterns.

Step 4, imbalance: train on the natural distribution or undersample the negatives (keep all fraud, 5-10 percent of legit, with weights) for speed; use scale_pos_weight moderately; avoid heavy SMOTE since fraud patterns are not smooth interpolations. Recalibrate probabilities if resampled.

Step 5, metric and threshold: PR-AUC and recall at fixed false-positive rate or review capacity. Choose the threshold from expected cost: with cost ratio 40:1 and calibrated probability, decline when $p > 1/41$ (about 2.4 percent), then adjust for review capacity and customer friction; use a three-way outcome (approve, step-up authentication, decline).

Step 6, explainability and ops: SHAP reason codes for analysts, monitoring of PSI, alert volume, chargeback rate, and champion-challenger deployment in shadow mode; retrain weekly with recency weighting; feedback loop awareness (declined transactions never reveal their true label, so keep a small randomized approve sample for unbiased evaluation).

**A62. A retail bank wants to replace its logistic regression application scorecard with a more accurate default-prediction model, but regulators r...**

Step 1, set the benchmark: keep the scorecard as the champion; define the target (e.g. 90+ days past due within 12 months), observation and performance windows, and the metrics: Gini/AUC, KS, calibration by decile, PSI, plus approval-rate/bad-rate tradeoff.

Step 2, handle reject inference/selection bias: the data only has outcomes for approved applicants; consider reject inference or at least acknowledge limitation and test on a through-the-door sample if possible.

Step 3, model candidates: (a) WoE-binned logistic regression as baseline; (b) gradient boosting with monotonic constraints on features with a known direction (income, delinquencies, utilization) and limited depth (3-5); (c) Explainable Boosting Machine as an inherently interpretable alternative. Compare out-of-time AUC/Gini and calibration. Typically GBM adds a few Gini points.

Step 4, explainability: TreeSHAP for local explanations; group correlated features into reason categories; top-4 risk-increasing contributions become adverse action reasons; stability checks across time; documentation for model risk (SR 11-7): conceptual soundness, benchmarking, sensitivity analysis, back-testing.

Step 5, fairness: test disparate impact and equal opportunity across protected classes, inspect proxy features (zip code), consider less-discriminatory alternatives.

Step 6, calibration and policy: calibrate to PD (isotonic/Platt) for pricing and capital; set cut-offs using expected loss and profit; a challenger rollout with A/B or shadow period.

Justification: the GBM with constraints gives accuracy gain, remains monotone and explainable, and can pass validation; if the gain is small versus the scorecard, keep the scorecard.

**A63. A grocery retailer needs weekly demand forecasts for 20,000 SKUs across 500 stores, with many intermittent (zero-heavy) series, promotions, ...**

Step 1, framing: a global model across all series (one LightGBM with series identifiers and features) beats per-series models because most series are short or sparse and share structure. Multi-horizon strategy: direct (a model per horizon) or one model with horizon as a feature.

Step 2, features: lags and rolling stats (lag 1,2,4,52 weeks; rolling mean/std), calendar (week of year, holidays, days to holiday), price and promotion flags, price relative to baseline, store/SKU attributes, category embeddings or native categorical, weather and local events, stock-out indicators (censored demand). Only use features known at forecast time; future-known covariates (promotions) can be used for target dates.

Step 3, objective: sales are non-negative, skewed and zero-inflated: use Tweedie or Poisson objective (log link), or a log1p target; for service-level decisions train quantile models (e.g. 0.8 or 0.9) rather than the mean.

Step 4, trend/extrapolation: trees cannot extrapolate, so forecast ratios to a baseline or detrended series, or use a hybrid with a statistical model; retrain regularly.

Step 5, validation: rolling-origin backtests with several cutoff dates including peak seasons; metrics like WAPE, bias, RMSSE (scale-free), and pinball loss for quantiles; evaluate by segment (fast vs intermittent movers). Compare with naive seasonal baseline, ETS/ARIMA, and Croston/TSB for intermittent series.

Step 6, deploy: batch weekly scoring, hierarchical reconciliation across store/region/category if needed, monitor forecast bias and drift, override process for new products (cold start by attribute-based similar SKU).

**A64. A telecom operator wants a churn model. The first random forest has 0.97 AUC on validation, but a colleague finds that the top feature is "r...**

Diagnosis: this is target leakage. The retention offer is made to customers who already signaled intent to leave (for example called to cancel), so the feature is a consequence of the outcome or a near-proxy for it; it will not be available, or will not carry the same meaning, at prediction time. An AUC of 0.97 for churn is implausible and is a red flag.

Steps: (1) Define the prediction moment precisely (e.g. score every customer on day 1 of the month to predict churn in the next 30 days). (2) Audit every feature for timestamp: only data available strictly before the scoring date. Build features using point-in-time snapshots. (3) Remove retention_offer_accepted and its relatives (cancellation requests, final-bill flags, number of calls to cancellation team). (4) Re-run with time-based splits (train on earlier months, validate on later ones) and GroupKFold by customer. (5) Check suspicious features with single-feature AUC and importance rankings; use adversarial validation between training and live scoring data. (6) Expect a realistic AUC of 0.75-0.85, then evaluate lift in the top decile and the campaign ROI. (7) Add a data review checklist and a shadow-mode run before launch to compare offline with online performance.

**A65. A health-tech startup has 3,000 patient records with 12,000 genomic and lab features and a rare disease label (4 percent positive). They ask...**

Challenges: p much larger than n, class imbalance, high variance, risk of overfitting and optimistic validation, need for interpretability for clinicians.

Step 1, validation: repeated stratified K-fold or nested CV because a single split is noisy with about 120 positives; group by patient/site if multiple samples exist; keep a truly held-out set or external cohort if available. All preprocessing and feature selection must live inside the CV pipeline.

Step 2, dimension reduction: filter variance and univariate tests inside folds, remove highly correlated features, optionally PCA or pathway-level aggregation; use embedded selection with L1/elastic-net logistic regression.

Step 3, models: regularized logistic regression (elastic net) as primary baseline, linear SVM (works well in high dimensions), random forest/ExtraTrees with max_features tuned and balanced_subsample, and small-depth

gradient boosting with strong regularization. RBF SVM or KNN are weak in 12,000 noisy dimensions without selection.

Step 4, imbalance: class weights and PR-AUC/recall at fixed specificity rather than accuracy; avoid SMOTE in high-dimensional space where nearest neighbors are unreliable.

Step 5, report with confidence intervals (bootstrap), calibrate probabilities, use SHAP or coefficients with stability selection to report robust biomarkers.

Recommendation: elastic-net logistic regression and a regularized ensemble, compared by nested-CV PR-AUC; choose the simpler one if differences are within noise.

**A66. An insurance company wants to price auto policies by predicting claim cost per customer. Data: 2 million policies, 93 percent have zero clai...**

Step 1, understand the target: pure premium = expected claim frequency x expected severity. Most rows are zero, severity is right-skewed, exposure (policy duration) varies.

Step 2, options: (a) frequency-severity split: Poisson (or negative binomial) model for claim count with log(exposure) as offset, and Gamma model for severity on claim rows; multiply the two. (b) Tweedie model directly on loss cost with power between 1 and 2 handles the point mass at zero and continuous positive tail.

Step 3, algorithms: GLMs are the regulatory standard and a baseline; gradient boosting (LightGBM/XGBoost with objective poisson, gamma or tweedie) captures nonlinearity and interactions and typically lowers deviance. Use exposure as weight or offset (init_score) rather than feature.

Step 4, constraints and governance: monotonic constraints (e.g. vehicle value), limit interaction depth, explainability with SHAP or partial dependence to satisfy regulators, avoid prohibited rating factors and proxies.

Step 5, validation: out-of-time split by policy year, metrics such as Poisson/Gamma/Tweedie deviance, Gini on lift curve (ordered Lorenz curve), and calibration by decile (actual vs predicted loss ratio) because pricing needs accurate levels, not just ranking. Cap or treat large claims separately (large-loss loading) to stabilize severity.

Step 6, deployment: calibrate overall to the book's loss ratio, monitor drift, and test via champion-challenger.

**A67. A ride-hailing company deploys an ETA/price model and after two months its performance declines. Offline validation looked fine. How do you ...**

Step 1, quantify: compare current performance (MAE, calibration, business KPIs) with the baseline by segment (city, hour, vehicle type); check whether the decline is gradual (drift) or abrupt (pipeline break, release).

Step 2, rule out engineering issues: schema or unit changes, null rates, feature pipeline delays, training/serving skew (offline features computed differently than online), model version rollbacks, logging gaps.

Step 3, data drift analysis: PSI and KS per feature, prediction distribution drift, adversarial validation between training period and last two weeks; identify drifting features (new city launch, road closures, fuel price change, seasonality, holiday).

Step 4, concept drift: check whether the relationship between features and target changed using recent labeled data; compare residuals vs time; examine feedback loops (the model's prices change demand, which changes the data).

Step 5, remediate: if pipeline bug, fix and backfill; if covariate or concept drift, retrain with recent data (sliding or recency-weighted window), add features capturing the new regime (events, weather), possibly segment-specific models; test the challenger offline on out-of-time data, shadow it, then roll out via canary.

Step 6, prevent recurrence: automated drift and performance dashboards with alerts, scheduled retraining, data validation (Great Expectations), post-mortem, and holdout groups for unbiased evaluation.

**A68. An e-commerce marketplace wants to predict seller default on payouts using a feature that has 40,000 distinct categories (seller_category_co...**

Why it failed: 40,000 dummy columns are sparse; most categories have few rows, so trees split on rare dummies and memorize individual sellers' outcomes; memory and training time inflate.

Approach: (1) Consolidate: group rare codes into "other" (e.g. frequency < 50) and use hierarchical roll-ups (code to category to department) as additional lower-cardinality features. (2) Encode: out-of-fold smoothed target encoding (m-estimate) or CatBoost ordered target statistics, computed within each training fold; add count/frequency encoding. Time-aware: compute encodings from data before the prediction date. (3) Use native categorical handling in CatBoost (best) or LightGBM with cat_smooth and min_data_per_group, which limit overfitting on rare categories. (4) Consider learned embeddings if a neural model is used in an ensemble. (5) Validate by time and group split, comparing against dropping the feature; check that the feature importance is plausible and that performance on unseen categories (cold start) is acceptable, handling unseen codes with the prior mean.

Pick CatBoost as primary because it was designed for this case and needs less manual encoding; verify the gain with out-of-time AUC and PR-AUC.

**A69. A subscription streaming company trained an XGBoost churn model with 500 trees at depth 10 and learning rate 0.3. Training AUC is 0.99 and v...**

Diagnosis: large train-validation gap with a deep, fast-learning model indicates high variance and overfitting.

Step 1: lower complexity: max_depth 3-6, min_child_weight 5-20 (larger on noisy data), gamma > 0.

Step 2: lower learning_rate to 0.03-0.1 and use early_stopping_rounds on a validation fold (watch aucpr/logloss) so n_estimators is learned.

Step 3: stochastic regularization: subsample 0.6-0.8, colsample_bytree 0.5-0.8, plus reg_lambda (1-10) and reg_alpha.

Step 4: data checks: leakage (features computed with future info would show the opposite pattern, but check for ID-like features or duplicated customers across folds), noisy labels, high-cardinality features; use GroupKFold by customer.

Step 5: feature pruning of unstable or noisy features via permutation importance on validation data.

Step 6: re-evaluate with learning curves; if the gap persists, collect more data, simplify features, or blend with a bagged model. Expect training AUC around 0.85-0.9 and validation AUC to rise to 0.80 or so; the point is better validation, not lower training. Keep a final test set unused during tuning.

**A70. A bank's anti-money-laundering team has 50 confirmed laundering cases among 5 million accounts, and investigators can review only 500 alerts...**

Step 1, reframe: with 50 positives (0.001 percent), supervised learning alone is fragile. Combine rules, unsupervised anomaly detection and a supervised model trained on investigator outcomes (alerts reviewed, including false positives) which provide many more labels than confirmed cases.

Step 2, features: transaction graph features (counterparties, fan-in/fan-out, cycles, shortest paths), velocity and structuring patterns (amounts just below thresholds), geography and counterparty risk, peer-group deviation (compare an account to similar customers), changes in behavior over time.

Step 3, models: isolation forest, autoencoders or peer-group z-scores for unlabeled anomalies; gradient boosting on investigator-labeled alerts with strong regularization, class weights, and simple depth; graph-based embeddings or GNN if graph structure is rich. Ensemble scores by rank averaging.

Step 4, metrics and capacity: precision@500 and recall@500 per week, lift over the current rules, and investigator hit rate; bootstrap confidence intervals because of few positives; repeated time-based CV.

Step 5, explainability: SHAP and typology narratives, required by regulators and investigators; sanity-check with domain experts.

Step 6, feedback loop: investigator outcomes retrain the model; beware selection bias (only alerts get reviewed) and use random sampling of unreviewed accounts to estimate missed cases. Monitor drift as launderers adapt.

**A71. A SaaS company serves a lead-scoring model through an API with a 20 ms p99 latency limit. The current stacked ensemble of 5 models (XGBoost,...**

Step 1, measure the gain: check what stacking adds over the best single model on out-of-time data. If the lift is under about 0.5 AUC point, drop the stack.

Step 2, prune: remove models with high correlation to others and small meta-weights (SVM and random forest with large trees are typically slowest and add little over GBMs); keep one or two GBMs.

Step 3, optimize inference: reduce trees and depth (retrain with higher learning rate), use compiled inference (Treelite, ONNX Runtime, LightGBM C API), batch requests, avoid Python overhead, precompute expensive features offline in a feature store, quantize or cache scores for repeated leads.

Step 4, distillation: train a single fast model (small GBM or logistic regression) on the stack's predictions (soft labels) to retain most accuracy.

Step 5, architecture: asynchronous scoring where real-time is not needed, two-stage cascade (cheap model filters most leads, the heavy model re-ranks the top fraction).

Step 6, validate: compare the new model offline and online (A/B) for both quality and latency percentiles, and monitor p99 under load.

Outcome: a single optimized LightGBM or distilled model meeting 20 ms with minimal accuracy loss.

**A72. A marketing team wants to target a promotion at customers most likely to respond. Only 5 percent of customers respond, and each promotion co...**

Step 1, objective: maximize expected profit, not accuracy. Contact a customer if $p * 30 > 2$, i.e. $p > 6.7$ percent (if probabilities are calibrated). Base rate is 5 percent so a good model must find segments above 6.7 percent.

Step 2, model: gradient boosting or regularized logistic regression on RFM, engagement and campaign history; calibrate with isotonic or Platt. Evaluate by lift, cumulative gains and profit curve over the sorted list.

Step 3, causal consideration: customers who would buy anyway waste the promotion; the right goal is uplift (incremental response). Use an A/B holdout and uplift modeling (two-model approach, class-transformation, causal forests) to target persuadables, not just likely responders.

Step 4, validation: out-of-time split, profit curve at several budget sizes, bootstrap interval of profit.

Step 5, rollout: run a campaign with a random control group to measure actual incremental profit; monitor response rate by decile and calibration; update the model with campaign outcomes.

Recommendation: calibrated GBM for response with threshold at the profit break-even, then upgrade to uplift modeling once a randomized test exists.

**A73. A bank's fraud model performs well offline (PR-AUC 0.45) but in production the same model catches far fewer frauds, and analysts suspect fra...**

Do not just retrain; investigate first.

Step 1, verify data and features: compare production feature distributions against training (PSI, null rates), check for training-serving skew, e.g. features computed with batch aggregates offline but delayed online, and time-zone or unit differences.

Step 2, check the labels: fraud labels come with delay, so recent performance is understated because many frauds are not yet labeled; compute performance only on matured cohorts, or estimate with expected-delay corrections.

Step 3, evaluate on out-of-time data: compare performance by month, attack type, channel, merchant category; determine whether a specific new pattern (e.g. account takeover through a new device type) accounts for the drop.

Step 4, differentiate drift types: prediction drift (score distribution shift), covariate drift (new devices, geos), concept drift (same features now map to fraud differently).

Step 5, action: if skew or bug, fix pipeline; if drift, retrain with recent data (weighting recent fraud heavily), add features for the new tactic (device fingerprint reputation, behavioral biometrics), update rules for immediate containment; evaluate challenger in shadow mode before release.

Step 6, process: monitoring by mature cohort, alerts on score drift, faster feedback from analysts, and scheduled retraining. Answer to the PM: retrain is likely part of the fix but only after confirming the root cause, otherwise the new model inherits the same problem.

**A74. A real-estate platform wants to predict house prices with gradient boosting. The data has price in a wide range (50k to 20M), and the model ...**

Step 1, target transformation: prices are right-skewed with multiplicative noise. Train on log(price) (or use a gamma objective) so errors are relative; invert with exp and apply a bias correction (smearing) if the mean prediction is needed. Alternatively predict price per square meter.

Step 2, metrics: use MAPE/median absolute percentage error and RMSLE rather than RMSE, which is dominated by expensive houses; report errors by price band.

Step 3, extrapolation: trees cap predictions at the highest training leaves. Predict price relative to a local baseline (neighborhood median price per sqm) so luxury listings in new regions scale properly; use a hybrid

(linear/hedonic model for trend plus GBM on residuals) or LightGBM linear_tree; add log area and interaction features.

Step 4, features: geospatial (lat/lon clusters, distance to transit, school ratings), neighborhood aggregates computed out-of-fold, time index to capture market trends with time-based splits.

Step 5, uncertainty: quantile models (0.1, 0.5, 0.9) for price ranges; conformal prediction for calibrated intervals.

Step 6, validation: out-of-time test; check residuals vs predicted, by segment; monitor drift with the market.

**A75. A hospital wants to deploy a sepsis early-warning model. In validation AUC is 0.88, but clinicians complain about alert fatigue. How do you ...**

Step 1, understand the clinical workflow: define the action triggered by an alert, capacity of the rapid-response team, and the cost of a missed case versus a false alarm. A high AUC does not imply acceptable precision at a low prevalence (sepsis maybe 2-5 percent of patient-hours).

Step 2, evaluate at the operating point: examine PR curve, positive predictive value, number needed to evaluate (1/PPV), alerts per 100 patients per day, and lead time before onset. Choose the threshold to achieve PPV and alert volume the staff can sustain while keeping sensitivity above a clinical floor, e.g. sensitivity 70 percent with fewer than X alerts per nurse per shift.

Step 3, reduce noise: patient-level rather than hour-level alerts, suppression windows (no repeat alerts for N hours), tiered risk levels, require trend persistence, combine with clinical rules.

Step 4, calibrate probabilities and validate prospectively and on other hospital sites (temporal and external validation); check subgroup performance and fairness.

Step 5, explainability: show top contributing vitals/labs (SHAP) so clinicians can verify and trust.

Step 6, deployment: silent shadow mode, measure outcomes (time to antibiotics, mortality), monitor drift when protocols change, and collect clinician feedback to retrain.

**A76. An ad-tech company predicts click-through rate with 200 million rows, extremely sparse categorical features with millions of values, and a 0...**

Step 1, characterize: huge scale, high-cardinality sparse features, strong imbalance, probabilities needed for bidding (expected value = pCTR x value), so calibration is crucial, and low latency and frequent updates are needed.

Step 2, model options: logistic regression with hashed features and feature crosses (trained with FTRL-Proximal online or SGD) is the classic scalable baseline; factorization machines / field-aware FM capture pairwise interactions in sparse space; deep CTR models (Wide and Deep, DeepFM, DCN) with embeddings for categorical features; GBDT (LightGBM) to create dense interaction features or for dense-heavy subsets, used alone or as feature transformer feeding LR.

Step 3, why not SVM or KNN: kernel SVM cannot scale to 200 million rows; KNN prediction is too slow.

Step 4, data handling: negative downsampling (e.g. keep 10 percent of non-clicks) with probability correction $p = p_s/(p_s + (1 - p_s)/w)$, where w is the sampling rate; time-based validation; log loss and calibration, AUC, and business metrics like revenue per mille.

Step 5, infra: distributed training (Spark, Parameter server, GPU), embedding tables with hashing, daily or hourly incremental updates, a feature store, and online serving with latency budgets of a few milliseconds.

Step 6, monitoring: calibration by segment, drift, and exploration traffic to avoid feedback loops.

**A77. A manufacturer wants to predict equipment failures from sensor data. Failures are rare (0.3 percent of machine-days), the data is time order...**

Step 1, define the target: failure within the next H days (e.g. 7) given data up to today, so label windows must not overlap in ways that leak; remove records after failure and handle censoring.

Step 2, features: rolling statistics (mean, std, min, max, slope, FFT bands) over several windows, time since last maintenance, usage cycles, deviation from the machine's own baseline (z-score vs its history), cross-sensor ratios, operating conditions.

Step 3, validation: time-ordered splits with an embargo equal to the prediction horizon, grouped by machine (some machines wholly held out) to test generalization to new machines; report both. Metrics: PR-AUC, recall at a maintenance-capacity-constrained alert rate, lead time distribution, expected cost (unplanned downtime vs unnecessary maintenance).

Step 4, models: gradient boosting with class weights as a primary, survival models (Cox, random survival forest) for time-to-failure, anomaly detectors (isolation forest) when failures are too rare; a rule-based baseline from engineers.

Step 5, interpretability: SHAP to show which sensors drive the alert; engineers validate physically.

Step 6, deployment: batch daily scoring, alert triage, drift monitoring when equipment is serviced or firmware changes, retrain as failure labels accumulate.

**A78. A data scientist reports that SMOTE improved the F1 score of a random forest from 0.42 to 0.71 on the test set. You suspect something is wro...**

A jump that large from SMOTE is suspicious. Checks:

1. Order of operations: was SMOTE applied to the entire dataset before the train/test split or before cross-validation? Then synthetic samples derived from test points leak into training and the test contains near-copies of training points. Rerun with an imblearn pipeline so only training folds are resampled.

2. Was the test set itself resampled? Its class distribution must remain natural; F1 on a balanced test set is not comparable.

3. Duplicates: SMOTE or random oversampling before splitting creates near duplicates across splits.

4. Metric: compare against the baseline with threshold tuning and class weights, using PR-AUC; part of the F1 gain could be just a threshold shift, which can be achieved without SMOTE.

5. Variance: check across repeated stratified CV with confidence intervals.

6. Prior shift: SMOTE changes predicted probabilities, so any downstream decision needs recalibration.

Conclusion: only trust the result after reproducing it with proper pipeline-based resampling on an untouched natural-distribution holdout.

**A79. A bank is migrating a credit-card default model to production with monthly scoring. Describe your monitoring and retraining plan, including ...**

Step 1, inputs: data quality checks per batch (schema, null rates, ranges, referential integrity); feature PSI vs the development sample (alert at 0.1, action at 0.25); new categories.

Step 2, outputs: score distribution PSI, average predicted PD vs overall approval and bad rates, and score band migration.

Step 3, performance: outcomes mature over 12 months, so use vintage analysis and early-delinquency proxies (first-payment default, 30+ DPD at 3 months) while waiting; once matured, track Gini, KS, calibration by decile (observed vs predicted default; binomial test), and Hosmer-Lemeshow-style checks. Alert thresholds: Gini drop of more than 10 percent relative, calibration out of tolerance in several deciles.

Step 4, segments and fairness: performance across product, region, and protected-class proxies; stability of reason codes.

Step 5, retraining: scheduled annual redevelopment, with triggered review on breaches; challenger models trained on updated data, backtested on out-of-time sets, validated by an independent model-risk team; staged rollout and policy cut-off reassessment.

Step 6, governance: model inventory, documentation, change control, approval committees, audit trail of predictions, versions and explanations, and a fallback (previous model or scorecard) with a rollback procedure.

**A80. A recommender team at a news publisher uses KNN on user embeddings to find similar articles. Latency is high at 100 million items, and quali...**

Step 1, latency: exact KNN is O(n*d) per query. Replace with approximate nearest neighbor search: HNSW graphs, IVF-PQ in FAISS, ScaNN or Annoy. Tune recall vs latency (efSearch, nprobe), shard the index, use product quantization to cut memory, and cache popular queries. Reduce embedding dimension (PCA or trained lower-dimension embeddings) and use inner product or cosine with normalized vectors.

Step 2, quality: check the distance metric matches how embeddings were trained; combine ANN retrieval (candidate generation of a few hundred) with a second-stage ranker (gradient boosting or neural ranker using click features, freshness, popularity) trained with lambdarank.

Step 3, cold start: for new users use content-based signals (location, device, onboarding choices), popularity and contextual bandits for exploration; for new items use content embeddings of text. Blend collaborative and content features.

Step 4, evaluation: offline recall@k, NDCG, coverage and diversity with time-based splits, then online A/B tests measuring clicks, dwell time and retention.

Step 5, operations: incremental index updates, index rebuild schedule, monitoring of recall via a brute-force sample, and drift in embedding distributions after model retraining (version embeddings with indexes).

## Advanced ML: Unsupervised, Deep Learning, Time Series and Recommenders

**A81. Explain how the K-means algorithm works, what objective it optimises, and list its main limitations.**

K-means partitions n points into k clusters by minimising the within-cluster sum of squares (inertia): the sum over clusters of squared Euclidean distances from each point to its cluster centroid. It uses Lloyd's algorithm: (1) initialise k centroids; (2) assignment step: assign every point to the nearest centroid; (3) update step: recompute each centroid as the mean of its assigned points; (4) repeat until assignments stop changing or inertia improvement falls below a tolerance. Each step cannot increase the objective, so it converges, but only to a local minimum, so results depend on initialisation. Complexity is roughly O(n * k * d * iterations).

Limitations: k must be chosen in advance; sensitive to initialisation (use k-means++ and multiple restarts); assumes roughly spherical, similarly sized, similarly dense clusters because of Euclidean distance and mean centroids; sensitive to outliers (means get pulled) and to feature scale (standardise first); not suitable for categorical features (use k-modes or k-prototypes); suffers in very high dimensions where distances

concentrate; hard assignments only (no uncertainty). Alternatives: k-medoids for robustness, DBSCAN for arbitrary shapes, GMM for elliptical clusters and soft assignments.

**A82. How do you choose the number of clusters k? Compare the elbow method, silhouette analysis and the gap statistic.**

Elbow method: plot inertia against k and look for the point where gains flatten. It is easy but subjective, and many datasets show no clear elbow because inertia always decreases with k.

Silhouette analysis: for each point compute a = mean distance to other points in its own cluster and b = mean distance to points in the nearest other cluster; silhouette s = (b - a) / max(a, b), in [-1, 1]. Average silhouette across k values; choose the k with the highest score, and also inspect per-cluster silhouette plots for clusters with many negative values or very uneven widths. It favours compact, well separated convex clusters and costs O(n^2).

Gap statistic: compares log(inertia) with its expectation under a reference null distribution (uniform random data over the same range); pick the smallest k where gap(k) >= gap(k+1) - standard error. More principled but computationally heavier.

In practice combine these with stability analysis (re-cluster on bootstrap samples and compare with adjusted Rand index) and, most importantly, business usefulness: segments must be actionable, interpretable and of a workable size. For GMMs use BIC instead.

**A83. What is the silhouette coefficient, what are its strengths and weaknesses, and what other internal or external clustering metrics exist?**

The silhouette of a point is s = (b - a) / max(a, b), where a is the mean intra-cluster distance and b is the mean distance to the nearest neighbouring cluster. Values near 1 mean the point is well matched to its cluster, near 0 means it sits on a boundary, and negative values suggest it may be in the wrong cluster. The overall score is the mean over points.

Strengths: needs no ground truth, bounded and interpretable, works with any distance metric, and per-sample values diagnose individual clusters. Weaknesses: biased towards convex, compact clusters (so it penalises valid density-based clusters from DBSCAN), O(n^2) cost, and it is not comparable across different metrics or feature spaces.

Other internal metrics: Calinski-Harabasz (ratio of between to within cluster dispersion; higher is better), Davies-Bouldin (average similarity of each cluster to its most similar one; lower is better), density-based validity (DBCV) for arbitrary shapes. External metrics, when labels exist: adjusted Rand index, normalised mutual information, homogeneity, completeness and V-measure, Fowlkes-Mallows.

**A84. Explain agglomerative hierarchical clustering. What are the common linkage criteria and when would you prefer it to K-means?**

Agglomerative clustering starts with every point as its own cluster and repeatedly merges the two closest clusters until one remains, producing a dendrogram. Cutting the dendrogram at a chosen height gives a flat clustering, so k need not be fixed upfront. Linkage defines the inter-cluster distance: single (minimum pairwise distance; can chain and find elongated shapes but is noise sensitive), complete (maximum pairwise distance; compact clusters, sensitive to outliers), average (mean pairwise distance; a compromise), Ward (merge that minimises the increase in within-cluster variance; behaves like K-means, needs Euclidean distance, usually the best default), and centroid.

Prefer it when you want a hierarchy (for example product taxonomies or nested customer segments), when the dendrogram helps choose k, when you want to use arbitrary distance matrices (for example Gower distance with mixed data), or with small datasets. Drawbacks: O(n^2) memory and O(n^2 log n) to O(n^3) time, so it does not scale to large n, and merges are greedy and irreversible.

**A85. Describe DBSCAN: its parameters, point types, advantages and weaknesses, and how you would choose eps and min_samples.**

DBSCAN is density-based. Parameter eps is the neighbourhood radius and min_samples is the number of points needed within eps to form a dense region. A core point has at least min_samples points (including itself) within eps; a border point is within eps of a core point but is not core itself; a noise point is neither. Clusters are formed by connecting core points that are within eps of each other (density-reachability) and attaching their border points. Points left over are labelled noise (-1).

Advantages: no need to specify the number of clusters, finds arbitrarily shaped clusters, explicitly identifies outliers, and is robust to them. Weaknesses: a single eps cannot handle clusters of varying density (HDBSCAN addresses this), sensitive to eps and the distance metric, degrades in high dimensions, border points can be order dependent, and neighbour queries cost O(n log n) with an index (O(n^2) worst case).

Choosing parameters: min_samples is often >= dimensions + 1 (commonly 2 * d for larger or noisy data); eps comes from a k-distance plot: sort every point's distance to its k-th nearest neighbour (k = min_samples) and choose the eps at the knee. Always scale features first.

**A86. How do Gaussian Mixture Models differ from K-means? Explain the EM algorithm and how to choose the number of components.**

A GMM models data as a mixture of k Gaussians, p(x) = sum_k pi_k N(x | mu_k, Sigma_k). Unlike K-means it learns covariances (elliptical clusters of different sizes and orientations) and gives soft assignments (posterior responsibilities). K-means is the limiting case of a GMM with equal spherical covariances going to zero variance and hard assignments.

EM alternates: E-step computes responsibilities r_ik = pi_k N(x_i | mu_k, Sigma_k) / sum_j pi_j N(x_i | mu_j, Sigma_j) using current parameters; M-step re-estimates pi_k = (sum_i r_ik) / n, mu_k as the responsibility-weighted mean, and Sigma_k as the responsibility-weighted covariance. Each iteration does not decrease the log-likelihood, converging to a local maximum, so initialise with K-means and use multiple restarts. Pitfalls: singular covariances when a component collapses on one point (add a regularisation term to the diagonal), and the covariance type choice (full, tied, diagonal, spherical) trades flexibility for parameter count.

Choosing k: likelihood always increases with k, so use penalised criteria: Bayesian information criterion (BIC = p ln n - 2 ln L, usually preferred, consistent) or Akaike information criterion (less penalty), or cross-validated held-out likelihood. Variational Bayesian GMMs can prune unneeded components automatically. GMMs also support density-based anomaly detection (low likelihood points).

**A87. Explain PCA mathematically. How is it computed, what do the components and explained variance mean, and what are its assumptions?**

PCA finds orthogonal directions of maximal variance. Steps: centre (and usually standardise) the data matrix X; compute the covariance matrix C = X^T X / (n - 1); eigen-decompose C = V Lambda V^T. Eigenvectors (columns of V) are the principal components (loadings); eigenvalues are the variances along them. Project with Z = X V_k for the top k. Equivalently use the SVD of X = U S V^T, which is more numerically stable; the eigenvalues

are s_i^2 / (n - 1). The explained variance ratio of component i is lambda_i / sum(lambda). PCA minimises reconstruction error (squared) among all rank-k linear projections.

Assumptions and caveats: linear structure; variance equals information (so high variance noise features dominate unless standardised); sensitive to outliers and scale; components are hard to interpret; components are uncorrelated but not necessarily independent (unless Gaussian); it is unsupervised so the top components need not predict the target. Fit on the training set only to avoid leakage. Variants: kernel PCA for non-linear structure, sparse PCA for interpretable loadings, incremental and randomised PCA for large data.

### A88. Compare PCA and t-SNE. When would you use each?

PCA is a linear, deterministic method preserving global variance structure; it gives an explicit projection that can be applied to new data, is fast, and the axes are interpretable via loadings. t-SNE is a non-linear, stochastic method that converts pairwise distances to neighbour probabilities (Gaussian in high dimension, Student-t with one degree of freedom in low dimension) and minimises the KL divergence between them. It preserves local neighbourhoods and reveals clusters but not global geometry. Use PCA for preprocessing, noise reduction, decorrelation, feature compression and as a first look; use t-SNE (or UMAP) for 2D visualisation of cluster structure in embeddings. Typical workflow: PCA to 30-50 dimensions to denoise and speed up, then t-SNE. t-SNE is not suitable as a feature extractor for models, has no out-of-sample transform in the standard implementation, costs O(n^2) (O(n log n) with Barnes-Hut), and results change with the random seed and perplexity.

### A89. What are the common pitfalls when interpreting t-SNE plots, and what is perplexity?

Perplexity is roughly the effective number of neighbours each point considers; it sets the Gaussian bandwidth per point via a binary search so the conditional distribution has the target entropy. Typical range is 5-50; low values emphasise very local structure and can fragment clusters, high values capture more global structure.

Pitfalls: (1) cluster sizes in the plot do not reflect real sizes or densities because bandwidths adapt locally; (2) distances between clusters are not meaningful, and relative positions can change between runs; (3) random-looking noise can appear as clusters at low perplexity; (4) too few iterations gives a pinched, unconverged map (use enough steps and early exaggeration); (5) different seeds give different layouts, so run several; (6) the crowding problem is mitigated but not eliminated; (7) you cannot reuse it to embed new points reliably. Good practice: try several perplexities, verify with another method such as UMAP, colour by known labels, and do not use t-SNE coordinates for downstream clustering claims.

### A90. How does UMAP work and how does it compare to t-SNE?

UMAP builds a weighted k-nearest-neighbour graph in the high-dimensional space, converting distances to fuzzy membership strengths (using local connectivity so every point is linked to at least its nearest neighbour), which is a fuzzy topological representation grounded in Riemannian geometry and simplicial sets. It then optimises a low-dimensional layout to have a similar fuzzy graph by minimising cross-entropy with attractive and repulsive forces using stochastic gradient descent. Key parameters: n_neighbors (local versus global balance) and min_dist (how tightly points pack).

Versus t-SNE: usually much faster and scales to millions of points; tends to preserve more global structure (though inter-cluster distances are still not strictly reliable); provides a transform method for new points; can embed to more than 2 dimensions and be used as preprocessing for clustering, and supports supervised and

semi-supervised variants and arbitrary metrics. Like t-SNE, it is stochastic, can create misleading apparent clusters, and the parameters materially change the picture, so interpret with care.

**A91. What is Linear Discriminant Analysis (LDA) as a dimensionality reduction technique and how does it differ from PCA?**

Linear Discriminant Analysis is a supervised method that finds linear projections maximising class separability: it maximises the ratio of between-class scatter S_B to within-class scatter S_W, solving the generalised eigenproblem S_W^-1 S_B w = lambda w. With C classes it yields at most C - 1 discriminant directions. As a classifier it assumes each class is Gaussian with a shared covariance matrix, giving linear decision boundaries (quadratic discriminant analysis relaxes the shared covariance).

Difference from PCA: PCA is unsupervised and maximises total variance regardless of labels; LDA uses labels and maximises class discrimination, so it often gives better features for classification, especially when the high-variance directions are irrelevant. Limitations: needs labels, limited to C - 1 dimensions, assumes Gaussian classes with equal covariance, can overfit when features exceed samples (use shrinkage / regularised LDA), and is sensitive to outliers. Note this LDA is unrelated to Latent Dirichlet Allocation used for topic modelling, a common source of confusion in interviews.

**A92. Explain how Isolation Forest detects anomalies and compare it with Local Outlier Factor and One-Class SVM.**

Isolation Forest builds many random trees by repeatedly choosing a random feature and a random split value between its min and max. Anomalies are few and different, so they are isolated in fewer splits (short average path length). The anomaly score is s(x) = 2^(-E[h(x)] / c(n)) where h is path length and c(n) normalises by the average path length of an unsuccessful search in a binary search tree; scores near 1 are anomalous. It uses subsampling (for example 256 points), is linear in n, needs no distance computation, handles high dimensions reasonably, and has a contamination parameter to set the threshold. Weaknesses: axis-parallel splits can miss anomalies in rotated structure (Extended Isolation Forest helps), and local anomalies in varying density regions can be missed.

Local Outlier Factor compares a point's local density (via reachability distance to its k neighbours) with its neighbours' densities, so it detects local outliers in regions of varying density, but is O(n^2) without indexing and sensitive to k. One-Class SVM learns a boundary (usually with an RBF kernel) around normal data using nu to bound the outlier fraction; it works for small to medium data, is sensitive to kernel parameters and scaling, and scales poorly. Choose by data size, density variation and whether labelled anomalies exist for evaluation.

**A93. What categories of anomaly detection approaches exist, and how do you evaluate an anomaly detector when labels are rare or absent?**

Categories: (1) statistical (z-score, modified z-score using median absolute deviation, Grubbs, Gaussian or KDE density); (2) distance and density based (kNN distance, LOF, DBSCAN noise); (3) isolation and tree based (Isolation Forest); (4) boundary based (One-Class SVM, SVDD); (5) reconstruction based (PCA reconstruction error, autoencoders); (6) probabilistic (GMM likelihood, HMM); (7) time-series specific (residuals from a forecast model, seasonal-hybrid ESD, STL decomposition plus thresholding); (8) supervised or semi-supervised when some labels exist (train on normal only, or classification with imbalance handling). Types of anomalies: point, contextual (anomalous only given context, for example time of day), collective (a sequence is unusual).

Evaluation with few labels: use precision-recall metrics (PR-AUC, precision at k, recall at fixed alert budget) rather than accuracy or ROC-AUC; inject synthetic anomalies or replay known incidents; have analysts label the top-ranked alerts and measure precision over time; track alert volume, mean time to detect, and false-positive cost. Without labels, check stability across samples and parameters, sanity-check the top anomalies manually, and tune the threshold to a tolerable alert budget. Note that contamination and threshold choice are business decisions.

**A94. Explain backpropagation. Why is it efficient and what role does the chain rule play?**

Backpropagation computes the gradient of the loss with respect to every parameter by applying the chain rule layer by layer. Forward pass: compute activations a_l = f(W_l a_(l-1) + b_l) and store them, then compute the loss. Backward pass: starting with dL/da_L, compute the error signal delta_l = dL/dz_l = (W_(l+1)^T delta_(l+1)) elementwise-multiplied with f'(z_l); then dL/dW_l = delta_l a_(l-1)^T and dL/db_l = delta_l. Gradients are then used by an optimiser. It is efficient because it is reverse-mode automatic differentiation: intermediate results are reused, so the cost of computing gradients for all parameters is a small constant multiple (about 2-3x) of one forward pass, instead of one pass per parameter as in finite differences. The cost is memory to store activations (mitigated with gradient checkpointing). For a softmax with cross-entropy output the output delta simplifies to (prediction - target). Modern frameworks build a computation graph and do this automatically; you can sanity check with numerical gradient checks.

**A95. Compare common activation functions (sigmoid, tanh, ReLU, Leaky ReLU, GELU, softmax) and where each is used.**

Sigmoid 1/(1+e^-x) outputs (0,1); saturates for large |x| so gradients vanish, and is not zero-centred; used in output layers for binary probabilities and in gates. Tanh outputs (-1,1), zero-centred, still saturates; used in RNN cells. ReLU max(0,x): cheap, non-saturating for positive inputs, sparse activations, fastest convergence in practice, but units can die (always output 0 with zero gradient) if pushed into the negative region. Leaky ReLU / PReLU / ELU give a small slope for negatives to avoid dead units. GELU (x times the Gaussian CDF of x) and SiLU/Swish are smooth and standard in transformers. Softmax turns logits into a probability distribution across mutually exclusive classes, used in the output layer with cross-entropy; compute with the log-sum-exp trick for stability. Linear activation for regression outputs. Why non-linearity at all: without it a stack of layers collapses to one linear map. Pair with proper initialisation (He for ReLU-family, Xavier/Glorot for tanh/sigmoid).

**A96. What are vanishing and exploding gradients, why do they happen, and how do you mitigate them?**

In deep or recurrent networks the gradient to early layers is a product of many Jacobians. If their typical norm is below 1 (for example sigmoid derivative at most 0.25, or small weights) the product shrinks exponentially so early layers learn very slowly (vanishing); if above 1 it grows exponentially, causing huge updates, NaN losses and unstable training (exploding). In RNNs the same recurrent weight matrix is multiplied over time steps, so its largest singular value governs this.

Mitigations: non-saturating activations (ReLU family, GELU); careful initialisation (He, Xavier, orthogonal for recurrent weights); normalisation layers (batch norm, layer norm); residual / skip connections that give gradients an identity path (ResNet, transformers); gated architectures (LSTM, GRU) with additive cell state updates; gradient clipping by norm (specifically for exploding gradients); shallower networks or truncated backpropagation through time; appropriate learning rates and optimisers such as Adam. Diagnosis: monitor

per-layer gradient norms and activation statistics; loss that stalls early suggests vanishing, sudden spikes or NaNs suggest explosion.

**A97. Compare SGD, momentum, RMSProp and Adam. Also explain learning-rate schedules and why AdamW is often preferred to Adam with L2 regularisatio...**

SGD updates w <- w - lr * g on mini-batches: noisy but often generalises well; slow in ravines and sensitive to learning rate. Momentum keeps an exponentially decaying average of gradients (velocity), damping oscillations and speeding progress along consistent directions; Nesterov momentum looks ahead. RMSProp divides the gradient by a running average of its squared magnitude, giving per-parameter adaptive step sizes. Adam combines momentum (first moment m, beta1 = 0.9) and RMSProp (second moment v, beta2 = 0.999) with bias correction: w <- w - lr * m_hat / (sqrt(v_hat) + eps). It is robust with little tuning and the default for transformers and many tasks; SGD with momentum is still common in vision where it may generalise a bit better.

Learning-rate schedules matter: warm-up (essential for transformers), step decay, cosine annealing, one-cycle, reduce on plateau. Adam with classic L2 regularisation adds lambda*w to the gradient, which then gets rescaled by the adaptive denominator, so regularisation strength differs per parameter. AdamW decouples weight decay, applying it directly to weights (w <- w - lr * lambda * w), giving proper regularisation and better generalisation. Also use batch size and learning rate scaling consistently, and gradient clipping for stability.

**A98. How does batch normalisation work, and what is the difference between its behaviour in training and inference?**

For each feature (channel) batch norm computes the mini-batch mean and variance, normalises x_hat = (x - mu) / sqrt(var + eps), then applies learnable scale gamma and shift beta so the layer can recover any representation. Benefits: stabler and faster training, allows higher learning rates, reduces sensitivity to initialisation, and has a mild regularising effect from batch noise. (The original explanation, reducing internal covariate shift, is debated; smoothing of the loss landscape is a commonly cited reason.)

Training: uses current batch statistics and updates exponential moving averages of the mean and variance. Inference: uses the stored running statistics so outputs do not depend on other samples in the batch. Pitfalls: very small batches give noisy statistics; train/eval mode mismatch (forgetting model.eval()) causes odd predictions; distribution shift makes running statistics stale; poorly suited to RNNs and variable-length sequences. In convolutional networks statistics are computed over batch and spatial dimensions per channel. It is usually placed before or after the activation (both work).

**A99. Compare batch normalisation, layer normalisation and group normalisation. Why do transformers use layer norm?**

Batch norm normalises each feature across the batch (and spatial positions); depends on batch size and batch composition. Layer norm normalises across all features of a single sample, independent of the batch, so it behaves identically in training and inference and works with batch size 1 and variable-length sequences. Group norm splits channels into groups and normalises within each group per sample, working well for convolutional models with small batches (detection, segmentation). Instance norm normalises each channel per sample (style transfer). Transformers use layer norm (often RMSNorm in recent large models) because sequence length varies, batches can be small or padded, batch statistics across tokens would leak information

and add train/inference mismatch, and per-token normalisation stabilises deep stacks. Pre-norm placement (norm before attention/MLP) trains more stably for deep transformers than post-norm.

**A100. What is dropout, how does it work at training and test time, and what other techniques regularise neural networks?**

Dropout randomly zeroes each unit with probability p during training, forcing the network not to rely on any single unit and effectively training an ensemble of thinned sub-networks, which reduces co-adaptation and overfitting. Implementation: inverted dropout scales kept activations by 1/(1-p) during training so no scaling is needed at inference, where dropout is disabled (model.eval()). Typical p is 0.1-0.5; use lower rates for convolutional layers (or spatial dropout) and for transformers. Monte Carlo dropout keeps dropout active at inference and averages multiple passes to estimate uncertainty. Dropout combined with batch norm can interact poorly (variance shift), so use one or the other carefully.

Other regularisers: L2 weight decay / L1, early stopping on validation loss, data augmentation, label smoothing, smaller models, more data, noise injection, mixup, stochastic depth, and ensembling. Diagnose with learning curves: a large gap between training and validation loss signals overfitting.

**A101. Explain how a convolutional neural network works, including convolution, pooling, parameter sharing and receptive field.**

A convolutional layer slides small learnable filters (kernels, for example 3x3xC_in) over the input, computing dot products to produce feature maps. Key ideas: local connectivity (each output depends on a small patch), parameter sharing (the same filter is applied everywhere, giving translation equivariance and far fewer parameters than a fully connected layer), and hierarchical features (early layers detect edges, deeper layers detect textures and objects). Hyperparameters: kernel size, stride, padding, number of filters, dilation. Output size = floor((W - K + 2P) / S) + 1. Parameters per layer = (K*K*C_in + 1) * C_out. Pooling (max or average) downsamples, reducing computation and adding small translation invariance; strided convolutions can replace it; global average pooling before the classifier replaces large dense layers. Receptive field grows with depth, stride and dilation. Modern practice: residual connections (ResNet), batch norm, 1x1 convolutions for channel mixing, depthwise-separable convolutions for efficiency. CNNs also apply to 1D signals such as time series and text.

**A102. Explain recurrent neural networks and backpropagation through time. What are their limitations?**

An RNN processes a sequence step by step, maintaining a hidden state h_t = f(W_h h_(t-1) + W_x x_t + b), with the same weights shared across time steps; outputs can be produced at every step or at the end. Training uses backpropagation through time (BPTT): unroll the network across the sequence, compute the loss, and backpropagate through all steps, summing gradients for the shared weights. Truncated BPTT limits the unrolled window to save memory and stabilise training.

Limitations: vanishing and exploding gradients make learning long-range dependencies hard (gradient involves repeated multiplication by the recurrent Jacobian); sequential computation prevents parallelism across time steps; fixed-size hidden state is a bottleneck for long inputs (especially in encoder-decoder models without attention); sensitivity to initialisation. Remedies: LSTM/GRU, gradient clipping, attention, and ultimately transformers. Bidirectional RNNs use both past and future context when the full sequence is available.

**A103. Describe the LSTM cell and its gates. Why does it mitigate vanishing gradients, and how does a GRU differ?**

An LSTM keeps a cell state c_t alongside the hidden state h_t. Gates (sigmoid outputs in 0-1) computed from [h_(t-1), x_t]: forget gate f_t decides how much of c_(t-1) to keep; input gate i_t decides how much of the new candidate g_t (tanh) to write; output gate o_t controls how much of the cell is exposed. Updates: c_t = f_t * c_(t-1) + i_t * g_t (elementwise), h_t = o_t * tanh(c_t). Because c_t is updated additively, the gradient dc_t/dc_(t-1) is just f_t (no repeated multiplication by a weight matrix and squashing), so when the forget gate is near 1 the error can flow across many steps (constant error carousel). A common trick is initialising the forget bias to 1. It still can suffer when gates saturate and can explode, so clipping is used.

GRU merges cell and hidden state and uses two gates: update z_t (blend between old and new state) and reset r_t (how much past to use when computing the candidate). It has fewer parameters, trains faster, and often performs comparably; LSTM may be better on tasks needing finer memory control. Both are largely superseded by transformers for long text, but remain useful for small data, streaming and low latency forecasting.

**A104. Explain the attention mechanism, including scaled dot-product attention and why the scaling factor is used.**

Attention lets a model compute, for each query position, a weighted average of values, where weights reflect relevance of keys to the query. For queries Q, keys K and values V: Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k)) V. Q K^T gives similarity scores between every query and key; softmax converts them to weights summing to 1; multiplying by V mixes the value vectors. Scaling by sqrt(d_k) is needed because the dot product of two random vectors with unit variance entries has variance d_k, so for large d_k scores grow large in magnitude, pushing softmax into saturated regions with tiny gradients; dividing normalises variance to about 1. Masks set disallowed scores to minus infinity (padding masks; causal masks in decoders so a token cannot see the future). Originally (Bahdanau, Luong) attention was added to encoder-decoder RNNs to avoid squeezing the whole source into one vector; self-attention applies it within a sequence so each token can directly attend to every other token. Cost is O(n^2) in sequence length for time and memory, motivating efficient variants.

**A105. Describe the transformer architecture: multi-head attention, positional encoding, residual connections, and the difference between encoder-o...**

A transformer block has multi-head self-attention followed by a position-wise feed-forward network (two linear layers with an activation), each wrapped with a residual connection and layer normalisation. Multi-head attention projects inputs into h different Q,K,V subspaces, runs attention in parallel and concatenates the results, letting heads specialise in different relations (syntax, position, coreference). Because attention is permutation invariant, positional information is injected: fixed sinusoidal encodings, learned embeddings, relative position schemes, or rotary embeddings. Residual connections and layer norm allow stacking dozens of layers with stable gradients.

Variants: encoder-only (BERT) uses bidirectional attention and is pretrained with masked language modelling; suited to classification, tagging and embeddings. Decoder-only (GPT family) uses causal masking and next-token prediction; suited to generation. Encoder-decoder (original transformer, T5) uses an encoder for the source and a decoder with cross-attention to it; suited to translation and summarisation. Advantages over

RNNs: parallel training, direct long-range connections, excellent scaling. Costs: quadratic attention complexity and large data and compute needs.

### A106. What is TF-IDF, how is it computed, and what are its limitations?

TF-IDF weights a term in a document by how frequent it is in that document (term frequency) and how rare it is across the corpus (inverse document frequency). Common form: tfidf(t,d) = tf(t,d) * log(N / df(t)), with smoothing variants such as log((1+N)/(1+df)) + 1 and L2 normalisation of each document vector (scikit-learn default). Common words like "the" get low weight; distinctive words get high weight. Used for search ranking, keyword extraction, document similarity (cosine) and as sparse features for linear classifiers.

Limitations: bag-of-words ignores word order and context; no semantic similarity (car and automobile are unrelated dimensions); very high-dimensional and sparse vocabulary; sensitive to preprocessing (tokenisation, stop words, n-grams, min_df/max_df); vocabulary must be fixed from training data (out-of-vocabulary words ignored); document length effects handled only through normalisation. BM25 is a stronger ranking function with term frequency saturation and length normalisation. Embeddings capture semantics but TF-IDF is still a strong, interpretable baseline.

### A107. Compare static word embeddings (word2vec, GloVe, fastText) with contextual embeddings (BERT-style). How would you use each?

Word2vec learns dense vectors by predicting context words (skip-gram) or a word from its context (CBOW), with negative sampling; GloVe factorises a global co-occurrence matrix; fastText adds character n-grams so it handles rare words and misspellings. These are static: one vector per word regardless of context, so "bank" in river bank and bank account share a vector. Vector arithmetic captures analogies and similarity by cosine. Contextual models (ELMo, BERT, sentence transformers) produce vectors that depend on the full sentence, handle polysemy, and give state-of-the-art performance after fine-tuning, at higher compute cost.

Usage: static embeddings averaged or TF-IDF weighted for lightweight document features, item2vec for recommender item embeddings (treat sessions as sentences), initialising small models. Contextual models for classification, semantic search (sentence embeddings plus approximate nearest neighbour index), clustering, NER and question answering. Cautions: embeddings reflect biases of their training data; domain mismatch may require fine-tuning; out-of-vocabulary handling differs (subword tokenisation in modern models).

### A108. Outline a robust approach to a text classification problem, from preprocessing to evaluation, including class imbalance.

1) Define labels and metric (macro-F1, PR-AUC, recall at a precision target) based on business cost. 2) Split data before any fitting, grouping by author, conversation or time to prevent leakage; remove duplicates. 3) Clean lightly: lowercasing, normalising URLs and numbers, handling emojis; avoid over-aggressive stop word or stemming for transformer models. 4) Baseline: TF-IDF (word 1-2 grams plus char n-grams) with logistic regression or linear SVM, tuned by cross-validation. 5) Stronger: fine-tune a pretrained transformer (small models such as DistilBERT first), or train a classifier on frozen sentence embeddings. 6) Imbalance: class weights, resampling, threshold tuning on a validation set, focal loss, and evaluate with precision-recall not accuracy. 7) Evaluation: confusion matrix, per-class metrics, error analysis on misclassified examples, calibration, performance across slices (language, length, source). 8) Interpretability: top weighted terms, LIME/SHAP, attention is not an explanation. 9) Deployment: version tokenisers, monitor input drift and label feedback, plan retraining.

**A109. What is stationarity in time series, why does it matter, and how do you test for and achieve it?**

A series is (weakly) stationary if its mean and variance are constant over time and the autocovariance depends only on the lag. Many classical models (ARMA) assume stationarity because it makes statistical relationships stable and estimable; non-stationary series produce spurious regressions and unreliable forecasts. Detect via plots (trend, changing variance), rolling mean/variance, ACF decaying very slowly, and tests: Augmented Dickey-Fuller (null: unit root, so non-stationary; a small p-value suggests stationarity) and KPSS (null: stationary). Use both: if ADF rejects and KPSS does not, stationary; conflicting results suggest trend-stationarity or a difference-stationary process. Achieve it by differencing (first difference removes a stochastic trend; seasonal differencing at lag s removes seasonality), detrending, variance-stabilising transforms (log, Box-Cox), and removing seasonality via decomposition. Avoid over-differencing, which introduces negative autocorrelation and inflates variance. Tree and neural models do not strictly require stationarity but often benefit from differenced or detrended targets.

**A110. Explain the ARIMA(p,d,q) model and how you select p, d and q using ACF and PACF and information criteria.**

ARIMA combines autoregression of order p (y_t depends on its last p values), differencing d times to achieve stationarity, and a moving-average component of order q (y_t depends on the last q forecast errors). Written on the differenced series: y'_t = c + phi_1 y'_(t-1) + ... + phi_p y'_(t-p) + e_t + theta_1 e_(t-1) + ... + theta_q e_(t-q).

Selection: choose d with unit-root tests and plots (usually 0, 1 or 2). On the stationary series: a pure AR(p) has a PACF that cuts off after lag p and an ACF that decays; a pure MA(q) has an ACF that cuts off after lag q and a PACF that decays; ARMA shows both decaying. Then fit candidates and compare using the Akaike information criterion (corrected for small samples) or Bayesian information criterion (more parsimonious), or automated search (auto_arima). Validate: residuals should look like white noise (Ljung-Box test, residual ACF), approximately normal for prediction intervals, and the model should beat naive and seasonal-naive baselines on out-of-sample rolling-origin evaluation. Limitations: linear, univariate (ARIMAX adds exogenous regressors), single seasonality, needs regular spacing, and long-horizon forecasts revert to the mean or trend.

**A111. What is SARIMA and how do you handle seasonality in a time series? Compare options for multiple seasonalities.**

SARIMA(p,d,q)(P,D,Q)_s adds seasonal AR, differencing and MA terms at lag s (for example 12 for monthly, 7 for daily-with-weekly-pattern). Seasonal differencing of order D subtracts the value one season ago. Identify s from the ACF (spikes at multiples of s), decomposition or periodogram. Alternatives for seasonality: classical or STL decomposition (additive versus multiplicative; multiplicative when seasonal amplitude grows with level, or log-transform), Holt-Winters exponential smoothing, Fourier terms as regressors with ARIMA (dynamic harmonic regression; handles long or multiple periods where SARIMA with large s is impractical), dummy variables, Prophet, TBATS, and tree/neural models with calendar features (hour, day-of-week, holiday flags) and lags.

Multiple seasonalities (for example hourly load with daily, weekly and annual cycles): SARIMA supports only one seasonal period, so use Fourier terms, TBATS, MSTL decomposition, Prophet, or gradient boosting/neural networks with calendar features. Always check holidays and special events and include them as regressors.

**A112. How does Facebook/Meta Prophet model a time series, and what are its strengths and weaknesses?**

Prophet is an additive regression model: y(t) = g(t) + s(t) + h(t) + e_t. g(t) is a piecewise linear (or logistic with capacity) trend with automatically detected changepoints regularised by a sparse prior; s(t) is seasonality modelled with Fourier series (yearly, weekly, daily, or custom); h(t) are holiday and event effects with windows; e_t is noise. It is fit with Bayesian inference (Stan) and supports extra regressors, multiplicative seasonality and uncertainty intervals.

Strengths: handles missing values and outliers reasonably, interpretable components, easy to add holidays, domain knowledge via changepoint and seasonality priors, fast to get a decent forecast for business series with strong seasonality and several seasons of data. Weaknesses: no autoregressive dynamics (does not use recent errors, so short-term accuracy can lag ARIMA or boosted models), uncertainty intervals cover only trend uncertainty by default, trend extrapolation can be unrealistic, performance on irregular or high-frequency series is not great, and in benchmarks it often does not beat well-tuned baselines. Always compare with seasonal naive and use cross-validation (the built-in cross_validation function does rolling origin evaluation).

**A113. How should forecasting models be validated, and which error metrics are appropriate (MAE, RMSE, MAPE, sMAPE, MASE)?**

Never use random k-fold cross-validation on temporal data because it leaks the future into training. Use time-ordered splits: a hold-out final period, and rolling-origin (walk-forward, expanding or sliding window) evaluation with several origins, with the forecast horizon matching the business use (for example 24 hours or 4 weeks ahead). Report metrics by horizon since error grows with it. Keep a gap if features use lags that would otherwise leak. Fit scalers and feature selection only on the training window, and ensure exogenous features (weather, price) are known at forecast time (use forecasts, not actuals, in backtests). Always compare with naive and seasonal naive baselines.

Metrics: MAE (robust, scale dependent), RMSE (penalises large errors, scale dependent), MAPE (percentage; undefined at zero, asymmetric, blows up for low volumes), sMAPE (symmetric attempt, still flawed), WAPE (sum of absolute errors divided by sum of actuals, good for aggregating across series), MASE (scaled by in-sample naive error; scale free and comparable across series, below 1 beats naive). For probabilistic forecasts use pinball loss / CRPS and interval coverage. For intermittent demand use MASE or RMSSE rather than MAPE. Pick the metric aligned to costs (for example stock-outs versus overstock are asymmetric, use quantile forecasts).

**A114. When would you use machine learning models such as gradient boosting or deep learning for forecasting rather than ARIMA or exponential smoot...**

Use ML when you have many related series (global models learn across series, helping short or sparse ones), rich exogenous drivers (price, promotions, weather, events), non-linear effects and interactions, or need to include hierarchy and categorical attributes. Statistical models excel for few series, short histories, strong interpretability needs, and as benchmarks; the M4/M5 competitions showed combinations and boosted/global models do very well.

Feature construction for gradient boosting: lag features (t-1, t-7, t-364, aligned to the forecast horizon so they are available at prediction time), rolling statistics (mean, std, min, max over windows, shifted to avoid including the current target), calendar features (hour, day of week, month, holiday flags, days to holiday), price and promotion features, static attributes (store, category), and target encodings fit without leakage.

Strategies for multi-step: recursive (feed predictions back, accumulates error), direct (separate model per horizon), or multi-output. Validate with rolling origin. Deep models (N-BEATS, Temporal Fusion Transformer, DeepAR) provide probabilistic and multi-horizon outputs but need more data and tuning. Trees cannot extrapolate trends, so detrend or difference the target.

**A115. Explain user-based and item-based collaborative filtering. What are the trade-offs?**

Collaborative filtering predicts a user's preference from the behaviour of similar users or items, without needing content features. User-based: find users similar to the target (cosine or Pearson correlation over co-rated items, adjusted for user mean rating), then predict a rating as a similarity-weighted average of neighbours' ratings for the item. Item-based: find items similar to the ones the user has liked (similarity computed from the item's rating column) and predict from the user's ratings on those neighbours. Trade-offs: item-based is usually more scalable and stable when there are far more users than items, since item similarities change slowly and can be precomputed, and recommendations are easier to explain ("because you watched X"); user-based can give more serendipity but similarities are volatile and expensive to maintain. Both suffer from sparsity, cold start, popularity bias, and O(n^2) similarity computation (use sparse matrices, approximate nearest neighbours, shrinkage on low-overlap pairs). Implicit data (views, clicks) should use appropriate similarity such as cosine or Jaccard.

**A116. Explain matrix factorisation for recommendations, including ALS versus SGD and how it handles implicit feedback.**

The user-item matrix R is approximated as P Q^T where each user u has a latent vector p_u and each item i a latent vector q_i of dimension k, predicting r_ui = mu + b_u + b_i + p_u . q_i (biases for global mean, user and item). Minimise squared error on observed entries plus L2 regularisation: sum over observed (r_ui - p_u.q_i)^2 + lambda(||p_u||^2 + ||q_i||^2). Optimisation: SGD updates for each observed rating (simple, flexible, easy to add features); Alternating Least Squares fixes one factor matrix and solves a closed-form regularised least squares for the other, alternating; ALS parallelises well and handles dense implicit-feedback objectives efficiently.

Implicit feedback (clicks, plays, purchases) has no negatives. Hu-Koren-Volinsky ALS treats the preference p_ui = 1 if interaction occurred with confidence c_ui = 1 + alpha * count, and weights all user-item pairs including unobserved zeros with low confidence. Alternatives: Bayesian Personalised Ranking optimises pairwise ranking between observed and sampled unobserved items; negative sampling. Choose k and lambda by validation on ranking metrics. Extensions: factorisation machines, neural collaborative filtering and two-tower models, which allow side features and help cold start.

**A117. What is the cold start problem in recommender systems and what strategies address it?**

Cold start occurs when there is not enough interaction data: new users, new items, or a whole new system. Collaborative filtering cannot embed users or items with no history. Strategies: for new users, ask onboarding preferences (pick genres or favourites), use contextual signals (device, location, referral, time), show popular or trending items (possibly by segment), then use bandits to explore and learn quickly, and move to personalisation as signals accumulate. For new items, use content-based features (metadata, text and image embeddings, audio features) to map the item into the same latent space, apply item-attribute based similarity, give exposure through exploration (epsilon-greedy, Thompson sampling, boosted slot in carousels), use creator or catalogue priors, and early engagement signals. Modelling approaches: hybrid models mixing content and collaborative signals, two-tower models whose item tower uses attributes, factorisation

machines, meta-learning, and cross-domain transfer. Evaluation should measure cold-start segments separately (for example metrics for users with fewer than 5 interactions), because aggregate metrics hide the problem.

**A118. How do you evaluate a recommender system offline and online? Cover ranking metrics, beyond-accuracy metrics and pitfalls.**

Offline: split interactions by time (not randomly) so you do not train on the future; per user hold out later interactions. For rating prediction use RMSE/MAE, but this correlates poorly with business impact. For top-N ranking use precision@k, recall@k, hit rate, MAP, MRR and NDCG@k (discounted gain rewarding relevant items ranked higher). Evaluate against all candidate items or sampled negatives carefully, because sampled metrics can distort model rankings. Beyond accuracy: coverage (share of catalogue recommended), diversity (intra-list dissimilarity), novelty, serendipity, popularity bias (average popularity of recommended items), fairness across creators or user groups, and calibration to user taste distribution.

Pitfalls: offline logs are biased by the previous recommender (exposure/selection bias; missing not at random), so offline gains may not transfer; use inverse propensity scoring or counterfactual estimators when logged propensities exist. Online: A/B test with the north-star metric (watch time, retention, conversion, revenue) plus guardrails (latency, diversity, complaints), check novelty effects and long-term effects with holdouts, and use interleaving for faster comparisons of rankers.

**A119. What is the fundamental problem of causal inference? Explain confounding, propensity scores and how they are used to estimate treatment effe...**

For each unit we observe only one potential outcome (treated Y(1) or untreated Y(0)), never both, so individual effects are unobservable; we estimate averages (ATE, ATT, conditional average treatment effect). Randomised experiments solve this by making treatment independent of potential outcomes. In observational data, confounders affect both treatment and outcome (for example sicker patients get treatment), so a naive comparison is biased.

Identification assumptions: unconfoundedness (conditional independence given covariates X), overlap/positivity ($0 < P(T=1|X) < 1$), and consistency/SUTVA (no interference between units). The propensity score $e(X) = P(T=1|X)$ is a balancing score: conditioning on it balances covariates. Methods: matching on propensity score, stratification, inverse probability weighting (weights $1/e$ for treated and $1/(1-e)$ for controls, with trimming of extreme weights), and doubly robust estimators (augmented IPW, targeted learning) that combine an outcome model with a propensity model and are consistent if either is correct. Check covariate balance (standardised mean differences), overlap plots, and run sensitivity analysis for hidden confounding (E-values, Rosenbaum bounds). Do not condition on colliders or mediators. Unobserved confounding needs designs like instrumental variables, regression discontinuity or difference-in-differences.

**A120. Explain difference-in-differences, instrumental variables and regression discontinuity. When is each appropriate and what are their key assu...**

Difference-in-differences compares the before/after change in a treated group with the before/after change in a control group, removing time-invariant group differences and common time shocks. Key assumption: parallel trends (absent treatment, groups would have moved in parallel); check with pre-period trends and placebo tests; staggered adoption needs modern estimators (Callaway-Sant'Anna). Use for policy or feature rollouts to some regions or stores.

Instrumental variables: an instrument Z affects treatment, affects the outcome only through treatment (exclusion restriction), and is independent of unobserved confounders. Two-stage least squares estimates the local average treatment effect for compliers; a weak instrument (low first-stage F, below about 10) gives biased estimates. Use for encouragement designs, such as random promotion nudges.

Regression discontinuity: treatment assigned by a threshold on a running variable (for example credit score cut-off). Compare units just above and below the cut-off, assuming units cannot precisely manipulate the running variable and other factors are continuous at the cutoff; estimate local effects with local linear regression and check density continuity (McCrary test) and bandwidth sensitivity. Estimates apply only near the threshold.

**A121. What is uplift modelling and how does it differ from standard response prediction? Describe the main modelling approaches and how uplift mod...**

Standard propensity-to-respond models predict P(outcome | X) and target those most likely to convert, even if they would have converted anyway. Uplift (heterogeneous treatment effect) modelling predicts tau(X) = P(Y=1 | X, treated) - P(Y=1 | X, control), i.e. the incremental effect, to target persuadables and avoid sure things, lost causes and sleeping dogs (people harmed by contact). It requires randomised (or well-controlled) treatment/control data.

Approaches: two-model (T-learner) fit separate models on treated and control and subtract predictions (simple, but errors do not cancel); single model (S-learner) with treatment as a feature plus interactions; class-transformation (Jaskowski) approach with transformed target Z = Y*T + (1-Y)*(1-T) under balanced assignment, giving tau = 2P(Z=1|X) - 1; X-learner and R-learner (orthogonalised, robust to imbalance and confounding); uplift trees and causal forests that split to maximise treatment effect difference; doubly robust learners. Evaluation cannot use per-user ground truth: use uplift/Qini curves and area (AUUC, Qini coefficient) by ranking users by predicted uplift and computing cumulative incremental gains versus random targeting, uplift by decile, and calibration of predicted versus observed uplift in bins. Final decisions combine uplift with cost and margin, and a follow-up A/B test of the policy.

**A122. Explain the basics of reinforcement learning: MDPs, policies, value functions, and Q-learning. How does it differ from supervised learning?**

Reinforcement learning has an agent interacting with an environment modelled as a Markov decision process: states S, actions A, transition probabilities P(s'|s,a), reward R and discount factor gamma. The agent learns a policy pi(a|s) maximising expected discounted return sum gamma^t r_t. The value function V^pi(s) is expected return from s under pi; the action-value Q^pi(s,a) is the return after taking a in s then following pi. They satisfy Bellman equations, e.g. Q*(s,a) = E[r + gamma max_a' Q*(s',a')].

Q-learning is off-policy temporal-difference learning: Q(s,a) <- Q(s,a) + alpha [r + gamma max_a' Q(s',a') - Q(s,a)]; with epsilon-greedy exploration it converges in the tabular case. Deep Q-networks approximate Q with a neural net using experience replay and target networks for stability. Policy-gradient methods (REINFORCE, actor-critic, PPO) optimise the policy directly and handle continuous actions. Difference from supervised learning: no labelled correct actions, feedback is a delayed, scalar reward; data distribution depends on the agent's own actions; there is an exploration-exploitation trade-off and credit assignment problem. In industry, simpler contextual bandits often suffice.

**A123. What is the exploration-exploitation trade-off and how do multi-armed bandits (epsilon-greedy, UCB, Thompson sampling) address it?**

Exploitation picks the option that currently looks best; exploration tries others to learn if something is better. Too little exploration locks into a suboptimal choice; too much wastes reward. In a multi-armed bandit each arm has an unknown reward distribution and we minimise regret over time. Epsilon-greedy picks a random arm with probability epsilon, otherwise the best so far (simple, regret linear unless epsilon decays). UCB picks the arm maximising mean + confidence bonus (for example sqrt(2 ln t / n_i)), exploring uncertain arms optimistically with logarithmic regret. Thompson sampling keeps a posterior per arm (Beta for conversions), samples from each and plays the highest sample; it is simple, effective in practice, and naturally adaptive. Contextual bandits (LinUCB, contextual Thompson) condition on user and context features, used for personalised recommendations, ad selection and notifications. Compared with fixed A/B tests, bandits shift traffic to winners during the test, but they complicate inference, are sensitive to non-stationarity (use discounting) and delayed rewards.

**A124. What is data drift and concept drift, and how would you monitor a deployed machine learning model?**

Data (covariate) drift: the input distribution P(X) changes. Label drift: P(Y) changes. Concept drift: the relationship P(Y|X) changes (customer behaviour shifts, fraud tactics evolve). Training-serving skew is a related bug where features computed in production differ from training.

Monitoring layers: (1) system health: latency, throughput, error rates, resource use; (2) data quality: schema, missing values, ranges, new categories; (3) input drift: population stability index, KL/JS divergence, Kolmogorov-Smirnov test per feature, and drift in embeddings or model score distributions; (4) performance: when labels arrive (often delayed) track accuracy, AUC, calibration, business metrics, and use proxy metrics or delayed-label estimates in the meantime; (5) fairness and segment slices. Set alert thresholds, dashboards and runbooks. Responses: retrain on a schedule or on a trigger, use rolling windows, shadow-deploy challengers, canary release, and automatic rollback. Not every drift matters, so tie alerts to impact on performance to avoid alert fatigue.

**A125. Describe the main components of an MLOps workflow: reproducibility, CI/CD for models, feature stores, model registry and deployment patterns...**

MLOps applies DevOps practice to machine learning. Components: version control for code, data and models (git, DVC, dataset snapshots); reproducible environments (containers, pinned dependencies, fixed seeds); experiment tracking (MLflow, Weights & Biases) logging parameters, metrics and artifacts; automated pipelines (Airflow, Kubeflow, Dagster) for data validation, training, evaluation and packaging; CI/CD with unit tests for feature code, data validation tests (Great Expectations), model quality gates versus a baseline, and automated deployment; a model registry with stages (staging, production), lineage and approval; a feature store that serves the same feature definitions for offline training and online inference, preventing training-serving skew and enabling point-in-time correct joins; serving: batch scoring, online REST/gRPC endpoints, streaming, edge; release patterns: shadow mode, canary, blue-green, A/B tests with rollback; monitoring and retraining loops; governance (documentation, model cards, access control, bias checks). Choose batch versus real time by latency needs and cost.

**A126. Describe the two-stage architecture (retrieval then ranking) of large-scale recommender systems and what a two-tower model is.**

Catalogues of millions of items cannot be scored by a heavy model per request. Stage 1, candidate generation (retrieval): fast, high-recall methods fetch hundreds to a few thousand candidates from multiple sources: two-tower embedding models with approximate nearest neighbour search, item-to-item co-occurrence, popularity and trending lists, recent-history-based, and content-based sources. Stage 2, ranking: a richer model (gradient boosting or deep network with cross features, user context, item features, real-time signals) scores the candidates to optimise a business objective (predicted watch time, click and purchase probabilities combined). Stage 3, re-ranking: apply business rules, diversity, freshness, de-duplication, fairness and exploration.

A two-tower model has a user tower and an item tower producing embeddings in the same space; relevance is their dot product or cosine. Trained with sampled-softmax or contrastive loss using in-batch negatives (corrected for popularity sampling bias). Item embeddings are precomputed and indexed in an ANN index; only the user tower runs online. Limitation: no feature interactions between user and item beyond the dot product, hence the second-stage ranker.

**A127. Write Python code with scikit-learn that standardises a customer feature matrix, fits K-means for k from 2 to 10, selects k using the silhou...**

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

def segment_customers(df: pd.DataFrame, k_range=range(2, 11), seed=42):
    X = StandardScaler().fit_transform(df)
    scores = {}
    for k in k_range:
        km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=seed)
        labels = km.fit_predict(X)
        # sample_size speeds up silhouette on big data
        scores[k] = silhouette_score(X, labels, sample_size=min(10000, len(X)),
random_state=seed)
    best_k = max(scores, key=scores.get)
    print("silhouette by k:", {k: round(v, 3) for k, v in scores.items()})
    final = KMeans(n_clusters=best_k, init="k-means++", n_init=10, random_state=seed).fit(X)
    out = df.copy()
    out["cluster"] = final.labels_
    profile = out.groupby("cluster").agg(["mean"]).round(2)
    profile["size"] = out.groupby("cluster").size()
    print(profile)
    return out, best_k
```

Notes: scale features first because K-means uses Euclidean distance; skewed variables such as spend should be log-transformed beforehand; the chosen k should also be sanity checked for cluster size and business meaning, and stability checked by re-running with different seeds or bootstrap samples.

**A128. Implement PCA from scratch using NumPy (via SVD), returning the projected data and the explained variance ratio, and show how to project new...**

```
import numpy as np

class MyPCA:
    def __init__(self, n_components):
        self.k = n_components

    def fit(self, X):
        self.mean_ = X.mean(axis=0)
        Xc = X - self.mean_
        # economy SVD: Xc = U S Vt
        U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
        n = X.shape[0]
        var = (S ** 2) / (n - 1)
        self.components_ = Vt[: self.k]              # shape (k, d)
        self.explained_variance_ = var[: self.k]
        self.explained_variance_ratio_ = var[: self.k] / var.sum()
        return self

    def transform(self, X):
        return (X - self.mean_) @ self.components_.T

    def inverse_transform(self, Z):
        return Z @ self.components_ + self.mean_

# usage
rng = np.random.default_rng(0)
X = rng.normal(size=(200, 5)) @ rng.normal(size=(5, 5))
pca = MyPCA(2).fit(X)
Z = pca.transform(X)
print(pca.explained_variance_ratio_, Z.shape)
```

Key points: centre the data (and standardise if features are on different scales); use the training mean and components when transforming new data; SVD is more stable than forming the covariance matrix; component signs are arbitrary so compare with scikit-learn up to sign.

**A129. Write code that detects anomalies in a numeric feature table with Isolation Forest, and evaluates it against a small set of labelled anomali...**

```
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import RobustScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import average_precision_score, precision_recall_curve

def run_iforest(X: pd.DataFrame, y_true=None, contamination=0.01, seed=0):
    pipe = make_pipeline(
        RobustScaler(),
        IsolationForest(n_estimators=300, max_samples=256,
```

```
                        contamination=contamination, random_state=seed, n_jobs=-1),
    )
    pipe.fit(X)
    # score_samples: higher = more normal, so negate to get an anomaly score
    scores = -pipe.score_samples(X)
    flags = pipe.predict(X) == -1
    if y_true is not None:
        print("PR-AUC:", round(average_precision_score(y_true, scores), 3))
        p, r, t = precision_recall_curve(y_true, scores)
        # choose the threshold meeting a precision target
        ok = np.where(p[:-1] >= 0.8)[0]
        if len(ok):
            print("threshold for precision>=0.8:", t[ok[0]], "recall:", r[ok[0]])
    return scores, flags
```

Notes: the contamination setting only decides the cut-off, so rank by scores and set the threshold from the alert budget or a precision target; use PR-AUC rather than ROC-AUC with rare anomalies; for time series build features from rolling statistics or forecast residuals rather than raw values.

**A130. Write a PyTorch training loop for a small feed-forward classifier that uses batch normalisation, dropout, AdamW, a learning-rate scheduler, ...**

```
import copy
import torch
import torch.nn as nn

class Net(nn.Module):
    def __init__(self, d_in, n_classes, hidden=128, p=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, hidden), nn.BatchNorm1d(hidden), nn.ReLU(), nn.Dropout(p),
            nn.Linear(hidden, hidden), nn.BatchNorm1d(hidden), nn.ReLU(), nn.Dropout(p),
            nn.Linear(hidden, n_classes),
        )
    def forward(self, x):
        return self.net(x)

def train(model, train_dl, val_dl, epochs=100, lr=1e-3, patience=8, device="cpu"):
    model.to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-2)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.5, patience=3)
    loss_fn = nn.CrossEntropyLoss()
    best, best_state, bad = float("inf"), None, 0
    for ep in range(epochs):
        model.train()                       # enables dropout, batch statistics
        for xb, yb in train_dl:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        model.eval()                        # disables dropout, uses running BN stats
```

```
        val, n = 0.0, 0
        with torch.no_grad():
            for xb, yb in val_dl:
                xb, yb = xb.to(device), yb.to(device)
                val += loss_fn(model(xb), yb).item() * len(xb)
                n += len(xb)
        val /= n
        sched.step(val)
        if val < best - 1e-4:
            best, best_state, bad = val, copy.deepcopy(model.state_dict()), 0
        else:
            bad += 1
            if bad >= patience:
                break
    model.load_state_dict(best_state)
    return model
```

Key points: train()/eval() toggling, zero_grad before backward, no_grad in validation, restoring the best checkpoint, and weight decay decoupled via AdamW.

**A131. Write Python code to perform rolling-origin (walk-forward) validation of a SARIMAX model on a daily series with weekly seasonality, reportin...**

```
import numpy as np
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

def walk_forward(y: pd.Series, order=(1,1,1), seasonal=(1,1,1,7),
                 initial=365, horizon=14, step=14):
    errs_model, errs_naive, scale_vals = [], [], []
    for start in range(initial, len(y) - horizon + 1, step):
        train, test = y.iloc[:start], y.iloc[start:start + horizon]
        res = SARIMAX(train, order=order, seasonal_order=seasonal,
                      enforce_stationarity=False, enforce_invertibility=False
                      ).fit(disp=False)
        fc = res.forecast(horizon)
        # seasonal naive: repeat the last observed week
        naive = np.tile(train.iloc[-7:].values, int(np.ceil(horizon / 7)))[:horizon]
        errs_model.append(np.abs(test.values - fc.values))
        errs_naive.append(np.abs(test.values - naive))
        # in-sample seasonal naive error scale for MASE
        scale_vals.append(np.mean(np.abs(train.values[7:] - train.values[:-7])))
    mae = np.mean(np.concatenate(errs_model))
    mae_naive = np.mean(np.concatenate(errs_naive))
    mase = mae / np.mean(scale_vals)
    print(f"MAE={mae:.2f}  seasonal-naive MAE={mae_naive:.2f}  MASE={mase:.3f}")
    return mae, mae_naive, mase
```

Notes: each fold only sees past data, so there is no leakage; add an exog argument (known-in-advance regressors such as holidays) if needed; MASE below 1 beats the in-sample seasonal naive; for speed, refit every few folds or use warm-start parameters.

**A132. Build a text classification pipeline with TF-IDF and logistic regression in scikit-learn, tune it with cross-validation, and handle class im...**

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.metrics import classification_report

def build_and_tune(texts, labels, seed=0):
    X_tr, X_te, y_tr, y_te = train_test_split(
        texts, labels, test_size=0.2, stratify=labels, random_state=seed)
    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(lowercase=True, sublinear_tf=True,
                                  strip_accents="unicode")),
        ("clf", LogisticRegression(max_iter=2000, class_weight="balanced")),
    ])
    grid = {
        "tfidf__ngram_range": [(1, 1), (1, 2)],
        "tfidf__min_df": [2, 5],
        "tfidf__max_df": [0.9, 1.0],
        "clf__C": [0.1, 1, 10],
    }
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
    gs = GridSearchCV(pipe, grid, scoring="f1_macro", cv=cv, n_jobs=-1)
    gs.fit(X_tr, y_tr)
    print("best params:", gs.best_params_)
    print(classification_report(y_te, gs.predict(X_te)))
    return gs
```

Why a pipeline: the vectoriser is fitted only on each training fold, preventing vocabulary and IDF leakage. class_weight handles imbalance without resampling; tune the decision threshold on validation data if a specific precision or recall is needed. Inspect the largest coefficients per class for interpretability.

**A133. Implement matrix factorisation with biases for explicit ratings using SGD in NumPy, and show how to compute RMSE on a held-out set and gener...**

```
import numpy as np

class MF:
    def __init__(self, n_users, n_items, k=32, lr=0.01, reg=0.05, epochs=20, seed=0):
        rng = np.random.default_rng(seed)
        self.P = rng.normal(0, 0.1, (n_users, k))
        self.Q = rng.normal(0, 0.1, (n_items, k))
        self.bu = np.zeros(n_users); self.bi = np.zeros(n_items)
        self.lr, self.reg, self.epochs = lr, reg, epochs

    def fit(self, users, items, ratings):
        self.mu = ratings.mean()
        idx = np.arange(len(ratings))
        for _ in range(self.epochs):
            np.random.shuffle(idx)
```

```
            for t in idx:
                u, i, r = users[t], items[t], ratings[t]
                err = r - (self.mu + self.bu[u] + self.bi[i] + self.P[u] @ self.Q[i])
                self.bu[u] += self.lr * (err - self.reg * self.bu[u])
                self.bi[i] += self.lr * (err - self.reg * self.bi[i])
                pu = self.P[u].copy()
                self.P[u] += self.lr * (err * self.Q[i] - self.reg * self.P[u])
                self.Q[i] += self.lr * (err * pu - self.reg * self.Q[i])
        return self

    def predict(self, u, i):
        return self.mu + self.bu[u] + self.bi[i] + self.P[u] @ self.Q[i]

    def rmse(self, users, items, ratings):
        preds = np.array([self.predict(u, i) for u, i in zip(users, items)])
        return np.sqrt(np.mean((ratings - preds) ** 2))

    def recommend(self, u, seen, n=10):
        scores = self.mu + self.bu[u] + self.bi + self.Q @ self.P[u]
        scores[list(seen)] = -np.inf          # exclude already seen items
        return np.argsort(-scores)[:n]
```

Notes: split train/test by time; tune k, lr, reg on validation; RMSE is a weak proxy, so also check ranking metrics such as NDCG@k; for implicit feedback use ALS with confidence weights or BPR instead.

**A134. Implement scaled dot-product attention with optional causal masking in NumPy, and explain the shapes involved.**

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)    # numerical stability
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def attention(Q, K, V, causal=False, pad_mask=None):
    # Q: (batch, n_q, d_k), K: (batch, n_k, d_k), V: (batch, n_k, d_v)
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(0, 2, 1) / np.sqrt(d_k)     # (batch, n_q, n_k)
    if causal:
        n_q, n_k = scores.shape[-2:]
        mask = np.triu(np.ones((n_q, n_k), dtype=bool), k=1)
        scores = np.where(mask, -1e9, scores)             # block future positions
    if pad_mask is not None:                              # (batch, n_k) True = pad
        scores = np.where(pad_mask[:, None, :], -1e9, scores)
    weights = softmax(scores, axis=-1)                    # rows sum to 1
    return weights @ V, weights                           # (batch, n_q, d_v)

# usage
rng = np.random.default_rng(0)
Q = rng.normal(size=(2, 5, 16)); K = rng.normal(size=(2, 5, 16)); V = rng.normal(size=(2, 5,
8))
out, w = attention(Q, K, V, causal=True)
```

```
print(out.shape, w[0].round(2))
```

Explanation: scores form an n_q by n_k matrix of similarities; dividing by sqrt(d_k) avoids saturated softmax; the causal mask makes the weight matrix lower triangular; multi-head attention applies learned projections to create h sets of Q, K, V, runs this function per head and concatenates the results.

**A135. Company context: A video streaming service (millions of subscribers, a large catalogue) wants to improve its home page recommendations. A la...**

Step 1, objectives and metrics: define success as 30-day retention and hours watched per user (not clicks alone), with guardrails for diversity and catalogue coverage.

Step 2, data: implicit signals (plays, completion percentage, searches, thumbs, abandon events), content metadata (genre, cast, synopsis embeddings, trailer/visual features), context (device, time, locale), and user attributes known at signup.

Step 3, architecture: two-stage. Retrieval from several sources: two-tower embedding model with ANN search, item-to-item similarity, trending by region, continue watching, new releases. Ranking: gradient boosting or a deep ranker predicting the probability of meaningful watch (more than a threshold of completion), combining user, item, and context features. Re-ranking for diversity, freshness and business rules.

Step 4, new users: onboarding survey (pick titles or genres), use signup context and cohort-level popular lists; run a contextual bandit (Thompson sampling) over row/title candidates to learn preferences within the first sessions; transition to personalised ranking after a few interactions via a feature of interaction count so the model blends content and collaborative signals.

Step 5, new titles: item tower uses metadata and content embeddings so it has a representation before any views; add an exploration budget (guaranteed impressions to a sample of likely-interested users chosen by similarity), and update item popularity features in near real time.

Step 6, offline evaluation: time-based split; report NDCG@k and recall@k separately for cold-start users and new items; use inverse propensity weighted estimates because logged data reflect the previous recommender.

Step 7, online: A/B test with retention and watch hours as primary metrics, segments for new versus tenured users, check novelty effect with extended duration and a long-term holdout.

Step 8, operations: refresh embeddings daily, retrain ranker weekly, monitor drift and popularity bias.

**A136. Company context: A grocery and general merchandise retailer with 3 million loyalty card customers wants a customer segmentation for marketin...**

Step 1, clarify use: the segments will drive campaigns, so they need to be interpretable, stable and of workable size (not 40 micro-segments); agree with marketing on the number of segments they can action (for example 5-8).

Step 2, build features per customer over a defined window: recency, frequency, monetary value (RFM), basket size, share of spend by category (fresh, household, promotions), discount sensitivity, channel mix (store versus online), visit regularity, tenure, price tier of products purchased. Exclude customers with only one transaction or treat them as a separate group.

Step 3, preprocess: log-transform skewed spend and frequency, cap outliers, standardise, check correlations and apply PCA (retaining about 90 percent variance) or drop redundant features.

Step 4, cluster: baseline K-means with k-means++ across k=3..10; compare with a Gaussian mixture (BIC) and hierarchical Ward on a sample for a dendrogram check. For mixed demographic features use k-prototypes or Gower distance. For 3 million rows, use mini-batch K-means, or cluster a sample then assign everyone.

Step 5, validate: silhouette and Calinski-Harabasz as guides; stability using bootstrap resampling and adjusted Rand index; time stability (do customers keep the same segment next quarter?); profile each cluster by feature means versus population (index scores) and name them (bargain hunters, loyal families, occasional top-up shoppers, high-value premium).

Step 6, validate business value: check that segments differ on outcomes not used in clustering (response to past campaigns, churn); run a test of segment-tailored offers against a generic offer.

Step 7, operationalise: store the scaler and model, score customers monthly, track segment migration, and refresh the segmentation yearly or when drift appears.

**A137. Company context: An energy utility needs hourly electricity load forecasts for the next 48 hours for its region to plan generation purchases...**

Step 1, framing: target is hourly load, horizon 1-48 hours, decision cost is asymmetric (under-forecasting forces expensive spot purchases), so produce probabilistic forecasts (quantiles) as well as point forecasts. Metrics: MAPE or WAPE for stakeholders plus pinball loss and peak-hour error.

Step 2, data: historical meter-aggregated load, weather (temperature, humidity, wind, cloud cover), calendar (hour, day of week, month, public holidays, school holidays, daylight saving shifts), special events, and economic or customer-count growth. Clean outages, meter errors and missing values (impute with seasonal interpolation, flag anomalies).

Step 3, EDA: multiple seasonalities (daily, weekly, annual), a non-linear U-shaped relation between temperature and load (heating and cooling), holiday effects, trend.

Step 4, baselines: seasonal naive (same hour last week), then SARIMAX/dynamic harmonic regression with Fourier terms and temperature regressors, and Prophet.

Step 5, main model: gradient boosting (LightGBM) with features: lags (24h, 48h, 168h aligned so they exist at forecast time), rolling means, temperature and its lags and squares or heating/cooling degree hours, calendar and holiday flags, hour-by-day interactions. Train quantile models for intervals. Optionally compare with a neural model (N-BEATS, TFT) and ensemble.

Step 6, weather input handling: during training use forecasted weather vintage (or add noise to actuals) to reflect real forecast error; using actual weather in backtests overstates accuracy.

Step 7, validation: rolling-origin evaluation over at least a full year to cover all seasons, evaluate by horizon, hour, season and holiday vs non-holiday; do not use random CV.

Step 8, deployment: daily pipeline, versioned features, monitoring of error by hour, drift alarms (new solar installations or demand-response programmes change the load shape), retrain monthly, and fallback to the baseline if the weather feed fails.

**A138. Company context: A telecom operator wants to detect network anomalies (cell tower outages, degraded throughput, unusual call-drop rates) fro...**

Step 1, define anomaly types and cost: point spikes, level shifts, and collective patterns; false alerts waste engineers' time, missed outages hurt customers, so agree on an alert budget per day and detection delay target.

Step 2, handle scale and seasonality: KPIs have daily/weekly patterns and differ by cell type, so model each cell (or cluster of similar cells, using clustering on historical profiles) relative to its expected behaviour rather than using one global threshold.

Step 3, baseline detector: robust seasonal z-score, using STL or a seasonal median (same time-of-week) and median absolute deviation for scale; flag residuals beyond a threshold with persistence rules (for example 3 consecutive intervals) to reduce noise.

Step 4, richer models: global forecasting model (gradient boosting or LSTM) producing expected value and prediction intervals from lags, time and neighbour-cell features, with anomalies defined by residuals outside intervals; multivariate detection across KPIs using Isolation Forest on residual features or an autoencoder reconstruction error; use neighbour cell correlation to distinguish a local fault from a region-wide event or planned maintenance.

Step 5, reduce false positives: suppress during maintenance windows and holidays/events; group alerts into incidents (same site, same time); score severity by affected subscribers.

Step 6, labels and evaluation: use the ticketing system's confirmed incidents as weak labels; evaluate recall of incidents and precision of alerts, time-to-detect, using precision-recall metrics, and replay history. Add a feedback loop where engineers mark alerts true or false and use them to tune thresholds and train a supervised re-ranker.

Step 7, deployment: streaming computation (windowed features), model per cell cluster, monitoring of alert volumes, periodic retraining as traffic patterns change.

**A139. Company context: A retail bank has a transaction fraud dataset where only 0.1 percent of transactions are confirmed fraud, and labels arrive...**

Step 1, objective: maximise fraud losses prevented subject to a limit on customer friction (false declines) and analyst capacity, so choose metrics such as recall at a fixed review budget, precision at k, and PR-AUC, plus monetary value captured; not accuracy.

Step 2, features: transaction amount relative to customer history, merchant category, geography and velocity (number of transactions in last 1, 10 and 60 minutes), device and IP change, time since last transaction, aggregates by card, merchant and device, graph features (shared devices or accounts). Compute with point-in-time correctness to avoid leakage.

Step 3, supervised model: gradient boosting with class weights or undersampling of negatives (calibrate afterwards), time-based train/validation/test splits that leave a gap for label delay, and cost-sensitive threshold selection.

Step 4, unsupervised/semi-supervised layer for novel fraud: Isolation Forest, autoencoder reconstruction error, or per-customer behavioural profiles (distance from the customer's usual pattern) to catch patterns never labelled; use their scores as additional features and as a separate alert queue.

Step 5, label delay and drift: monitor with recent confirmed labels, use rolling retraining windows, weight recent data higher, use chargeback proxies and analyst feedback, and champion-challenger deployment.

Step 6, operations: real-time scoring under tens of milliseconds, rules for hard blocks, tiered actions (approve, step-up authentication, review, decline), explainability (SHAP reason codes) for analysts and regulators, fairness checks, and feedback loops.

Step 7, evaluation online: shadow mode first, then limited rollout with holdout to measure incremental loss prevented.

**A140. Company context: An e-commerce retailer sends discount coupons by email and currently targets customers with the highest predicted purchase ...**

Step 1, reframe: the quantity that matters is incremental purchase probability due to the coupon (uplift), not purchase probability. Segments: persuadables (target), sure things (waste of margin), lost causes, and sleeping dogs (coupon reduces purchase).

Step 2, data: need randomised data. Run an experiment with a random treatment group (coupon) and control (no coupon) of sufficient size (power for subgroup effects; often 10-20 percent held out), with pre-treatment features: purchase history, recency, category affinity, past coupon use, engagement.

Step 3, modelling: baseline T-learner (two gradient boosting models, treated and control) and compare with an X-learner or causal forest and the class-transformation method; use cross-fitting and regularisation because uplift signals are weak and noisy.

Step 4, evaluation on a held-out randomised set: Qini and uplift curves, AUUC, uplift by predicted decile (observed difference in conversion between treated and control within each decile), compare with the propensity model ranking and random targeting.

Step 5, decision: expected profit of treating customer i = uplift_i * margin - coupon cost * P(redemption | treated) ; treat only if positive; choose a budget-constrained cutoff along the Qini curve.

Step 6, validation: run a second live test: policy group (targeted by uplift) versus business-as-usual (targeted by propensity) versus no coupon; measure incremental revenue and profit per customer.

Step 7, maintain: continue a small random holdout to retrain, monitor for drift in treatment effects, and guard against fatigue and cannibalisation.

**A141. Company context: A streaming service launched a new "autoplay next episode" feature to all users in one country first, without randomisation...**

Step 1, problem: no randomisation, so compare against a counterfactual. Naive pre/post comparison confounds the feature with seasonality and trends; naive treated versus untreated country comparison confounds country differences.

Step 2, design: difference-in-differences. Collect weekly watch time per user (or per active user) for both countries for a long pre-period and post-period. Effect = (treated post - treated pre) - (control post - control pre).

Step 3, check assumptions: plot pre-period trends (parallel trends); run an event-study regression with leads and lags and check that lead coefficients are near zero; placebo tests with fake launch dates; check no other simultaneous changes (price changes, big releases, holidays) in either country; consider spillovers.

Step 4, improve credibility: if parallel trends are doubtful, use synthetic control combining several comparison countries/regions to match the treated country's pre-trend, or covariate-adjusted DiD with user-level data and fixed effects; cluster standard errors at the country or region level (few clusters means use randomisation inference or wild bootstrap).

Step 5, user-level analysis if some users in the treated country opted out: use an instrumental variable approach or propensity-weighting with caution.

Step 6, interpret and report: effect size with confidence interval, heterogeneity (new versus long-tenured users), and also guardrails such as session length, retention, and complaints; note limitation that estimate applies to this country and period.

Step 7, recommendation: run a randomised holdout (for example 5 percent of users without feature) in subsequent launches to measure effect cleanly.

**A142. Company context: A retailer with 2,000 stores and 50,000 SKUs needs weekly demand forecasts per store-SKU for replenishment. Many items sell...**

Step 1, framing: decisions are order quantities, so cost of stock-outs differs from overstock; produce quantile forecasts (for example the 80th-95th percentile depending on service level) not only means. Evaluate with WAPE/RMSSE for accuracy and pinball loss for quantiles, plus business KPIs (fill rate, waste).

Step 2, segment the series by demand pattern (smooth, erratic, intermittent, lumpy using ADI and CV^2) because the best methods differ.

Step 3, model strategy: not 100 million separate models; train global models (LightGBM with Tweedie or Poisson objective, or a deep model such as DeepAR/TFT) across all series with features: lags and rolling statistics, price, promotion flags and depth, holidays, store attributes, product hierarchy (category, brand), weather, stock availability (censored demand when out of stock must be handled: mask or impute). For very intermittent items use Croston/TSB as benchmark.

Step 4, new products: no history, so use attribute-based models (category, price, brand, similar launched items as analogues), early sales ramp features after a few weeks, and hierarchical shrinkage towards category average.

Step 5, promotions: model uplift using promotion features, cannibalisation among similar items and halo/pull-forward effects; keep a clean calendar.

Step 6, hierarchy: reconcile store-SKU, store, region and total forecasts using MinT or bottom-up for coherent planning.

Step 7, validation: rolling-origin backtests over multiple seasons including promotions and holiday periods; compare to seasonal naive and last-year-same-week.

Step 8, production: weekly batch pipeline, feature store, monitoring of bias (systematic over or under-forecasting) by category, human override workflow for planners, and retraining cadence.

**A143. Company context: A subscription software company deployed a churn prediction model six months ago with an AUC of 0.84. The retention team re...**

Step 1, confirm the measurement: check that evaluation is computed correctly (same label definition, same observation window, enough recent labels; confidence intervals for AUC), and that there is no label delay artefact (recent customers have not had time to churn).

Step 2, locate the cause. Data pipeline: check schema, null rates, changed units, a broken upstream table, or training-serving skew (features calculated differently online). Input drift: compute PSI or KS statistics per feature versus the training period; look at score distribution shifts. Concept drift: check whether the relationship between features and churn changed, for example a price increase, new competitor, new product plan, or a change in cancellation policy. Population change: a new acquisition channel or a new customer

segment not represented in training. Feedback loop: the retention team's interventions on high-risk customers change outcomes, lowering the apparent performance (treatment contamination).

Step 3, slice analysis: compare performance by plan, region, tenure and channel to find where the degradation concentrates.

Step 4, remediation: fix pipeline bugs first; retrain on recent data with a rolling window, add features that capture new behaviours, recalibrate probabilities, consider modelling with time-based validation to estimate decay; for feedback loops keep a random holdout without intervention.

Step 5, prevention: automated drift monitors, scheduled retraining, champion-challenger evaluation, alert thresholds tied to business metrics, and documentation of expected performance decay.

**A144. Company context: A software company receives 20,000 customer support tickets per week and wants to route each ticket to the correct team (12...**

Step 1, label quality: audit a sample of labels with agreement analysis between annotators; merge or redefine ambiguous categories; use confident learning techniques to find mislabelled examples; write labelling guidelines.

Step 2, split carefully: split by time and deduplicate near-identical tickets (templated emails) so test performance is realistic; stratify by category.

Step 3, baseline: TF-IDF (word and character n-grams) with logistic regression, class weights for rare classes; report macro-F1, per-class precision and recall, and confusion matrix. This often gets far already.

Step 4, stronger model: fine-tune a small pretrained transformer or train a classifier on sentence embeddings (cheap and strong with 3,000 labels); use augmentation for rare classes; consider hierarchical classification if categories form a tree.

Step 5, use unlabelled data: pseudo-labelling with high-confidence predictions, active learning (have agents label the most uncertain or most diverse tickets), and clustering embeddings to discover new categories.

Step 6, deployment logic: output calibrated confidence; auto-route when confidence is high, send to human triage when low; log corrections as new labels for retraining.

Step 7, evaluate business impact: routing accuracy, time-to-resolution, reassignments per ticket; run an A/B test. Monitor category drift (new product launches create new ticket types) and retrain regularly.

**A145. Company context: A manufacturer has vibration, temperature and pressure sensors on 500 industrial pumps sampled every second. Failures are r...**

Step 1, define the target and economics: predict failure within a useful window (for example the next 7 days) so maintenance can be scheduled; cost of unplanned downtime versus unnecessary servicing determines the acceptable false-alarm rate.

Step 2, data engineering: align sensor data with maintenance and failure logs; clean sensor dropouts; resample to windows (for example 1 minute or 1 hour); remove periods of known maintenance or startup/shutdown or label them via operating-mode features.

Step 3, feature engineering: rolling statistics (mean, std, kurtosis), spectral features from vibration (FFT band energies), trends and slopes, deviation from the pump's own baseline and from peer pumps with similar duty, time since last maintenance, operating load.

Step 4, with few failures, start unsupervised/semi-supervised: model normal behaviour (autoencoder or PCA reconstruction error, Isolation Forest, forecasting residuals per pump) and track an anomaly score or health

index over time; degradation trending (rising score) gives lead time; use survival analysis or remaining useful life models if enough run-to-failure histories exist.

Step 5, supervised layer when labels allow: gradient boosting on windows labelled as pre-failure versus normal, with splits by pump and by time (never random windows from the same pump) to avoid leakage; use event-level evaluation (was a failure caught with enough lead time, how many false alarms per pump-month).

Step 6, validation with domain experts: review flagged cases with maintenance engineers to refine labels and failure modes.

Step 7, deployment: edge or batch scoring, alert thresholds tuned to maintenance capacity, dashboards of health indices, feedback from technicians to improve labels, and drift monitoring after repairs or replacements.

**A146. Company context: A news and media platform recommends articles using implicit feedback (clicks and reading time). The team notices the recom...**

Step 1, diagnose: measure popularity bias (average popularity rank of recommended items, share of impressions on top 1 percent of items), coverage, diversity across topics and publishers, and user-level topic concentration over time. Confirm that training data are biased by past exposures (position bias, selection bias).

Step 2, better labels: replace raw clicks with engagement-quality signals (dwell time above a threshold, scroll depth, shares, return visits), and debias clicks via position features at training time that are set to a constant at serving, or inverse propensity weighting.

Step 3, popularity correction: popularity-aware negative sampling (logQ correction in sampled softmax), regularisation or re-weighting of popular items, and a re-ranking stage with diversity (maximal marginal relevance or determinantal point process), topic calibration to a user's interests, and coverage constraints.

Step 4, freshness: news value decays fast, so include item age and recency features, a time-decayed popularity candidate source, content-based embeddings so new articles can be recommended immediately, and a bandit exploration budget for new content; retrain or update embeddings frequently (hourly for item stats).

Step 5, editorial and ethical constraints: blend editorial picks, enforce publisher diversity and avoid harmful content amplification; give users controls.

Step 6, evaluation: offline NDCG plus diversity and novelty metrics, counterfactual estimates, then A/B test measuring session depth, return rate and long-term retention along with topic diversity guardrails; consider a long-term holdout to detect filter bubble effects.

**A147. Company context: A ride-hailing company wants to decide which incentive (bonus level) to offer drivers in different zones and times to impro...**

Step 1, decide the framework: if an incentive decision mainly affects immediate reward and decisions are mostly independent (the bonus level for a zone-hour affects this hour's supply), a contextual bandit is appropriate and much simpler and safer than full reinforcement learning. Use full RL only if actions meaningfully change future states (for example repositioning drivers shifts future supply across zones) and a simulator or reliable off-policy data are available.

Step 2, define the reward: incremental completed trips or reduced unfulfilled requests minus incentive cost (net margin), not raw supply. Context: zone, time of day, weather, events, demand forecast, current supply-demand ratio, driver mix.

Step 3, action space: a small discrete set of bonus levels (including none), with budget constraints.

Step 4, algorithm: contextual Thompson sampling or LinUCB with a model of response per context; ensure sufficient exploration and keep a randomised holdout of zones/times at no incentive to measure incremental effect; use discounting or sliding windows for non-stationarity.

Step 5, safety: cap exploration per zone, spend limits, rules to avoid unfair treatment across driver groups, fallback to a heuristic policy.

Step 6, offline evaluation before launch: use logged data with propensities and inverse propensity scoring or doubly robust off-policy evaluation, then run an online experiment at the zone-time level (watch for interference: drivers move between zones, so randomise on larger geo clusters or use switchback experiments).

Step 7, monitor regret, spend, supply fill rate and driver satisfaction.

**A148. Company context: A coffee chain opens a new kind of drive-through format in 12 pilot stores and wants to know its effect on sales. The store...**

Step 1, recognise selection bias: pilot stores were chosen for high potential, so comparing them to others overstates the effect.

Step 2, build a comparison group: match each pilot store to control stores using pre-period sales trajectory, store size, catchment demographics, location type, and traffic; or use synthetic control to build a weighted combination of non-pilot stores that reproduces each pilot store's pre-launch sales path.

Step 3, estimate with difference-in-differences or synthetic control: compare changes in sales for pilot stores versus matched controls from before to after launch, using weekly data, with fixed effects for store and week, covering enough pre-weeks (at least 12-52) to test parallel trends.

Step 4, check robustness: event study with leads and lags (no pre-trend), placebo launches on control stores, leave-one-out of controls, sensitivity to matching variables, effect of excluding stores with concurrent promotions or construction.

Step 5, handle spillovers/cannibalisation: nearby non-pilot stores may lose sales; include them as an outcome (net chain effect, not only pilot store effect), and consider novelty effect by looking at the effect over time.

Step 6, uncertainty: with only 12 treated units use permutation or placebo-based inference rather than standard errors alone; report ranges.

Step 7, decision: translate incremental sales to margin versus capital cost, and recommend a randomised staggered rollout for the next wave to get clean evidence.

**A149. Company context: Your team trains a deep neural network for demand prediction. The training loss decreases for 2 epochs, then suddenly jumps...**

Case 1, NaN loss: Step 1, check the data for NaN, infinities, extreme outliers or unscaled features with huge magnitude; normalise inputs and targets, log-transform skewed targets. Step 2, reduce the learning rate (often 10x) and add warm-up; add gradient clipping by norm; inspect gradient norms around the failure. Step 3, check numerically unstable operations: log of zero, division by zero, softmax/cross-entropy computed manually (use the fused logits version), exploding activations (use normalisation layers, proper initialisation). Step 4, with mixed precision check loss scaling. Step 5, reproduce on a single batch.

Case 2, no learning: Step 1, sanity checks: loss at initialisation should match the theoretical value (ln(K) for K classes); overfit a tiny subset (16-64 samples) to near zero loss; if it cannot, there is a bug (wrong labels

alignment, shuffling features and labels separately, missing zero_grad, forgetting activation, frozen parameters, wrong loss/output pairing). Step 2, verify the data pipeline: visualise samples, confirm train and validation use the same preprocessing, and the target is not constant. Step 3, check the learning rate range with an LR finder, optimiser choice, and batch size. Step 4, compare with simple baselines (linear regression, gradient boosting): if they also fail, the features may simply lack signal. Step 5, ensure the model is in train mode, dropout not excessive, and enough capacity. Step 6, track metrics per epoch to catch overfitting versus underfitting patterns.

**A150. Company context: A subscription meal-kit company forecasts weekly orders with Prophet. Orders were stable until a pandemic surge, then a per...**

Step 1, diagnose: plot residuals and error by horizon; identify structural breaks (CUSUM, Chow test, changepoint detection such as PELT) and align them with known events (surge, price increase, marketing changes). Check whether seasonality shape changed or only the level.

Step 2, immediate modelling changes: add event/intervention regressors (pandemic period dummy, step dummy after the price change, a price or price index variable as an explanatory regressor), increase changepoint flexibility and recent-data emphasis (changepoint_prior_scale, changepoint_range), or train only on the post-break window when the old regime is no longer relevant; consider down-weighting the anomalous pandemic period or treating it as missing for seasonality estimation.

Step 3, model features: build a regression/gradient boosting model with recent lags, price, promotions, marketing spend, active subscriber counts and cohort retention, so the forecast is driven by subscribers and drivers rather than extrapolated trend; a bottom-up approach: forecast active subscribers (cohort survival curves) times orders per subscriber.

Step 4, validation: rolling-origin backtests concentrated on post-break periods; compare with simple adaptive baselines (exponential smoothing, seasonal naive with level adjustment); prefer methods that adapt quickly.

Step 5, communicate uncertainty: scenario forecasts (price elasticity scenarios), wider intervals around changes, and human overrides.

Step 6, monitoring: track forecast bias and tracking signal weekly, automatic alert when bias persists for several periods, and trigger retraining or model review.

**A151. Company context: An online marketplace must serve personalised recommendations to 20 million users with a response time under 100 ms on the ...**

Step 1, latency budget: split the 100 ms between feature fetching (about 10-20 ms), candidate retrieval (about 10-20 ms), ranking (about 20-30 ms), re-ranking and response (about 10 ms).

Step 2, offline training pipeline: train two-tower/embedding models and item-to-item similarity on interaction logs daily; train the ranker (gradient boosting or deep) with features logged at serving time to avoid skew; evaluate with time-split offline metrics and register models.

Step 3, precomputation: compute item embeddings and index them in an approximate nearest neighbour service (HNSW); precompute user long-term embeddings daily and store them in a low-latency key-value store; cache popular lists and per-segment fallbacks.

Step 4, online features: stream processing (Kafka plus Flink or Spark streaming) updates session features (last viewed items, categories in the current session) in an online feature store (Redis or similar) with the same definitions as offline features.

Step 5, request flow: fetch user and session features; retrieve candidates from several sources (ANN using session-aware user vector, popular, recently viewed related items); filter (out of stock, already purchased, policy); rank the top few hundred with the ranker; apply diversity and business rules; log impressions, positions and model versions with propensities.

Step 6, reliability: timeouts with fallback to cached recommendations, autoscaling, canary releases, shadow evaluation, and feature availability checks.

Step 7, experimentation and monitoring: A/B testing platform, latency percentiles, click and conversion metrics, drift and data quality monitors, and periodic retraining. Use model optimisation (quantisation, distillation) if the ranker is too slow.

**A152. Company context: A credit scoring team trained a gradient boosting model in notebooks with features computed by ad hoc SQL. After deployment...**

Step 1, reproduce: log the exact feature vector used at serving for a sample of requests and recompute the same features offline using the training code for the same entities and timestamps; compare values to identify mismatches.

Step 2, typical causes: different feature logic in SQL versus the service (time zones, null handling, rounding, categorical encodings), point-in-time leakage in training (features computed using data available only after the decision time, such as future repayment status or end-of-month aggregates), stale batch features in production, different preprocessing (scalers or encoders refitted instead of loaded), missing values defaulting differently, and population differences (training on approved applicants only, selection bias, reject inference problem).

Step 3, fix: define each feature once in a feature store or shared library used for both batch training and online serving; build training sets with point-in-time correct joins; package preprocessing with the model (single pipeline artifact); add unit tests with golden examples and data validation tests (expected ranges, null rates, distributions).

Step 4, monitoring: compare serving feature distributions with training distributions (PSI), log predictions and later outcomes, and alert on skew.

Step 5, process: CI/CD with model registry, reproducible training from versioned data, shadow deployment before launch, and champion-challenger comparison with monitoring of business metrics.

Step 6, governance: model documentation, explainability and fairness checks required in credit decisions.

**A153. Company context: An online learning platform is considering an LSTM or transformer model to predict which students will drop out of a course...**

Step 1, frame the problem: weekly prediction per enrolled student of drop-out in the next 14 days; labels from enrolment status; evaluate with PR-AUC and recall at a fixed outreach capacity; split by time cohort and by student (a student must not appear in both train and test).

Step 2, baseline first: aggregate features per student up to the prediction date (activity in last 1, 7 and 14 days, trend of activity, days since last login, quiz average and trend, assignments missed, course progress percent, demographics, course attributes) and train logistic regression and gradient boosting. These models are strong for tabular summaries, fast, interpretable (SHAP), and often match deep models.

Step 3, sequence model: if raw sequences carry extra signal, build an LSTM/GRU or small transformer over daily activity vectors (last 60 days), concatenated with static features. Compare against the baseline via the same time-based validation; use dropout, early stopping and a modest size since the dataset may be limited.

Step 4, decision criteria: accuracy gain on PR-AUC and calibration, stability across courses, latency and maintenance cost, interpretability needed for advisers and fairness audits. Choose the sequence model only if the gain is meaningful (for example more than a few points of PR-AUC) and consistent.

Step 5, link to action: the goal is to intervene, so consider uplift modelling with randomised outreach to target students who benefit, not merely those at risk.

Step 6, deploy and monitor: weekly batch scoring, drift monitoring across semesters, fairness checks across groups, and feedback from advisers.

**A154. Company context: A mobile game company wants to personalise in-game offers (for example discounted bundles) to maximise long-term revenue an...**

Step 1, clarify the objective: long-term player value (revenue plus retention) with guardrails on churn, spend concentration (avoid exploiting vulnerable players), and fairness. Because offers affect future behaviour (fatigue, price anchoring), the decision is sequential, so RL is conceivable; but start simple.

Step 2, start with a causal baseline: randomised offer experiments (including a no-offer control) to estimate heterogeneous effects via uplift models for each offer type; select the best offer by incremental expected value net of discount cost.

Step 3, contextual bandit with exploration for the immediate reward plus constraints, with short-term proxy metrics validated against long-term outcomes; include features such as days since last offer and number of offers in past week to capture fatigue.

Step 4, move to RL only if evidence shows meaningful sequential effects: use offline RL or off-policy evaluation on logged randomised data (importance sampling, fitted Q evaluation, doubly robust estimators), conservative methods that stay near the logged policy, and a simulator validated on real data; model the state with user history embeddings.

Step 5, safety and ethics: frequency caps, spending limits, exclude minors and signs of problematic spending, constrain the policy (constrained optimisation), monitor guardrails, and keep a persistent random holdout to detect long-run harm.

Step 6, evaluate with long-horizon A/B tests and metrics such as 30-day and 90-day retention and lifetime value, compare with the heuristic and bandit policies, and be prepared to roll back.

**A155. Company context: A telecom operator clusters its subscribers using usage data (call minutes, data volume, roaming, top-up patterns) to desig...**

Step 1, diagnose: usage variables are heavily right-skewed with outliers (a few extreme data users), and K-means with unscaled raw features lets large-magnitude variables and extremes dominate, putting nearly everyone in one cluster.

Step 2, transform: log or Box-Cox transform skewed variables, then standardise; cap or winsorise extreme values or treat them as a separate heavy-user group identified by rules; convert counts to shares (for example proportion of evening usage) and ratios to capture behaviour rather than just volume.

Step 3, reduce redundancy: correlated usage metrics overweight one concept, so check correlation, apply PCA or select representative variables, or weigh variable groups.

Step 4, try alternatives: Gaussian mixture models (elliptical and unequal sizes), hierarchical Ward clustering on a sample, or two-step approach (first remove outliers with Isolation Forest or DBSCAN noise, cluster the rest,

then assign extreme users to nearest cluster or a premium cluster). Evaluate several k with silhouette, BIC and stability.

Step 5, check business sense: segment sizes should be actionable (no tiny clusters unless valuable, like roaming-heavy business travellers), and segment profiles should differ on revenue, churn or tariff sensitivity.

Step 6, operationalise: freeze transformations and cluster centres, score monthly, monitor segment migration, and test tariff offers per segment.

**A156. Company context: A streaming music service's offline evaluation shows a new matrix factorisation model improves RMSE on held-out ratings by ...**

Step 1, point out the mismatch: RMSE evaluates rating prediction on observed explicit ratings; the product ranks tracks from implicit feedback, so ranking quality and engagement matter. Observed ratings also reflect selection bias (users rate what they chose to play).

Step 2, offline redesign: use implicit outcomes (completed plays, saves, skips as negative signals), time-based splits per user, and ranking metrics (recall@k, NDCG@k, MAP) against the full candidate set or properly sampled negatives; report by user activity level, new versus established users and long-tail versus popular tracks; add coverage, diversity and novelty.

Step 3, bias correction: since logged exposures come from the old recommender, use inverse propensity scoring or doubly robust offline estimates if propensities are logged; check that gains do not come only from recommending more popular tracks.

Step 4, online experiment: randomised A/B test with primary metric such as listening time per user or 7-day retention, secondary metrics skip rate and saves, guardrails for diversity and latency; compute sample size from baseline variance, run through full weekly cycles, and check novelty effects.

Step 5, decision rule: ship only if the primary metric improves significantly without guardrail regressions; otherwise investigate segments. Consider interleaving as a faster pre-test.

Step 6, learn: keep a long-term holdout to track cumulative effects.

**A157. What is the curse of dimensionality and what are its consequences for clustering and nearest-neighbour methods? How do you deal with it?**

As dimensionality grows, data become sparse and distances concentrate: the ratio between the nearest and farthest neighbour distance approaches 1, so similarity and density lose discriminating power. Consequences: K-means and DBSCAN become unreliable because Euclidean distance is uninformative; kNN degrades; models need exponentially more data to cover the space; irrelevant features add noise; overfitting risk grows. Remedies: remove redundant or irrelevant features; reduce dimensionality (PCA, autoencoders, UMAP) before clustering; use cosine or other metrics suited to sparse high-dimensional data; subspace or spectral clustering; regularisation; learned embeddings; and domain-driven feature engineering. Always standardise and validate cluster stability, since high-dimensional clusters are often artefacts.

**A158. Which loss functions are used in deep learning for regression, classification and ranking, and how do you choose among them?**

Regression: MSE (penalises outliers heavily, assumes Gaussian noise), MAE (robust, targets the median), Huber (quadratic near zero, linear in the tails), quantile/pinball loss for prediction intervals, Poisson or negative-binomial deviance for counts. Binary classification: binary cross-entropy with a sigmoid output (use

the logits-based stable implementation). Multiclass: categorical cross-entropy with softmax; label smoothing reduces over-confidence. Imbalance: class-weighted loss or focal loss that down-weights easy examples. Multi-label: independent sigmoids with binary cross-entropy. Ranking and recommendation: pairwise losses (BPR, hinge), listwise losses (softmax cross-entropy over candidates, LambdaRank for NDCG) and contrastive/InfoNCE for embeddings. Choose the loss that matches the output distribution and business cost, remember it is a surrogate for the final metric (F1, AUC, NDCG), and check calibration if probabilities are used downstream.

**A159. How do learning curves help diagnose underfitting and overfitting in neural network training, and what sanity checks do you run first?**

Learning curves plot training and validation loss (and metric) versus epochs or training-set size. Both high and flat means underfitting (increase capacity, train longer, improve features, reduce regularisation). Training low with validation rising or far higher means overfitting (more data, augmentation, dropout, weight decay, early stopping, smaller model). Both converging with a small gap at an acceptable level is a good fit. A very noisy validation curve suggests a small validation set or too high a learning rate. A loss that never decreases points to bugs, a poor learning rate or vanishing gradients. Sanity checks before scaling up: overfit a tiny batch to near zero loss; verify the initial loss equals ln(K) for K-class softmax; check the input pipeline and label alignment; compare with a simple baseline. Modern double descent shows very large models can generalise well past interpolation, so always rely on held-out performance.

**A160. Explain exponential smoothing methods (simple, Holt, Holt-Winters, ETS) and how they relate to ARIMA.**

Simple exponential smoothing forecasts the level l_t = alpha*y_t + (1-alpha)*l_(t-1), giving geometrically decaying weights to older data; it suits series without trend or seasonality. Holt's method adds a trend (beta); Holt-Winters adds seasonality (gamma), additive or multiplicative. A damped trend (phi < 1) prevents unrealistic long-run extrapolation and often improves accuracy. The ETS framework (Error, Trend, Seasonal) casts these as state-space models fitted by maximum likelihood, giving prediction intervals and model selection by the Akaike information criterion. Relation to ARIMA: simple exponential smoothing equals ARIMA(0,1,1); Holt linear trend equals ARIMA(0,2,2); the classes overlap but are not identical, as ETS includes non-linear multiplicative forms and ARIMA covers more general autocorrelation structures. Both are strong baselines; ETS is intuitive and good for many series, ARIMA suits series with rich autocorrelation.